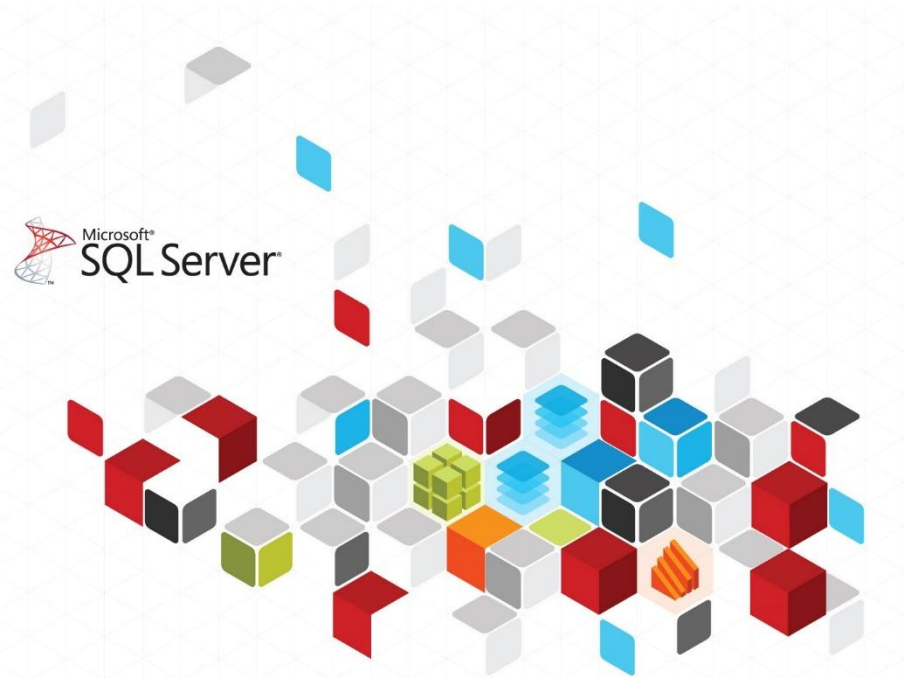
ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT

BÀI GIẢNG

CƠ SỞ DỮ LIỆU II

Tài liệu lưu hành nội bộ - dành cho sinh viên
Ngành CÔNG NGHỆ THÔNG TIN



Biên soạn:
ThS. NGUYỄN THỊ HÀ QUYÊN

Khoa Công nghệ số
Bộ môn Công nghệ Thông tin

Tháng 06/2019

CƠ SỞ DỮ LIỆU II

Mã học phần: 5505128

Mục tiêu của học phần

+ Kiến thức: giúp sinh viên nắm rõ các kiến thức thiết kế và chuẩn hóa cơ sở dữ liệu quan hệ; truy vấn dữ liệu với ngôn ngữ T-SQL; quản trị cơ sở dữ liệu với SQL Server
+ Kỹ năng: biết thiết kế và đặc tả mô hình dữ liệu quan hệ ERD; thiết kế được cơ sở dữl iệu cho một ứng dụng sử dụng hệ quản trị cơ sở dữ liệu SQL Server; vận dụng được ngôn ngữ T-SQL trong truy vấn dữ liệu
+ Thái độ: chuyên cần và chuẩn bị bài đầy đủ cho giờ lên lớp theo yêu cầu của giáo viên

Mô tả tóm tắt về học phần

Học phần này trang bị cho sinh viên các kiến thức và kỹ năng nâng cao của việc thiết kế, thực thi cơ sở dữ liệu, gồm: Thiết kế & chuẩn hóa cơ sở dữ liệu; Tối ưu hóa truy vấn dữ liệu; Tổ chức thủ tục, hàm và các trigger bảo đảm toàn vẹn dữ liệu; Giải quyết tương tranh trong môi trường đa người dùng; Quản lý transaction; Quản trị hệ thống Database Management System.

MỤC LỤC

Chương 1. TỔNG QUAN HỆ QUẢN TRỊ CSDL SQL SERVER 1
1.1. Giới thiệu về SQL Server 1
1.2. Kiến trúc và các thành phần 1
1.2.1. Các thành phần chính của SQL Server 1
1.2.1.1. Protocol Layer 2
1.2.1.2. Relational Engine 3
1.2.1.3. Storage Engine 4
1.2.2. Các dịch vụ (Services) 7
1.2.3. SQL Server Business Intelligence (BI) 9
1.3. Các phiên bản và cài đặt 10
Chương 2. THAO TÁC VỚI DỮ LIỆU TRONG SQL SERVER 2
2.1. Cơ sở dữ liệu quan hệ 2
2.1.1. Cơ sở dữ liệu (Database) 2
2.1.2. Các mô hình dữ liệu 2
2.1.2.1. Mô hình dữ liệu phân cấp và mô hình dữ liệu mạng 2
2.1.2.2. Mô hình dữ liệu quan hệ 3
2.1.2.3. Mô hình dữ liệu thực thể quan hệ 3
2.1.3. Các giai đoạn thiết kế cơ sở dữ liệu 5
2.1.4. Mô hình thực thể quan hệ (ERM) 7
2.1.4.1. Thực thể (Entity) ≡ Table 8
2.1.4.2. Thuộc tính (Attribute) ≡ Column 8
2.1.4.3. Khóa (Key) 9
2.1.4.4. Mối quan hệ (Relationship) 10
2.2. Ngôn ngữ T-SQL 10
2.2.1. Tổng quan ngôn ngữ T-SQL 10
2.2.1.1. Các kiểu dữ liệu (Data Type) 11
2.2.1.2. Toán tử (Operator) 13
2.2.1.3. Chú thích 13
2.2.1.4. Khai báo và sử dụng biến (Variable) 13
2.2.1.5. Các nhóm câu lệnh T-SQL 14
2.2.1.6. Khái niệm Schema 15
2.2.1.7. Quy tắc sử dụng tên 16
2.2.2. Ngôn ngữ định nghĩa dữ liệu (DDL) 16
2.2.2.1. Lệnh CREATE 16
2.2.2.2. Lệnh ALTER 19

2.2.2.3. Lệnh DROP 20
2.2.2.4. Lệnh TRUNCATE 20
2.2.3. Ngôn ngữ thao tác dữ liệu (DML) 21
2.2.3.1. Cập nhật dữ liệu 21
2.2.3.1.1. Lệnh INSERT 21
2.2.3.1.2. Lệnh UPDATE 22
2.2.3.1.3. Lệnh DELETE 23
2.2.3.2. Truy vấn dữ liệu 23
2.2.3.2.1. Lệnh SELECT 23
2.2.3.2.2. Phép hợp 26
2.2.3.2.3. Phép nối (JOIN) 28
2.2.3.2.4. Truy vấn con (Subquery/ Nesting Query) 34
2.3. Khung nhìn (VIEW) 36
2.4. Bảng thời gian (Temporal Table) 38
2.5. Tối ưu hóa câu truy vấn (Query Optimization) 39
2.5.1. Execution Plan 40
2.5.2. Các cơ chế truy vấn dữ liệu 40
2.5.2.1. Truy vấn dạng Scan 41
2.5.2.2. Truy vấn dạng Seek 41
2.5.3. Các kỹ thuật tối ưu hóa câu truy vấn 42
Chương 3. LẬP TRÌNH TRONG SQL SERVER 44
3.1. Khối lệnh (Statement Block) 44
3.2. Các lệnh điều khiển 44
3.2.1. Lệnh BREAK 44
3.2.2. Lệnh CONTINUE 44
3.2.3. Lệnh GOTO 44
3.3. Cấu trúc rẽ nhánh 44
3.3.1. Lệnh IF…ELSE 44
3.3.2. Lệnh CASE 45
3.4. Cấu trúc lặp (WHILE) 46
3.5. Thủ tục (Stored Procedure) 47
3.5.1. Định nghĩa 47
3.5.2. Tạo/Cập nhật Stored Procedure 47
3.5.3. Xóa Stored Procedure 48
3.5.4. Thực thi Stored Procedure 48
3.6. Hàm (FUNCTION) 49
3.6.1. Định nghĩa 49
3.6.2. Tạo/Cập nhật FUNCTION 49

3.6.3. Xóa Function 50
3.6.4. Gọi Function 50
3.6.5. Các hàm hệ thống thông dụng 50
3.7. Trigger 50
3.7.1. Định nghĩa 50
3.7.2. Tạo/Cập nhật Trigger 51
3.7.3. Xóa Trigger 52
3.7.4. Disable/Enable Trigger 53
3.8. Con trỏ (Cursor) 53
3.8.1. Cú pháp khai báo cursor 53
3.8.2. Các lệnh thông dụng cho cursor 54
3.8.3. Các biến thông dụng với cursor 55
3.8.4. Tạo cursor 55
3.9. Bảng tạm (Temporary Table) 56
Chương 4. QUẢN LÝ TRANSACTION 58
4.1. Khái niệm 58
4.1.1. Transaction 58
4.1.2. Các đặc điểm của Transaction (ACID) 58
4.1.3. Phân loại Transaction 58
4.2. Xử lý Transaction 59
4.2.1. Bắt đầu một Transaction 60
4.2.2. Đánh dấu vị trí trong Transaction 60
4.2.3. Kết thúc một Transaction 61
4.2.3.1. Lệnh COMMIT 61
4.2.3.2. Lệnh ROLLBACK 61
4.2.4. Các biến hệ thống với Transaction 61
4.2.4.1. Biến @@trancount 62
4.2.4.2. Biến @@error 62
4.2.4.3. Biến @@rowcount 62
4.3. Xử lý lỗi với TRY…CATCH 62
4.4. Ví dụ 63
4.5. Transaction lồng nhau 64
Chương 5. QUẢN LÝ TƯƠNG TRANH 65
5.1. Đặt vấn đề 65
5.2. Các vấn đề liên quan đến truy xuất đồng thời 65
5.3. Các khái niệm liên quan đến truy xuất đồng thời 68
5.3.1. Lịch biểu (Schedule) 68

5.3.1.1. Lịch biểu (Schedule) 68
5.3.1.2. Lịch biểu tuần tự (Serial Schedule) 68
5.3.1.3. Lịch biểu không tuần tự (Nonserial Schedule) 68
5.3.1.4. Lịch biểu khả tuần tự (Serializable Schedule) 68
5.3.2. Khóa (Lock) 69
5.3.2.1. Shared Lock (Read Lock) 70
5.3.2.2. Exclusive Lock (Write Lock) 70
5.3.2.3. Update Lock 70
5.3.2.4. Live Lock 70
5.3.2.5. Intent Lock 70
5.3.2.6. Schema Lock 71
5.3.2.7. Deadlock 71
5.3.2.8. Lưu ý khi sử dụng Lock 72
5.3.3. Nhãn thời gian (Timestamp) 72
5.4. Các mức cô lập dữ liệu (Isolation Level) 72
Chương 6. CÁC VẤN ĐỀ THỰC TIỄN TRONG DBMS 74
6.1. Tìm kiếm Full-text 74
6.2. Sao lưu dữ liệu (Database Backup) 76
6.2.1. Full backup 77
6.2.2. Differential Backup 78
6.2.3. Transaction log backup 78
6.2.4. Lập lịch backup dữ liệu với SQL Server Agent 78
6.3. Khôi phục dữ liệu (Restore Database) 79
6.3.1. Các chế độ khôi phục dữ liệu 79
6.3.2. Sao lưu và khôi phục 80
6.3.3. Thao tác khôi phục dữ liệu 81
6.4. Sao chép dữ liệu 82
6.5. Bảo mật dữ liệu 83
6.5.1. Login và User 83
6.5.2. Authentication 84
6.5.3. Server Role 84

CHƯƠNG 1. TỔNG QUAN HỆ QUẢN TRỊ CSDL SQL SERVER

1.1. Giới thiệu về SQL Server

SQL Server hay còn gọi là Microsoft SQL Server, viết tắt là MS SQL Server. Đây là một phần mềm được phát triển bởi Microsoft, là một hệ quản trị cơ sở dữ liệu quan hệ (Relational DataBase Management System – RDBMS) cho phép thiết kế và thực thi cơ sở dữ liệu quan hệ.

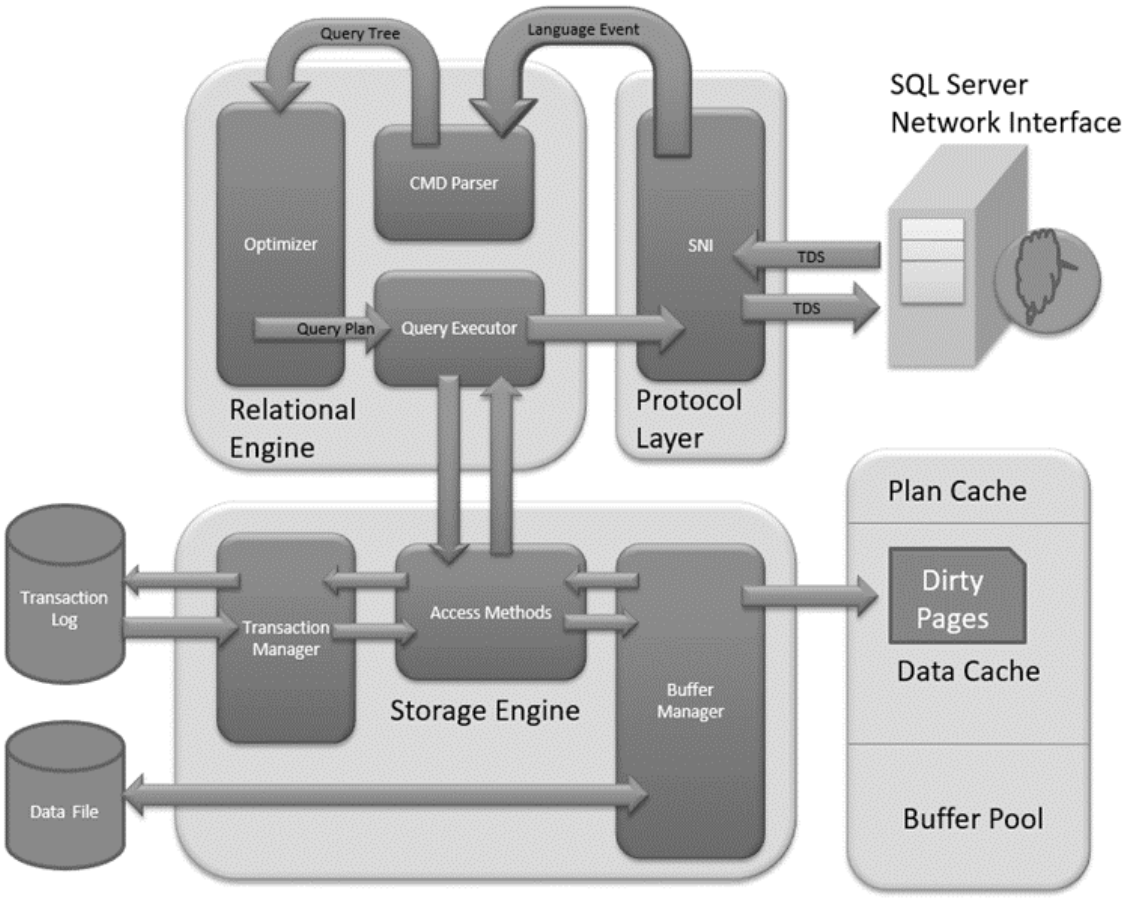
SQL Server hoạt động dựa trên mô hình kiến trúc máy khách/máy chủ (client/server) trong đó server (có chứa cơ sở dữ liệu) được chia sẻ, truy cập dựa trên ngôn ngữ T-SQL (xem 2.4).

SQL Server cung cấp đầy đủ công cụ để quản lý và thao tác dữ liệu, từ giao diện GUI cho đến việc sử dụng ngôn ngữ T-SQL.

1.2. Kiến trúc và các thành phần

1.2.1. Các thành phần chính của SQL Server

Tiến trình hoạt động của SQL Server bắt đầu từ khi ứng dụng khách gửi yêu cầu ➔ máy chủ SQL chấp nhận, xử lý và trả lời yêu cầu với dữ liệu được xử lý.



Hình 1.1: Sơ đồ kiến trúc các thành phần SQL Server

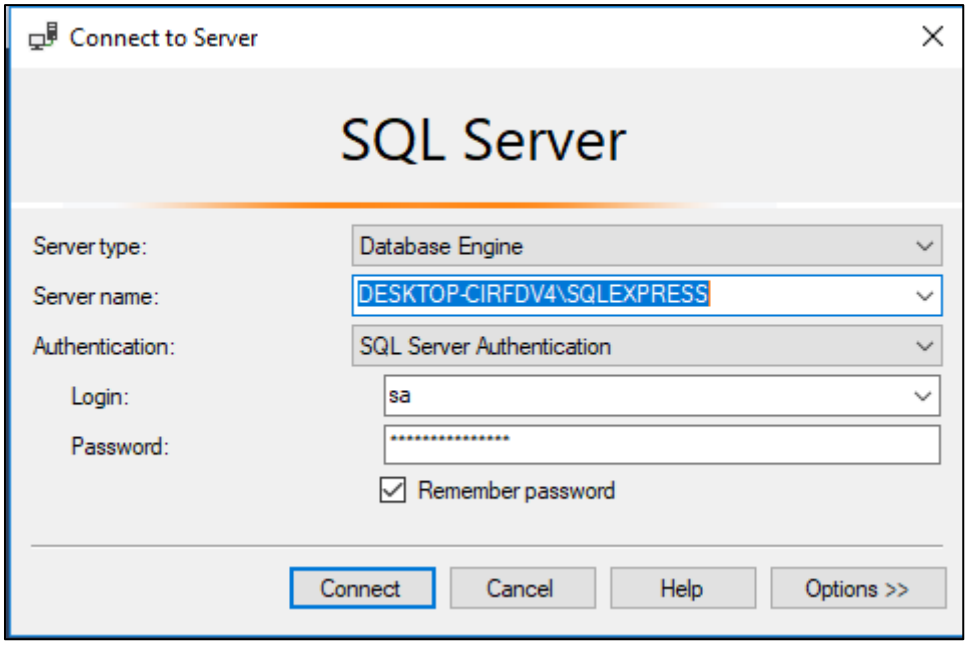
TDS (Tabular Data Stream) trong sơ đồ ở hình 1.1 là một giao thức được sử dụng để tương tác với các máy chủ cơ sở dữ liệu. Thời điểm kết nối TCP IP được thiết lập, một liên kết tương ứng đến điểm cuối TDS được tạo cho giao tiếp giữa máy khách và máy chủ.

Ba thành phần chính trong kiến trúc SQL Server ở hình 1.1 gồm:

1.2.1.1. Protocol Layer

Protocol Layer, là một thành phần trong SQL Server Network Interface (SNI), thành phần này cho phép SQL Server giao tiếp với môi trường bên ngoài, Protocol Layer của SQL Server hỗ trợ 3 kiểu kiến trúc client/server gồm:

- Shared Memory Protocol: đây là giao thức chia sẻ vùng nhớ, nghĩa là cả client và server đều chạy trên cùng một machine và đều có thể giao tiếp qua bộ nhớ chung. Giao diện khi khởi động MS SQL Server Management Studio – mục "Server name" chính là kết nối đến Local Database, "Server name" có thể nhận một trong 4 giá trị sau:
  + "."
  + "localhost"
  + "127.0.0.1"
  + "Machine\InstanceName"



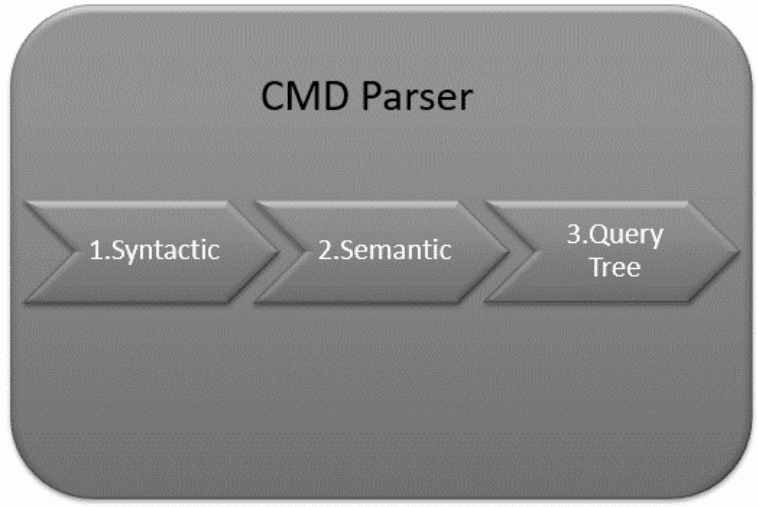
Hình 1.2: Giao diện khi khởi động MS SQL Server Management Studio

- TCP/IP Protocol: với giao thức này, client và SQL Server được cài đặt trên 2 machine riêng lẻ, mục "Server name" phải là lựa chọn "Machine\InstanceName", và lúc này SQL Server sử dụng port 1433 trong TCP/IP.
- Named Pipe Protocol: với giao thức này, client và SQL Server được kết nối với nhau thông qua mang LAN, và thông thường tùy chọn này mặc định bị disabled – chỉ được enabled bởi SQL Configuration Manager.

1.2.1.2. Relational Engine

Relational Engine hay còn được gọi là Database Engine, là bộ xử lý truy vấn – nghĩa là nó sẽ xác định chính xác những gì một truy vấn cần làm và làm thế nào nó có thể được thực hiện tốt nhất. Nó chịu trách nhiệm thực hiện các truy vấn của người dùng bằng cách yêu cầu dữ liệu từ công cụ lưu trữ và xử lý các kết quả được trả về. Relational Engine gồm 3 thành phần chính sau:

- **CMD Parser**: là trình phân tích cú pháp CMD, được hiểu là dữ liệu sau khi nhận được từ Protocol Layer ➔ sau đó được chuyển đến Relational Engine. "CMD Parser" là thành phần đầu tiên của Relational Engine nhận dữ liệu truy vấn. Công việc chính của CMD Parser là kiểm tra lỗi cú pháp và ngữ nghĩa của câu truy vấn ➔ nếu không có lỗi, CMD Parser sẽ tạo ra một cây truy vấn (Query Tree).



Hình 1.3: Các thành phần và hoạt động của CMD Parser

  + Syntactic check: giống như mọi ngôn ngữ lập trình khác, MS SQL Server cũng có bộ từ khóa (Key Word) được định nghĩa trước, ví dụ như INSERT – SELECT – DELETE – UPDATE.... Ngoài ra, SQL Server cũng có cú pháp (Grammar) riêng mà máy chủ SQL hiểu được. CMD Parser kiểm tra cú pháp, nếu đầu vào của người dùng không tuân theo các quy tắc cú pháp này, nó sẽ trả về lỗi.
  + Semantic check: thành phần này kiểm tra xem tên cột (Column name), tên bảng (Table name) được truy vấn có tồn tại trong Schema hay không. Và nếu nó tồn tại, liên kết nó với truy vấn. Điều này còn được gọi là Binding. Độ phức tạp tăng khi truy vấn người dùng chứa khung nhìn (View).
  + Query Tree: bước này tạo ra cây thực thi (Execution Tree) khác nhau mà trong đó truy vấn có thể được chạy. Lưu ý rằng, tất cả các cây thực thi khác nhau có cùng một đầu ra mong muốn.
- **Optimizer**: là trình tối ưu hóa, công việc của trình tối ưu hóa là tạo một kế hoạch thực hiện (Execution plan) cho truy vấn của người dùng. Đây là kế hoạch sẽ xác định cách truy vấn người dùng sẽ được thực hiện. Lưu ý rằng không phải tất cả các

truy vấn đều được tối ưu hóa. Tối ưu hóa được thực hiện cho các lệnh DML như SELECT – INSERT – DELETE và UPDATE. Các truy vấn như vậy được đánh dấu đầu tiên sau đó gửi đến trình tối ưu hóa. Các lệnh DDL như CREATE và ALTER không được tối ưu hóa, nhưng thay vào đó chúng được biên dịch thành một dạng bên trong (Internal form). Chi phí truy vấn được tính dựa trên các yếu tố như mức sử dụng CPU, mức sử dụng bộ nhớ và nhu cầu Đầu vào/Đầu ra (Output/Input).

- **Query Executor**: Query Executor gọi/làm việc với Access Method. Nó cung cấp một kế hoạch thực hiện cho việc tìm nạp dữ liệu cần thiết để thực thi truy vấn. Sau khi dữ liệu được nhận từ Storage Engine, kết quả sẽ được công bố cho thành phần Protocol Layer. Cuối cùng, dữ liệu được gửi đến người dùng cuối.

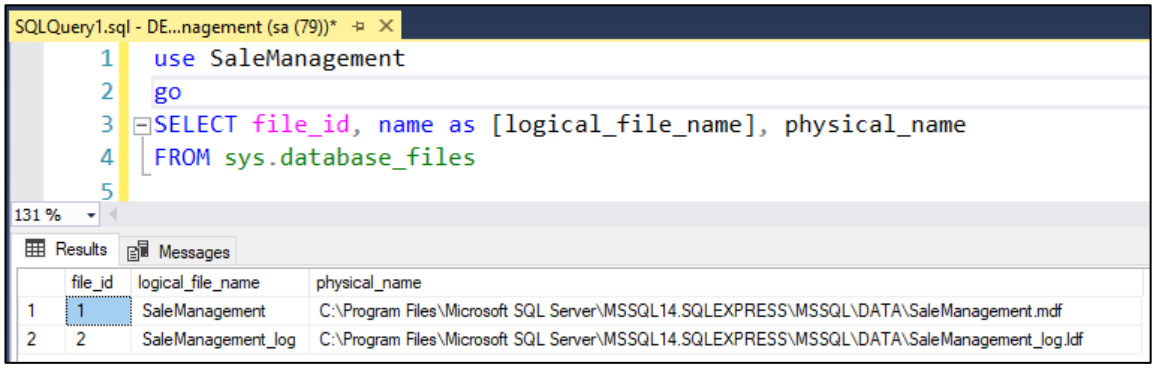
1.2.1.3. Storage Engine

Công việc của Storage Engine là lưu trữ dữ liệu trong một hệ thống lưu trữ như Disk hoặc SAN (Storage Area Network), và lấy dữ liệu khi cần.

Trước khi kể đến các thành phần trong Storage Engine, ta tìm hiểu các loại Data File và File Type trong SQL Server.

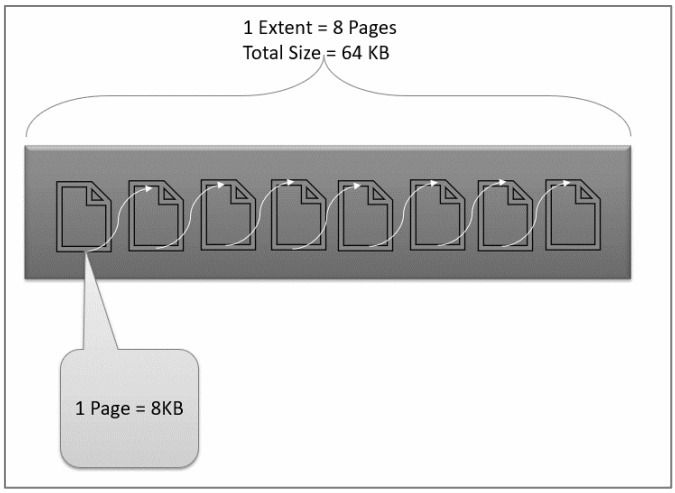
a) Data File

Data File là tệp dữ liệu, tên Data File có 2 tên là Logical File và Physical File.



Hình 1.4: Minh họa Logical File và Physical File

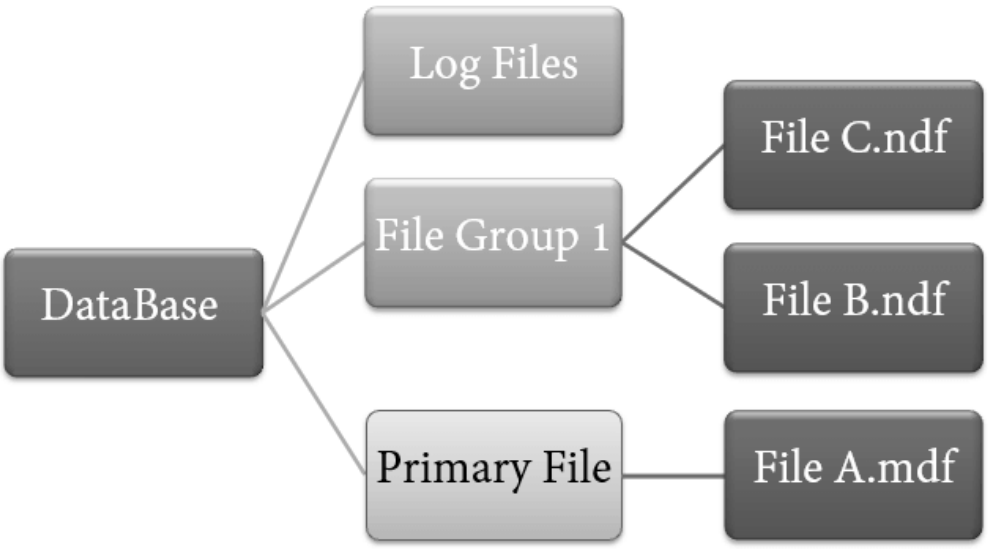
Data File được lưu trữ dưới dạng các trang dữ liệu (Page) hoặc Extent:



Hình 1.5: Minh họa Page và Extent

- Page: mỗi trang dữ liệu có kích thước 8KB, tạo thành đơn vị lưu trữ nhỏ nhất trong SQL Server. Data page có một phần được gọi là tiêu đề trang (Page Header) với kích thước 96 bytes, mang thông tin siêu dữ liệu về page như Loại trang (Page type), Số trang (Page number), Kích thước của không gian được sử dụng (Size of used space), Kích thước của không gian trống (Size of free space) và con trỏ (Pointer) đến page tiếp theo và page trước … Có 9 loại trang dữ liệu trong SQL Server gồm:
  + Data: Các hàng dữ liệu với tất cả dữ liệu từ text, ntext và ảnh.
  + Index: Các mục chỉ mục.
  + Tex\Image: Dữ liệu text, ntext và ảnh.
  + GAM: Thông tin về extent được chỉ định.
  + SGAM: Thông tin về extent được cấp phát ở mức hệ thống.
  + Page Free Space (PFS): Thông tin về không gian trống hiện có trên các trang.
  + Index Allocation Map (IAM): Thông tin về extent được sử dụng bởi bảng hoặc chỉ mục.
  + Bulk Changed Map (BCM): Thông tin về extent được sử đổi bởi hoạt động hàng loạt kể từ lệnh ghi sao lưu cuối cùng.
  + Differential Changed Map (DCM): Thông tin về extent đã thay đổi kể từ lệnh sao lưu cơ sở dữ liệu cuối cùng.
- Extent: Extent là một đơn vị lưu trữ cơ bản. Mỗi Extent là 8 trang liền kề hoặc 64KB. SQL Server có 2 loại Extent là Uniform và Mixed. Uniform được tạo thành từ một object duy nhất, Mixed được tạo thành từ tối đa 8 object.

b) File Type



Hình 1.6: Các loại File trong SQL Server

File Type là kiểu tập tin, được chia thành 3 loại sau:

+ Primary File: mỗi cơ sở dữ liệu chứa một tệp Chính (Primary File), Primary File này lưu trữ tất cả dữ liệu quan trọng liên quan đến bảng (Table), khung nhìn (View), Trigger... Vị trí của tất cả các file trong cơ sở dữ liệu được ghi lại trong cả cơ sở dữ liệu tổng thể và trong Primary File của cơ sở dữ liệu. Primary File thường có phần đuôi mở rộng là *.mdf.
+ Secondary File: Cơ sở dữ liệu có thể có hoặc không chứa nhiều tệp Thứ cấp (Secondary File). Đây là tùy chọn và chứa dữ liệu cụ thể của người dùng. Một cơ sở dữ liệu có thể có nhiều hoặc chỉ có một file dữ liệu phụ. Phần đuối mở rộng thường là *.ndf.
+ Log File: hay còn gọi là Write Ahead Logs, được sử dụng để quản lý các giao tác. Log File dùng để phục hồi cơ sở dữ liệu trong bất kỳ trường hợp không mong muốn nào xảy ra. Thực hiện nhiệm vụ quan trọng của Rollback đối với các giao tác không được cam kết. Phải có ít nhất một file log, hoặc có thể có nhiều hơn một file log cho một cơ sở dữ liệu. Log File thường có phần đuôi mở rộng là *.ldf.

Một database trong SQL Server gồm tối thiểu 2 file: file *.mdf và file *.ldf.

c) Các thành phần trong Storage Engine

- **Access Methods**: Access Methods hoạt động như một giao diện giữa người thực hiện truy vấn với Trình quản lý bộ đệm (Buffer Manager) hay Nhật ký giao tác (Log File). Bản thân Access Methods không thực hiện bất kỳ thao tác thực thi nào, hành động đầu tiên của Access Methods là xác định xem truy vấn thuộc loại DDL hay không? Nếu truy vấn là DDL, ví dụ câu lệnh SELECT, truy vấn được chuyển đến Trình quản lý bộ đệm để xử lý tiếp. Ngược lại, nếu là NON-SELECT (INSERT, DELETE hay UPDATE), truy vấn được chuyển đến Trình quản lý giao dịch (Transaction Manager).
- **Buffer Manager**:
  + Buffer Manager để truy cập và cập nhật các trang cơ sở dữ liệu, và Buffer Pool để cắt giảm I/O file vào database
  + Plan Cache: trình quản lý bộ đệm kiểm tra xem kế hoạch thực hiện có được lưu trữ trong Plan Cache hay không? Nếu Có, thì Plan Cache truy vấn vào bộ đệm dữ liệu (Data Cache) liên quan của nó được sử dụng. Nếu kế hoạch thực hiện truy vấn lần đầu tiên đang được chạy và phức tạp, thì việc lưu trữ nó trong Plan Cache là hợp lý. Điều này sẽ đảm bảo tính khả dụng nhanh hơn khi máy chủ SQL tiếp theo nhận được cùng một truy vấn.
  + Data Cache: Buffer Manager tìm kiếm dữ liệu trong Data Cache, nếu có thì dữ liệu được sử dụng bởi Query Executor và cải thiện hiệu suất việc thực thi từ I/O (vì quá trình đọc và ghi đĩa là một trong những hoạt động sử dụng nhiều

tài nguyên máy tính). Dirty Pages được lưu trữ như là tiến trình logic của Transaction Manager.

+ Buffer Pool: được chia thành nhiều phần, quan trọng nhất là Buffer Cache và Procedure Cache.
  - Buffer Cache: giữ các trang dữ liệu trong bộ nhớ để những dữ liệu thường xuyên truy cập có thể trích xuất từ bộ nhớ cache. Quá trình thay thế việc đọc các trang dữ liệu từ ổ đĩa.
  - Procedure Cache: giữ các thủ tục được lưu trữ và các Excecution Plan để tối ưu hóa số lần Excecution Plan được tạo. Chúng ta có thể tìm thấy thông tin về dung lượng và hoạt động trong Procedure Cache bằng cách sử dụng lệnh DBCC PROCCACHE.

- **Transaction Manager**: trình quản lý giao tác được gọi khi Access Methods xác định rằng truy vấn thuộc loại câu lệnh NON-SELECT.

1.2.2. Các dịch vụ (Services)

Service là một chương trình chạy (run) mà không yêu cầu bất kỳ tương tác người dùng nào. Một số ví dụ thông thường về service như trình điều khiển (Driver) cho bàn phím/chuột, phần mềm chống vi-rút máy tính (Antivirus Software), hay các service trong SQL Server.

SQL Server được cài đặt gồm nhiều services liên quan phụ thuộc lẫn nhau:

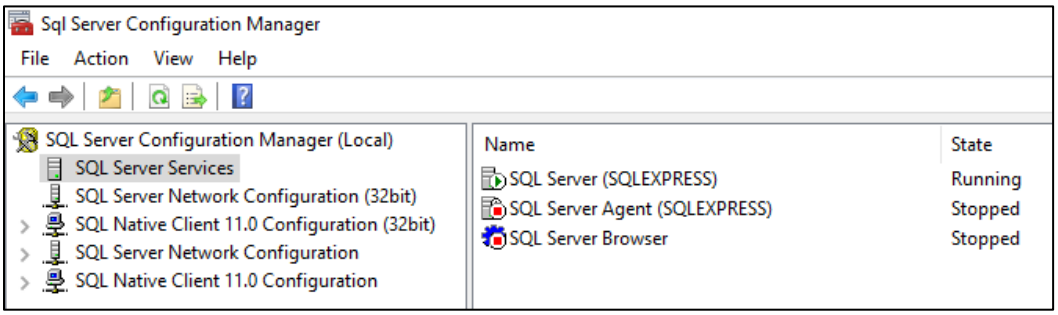
a) SQL Server Service, SQL Server Agent Service và SQL Server Browser Service

- SQL Server Service: SQL Server có tên hiển thị là MSSQLSERVER, và là dịch vụ cơ sở dữ liệu chính được sử dụng bởi SQL Server Database Engine. Dịch vụ này không phụ thuộc vào bất kỳ dịch vụ nào khác để chạy (run) và mặc định ở trạng thái (Status) "Running", và có thể được cấu hình để chạy như là một Domain User, hoặc Local User, hoặc Managed Service Accounts, hoặc Virtual Accounts, hoặc Built-in System Account. Microsoft khuyến cáo sử dụng tài khoản ảo mặc định (Default Virtual Accounts), tài khoản Managed Service Accounts với quyền truy cập yêu cầu tối thiểu. Việc thao tác với cơ sở dữ liệu chỉ có thể được thực hiện khi SQL Server Service ở status "Running".
- SQL Server Agent: SQL Server Agent cũng có tên hiển thị là MSSQLSERVER. Dịch vụ này chịu trách nhiệm chạy các lập lịch (Scheduled Tasks and Jobs). Mặc định, dịch vụ này ở trạng thái "Stopped" và khi cần thì dịch vụ này sẽ được khởi động thủ công. Tuy nhiên, để SQL Server đảm bảo các nhiệm vụ về scheduled maintenance, backup, and monitoring tasks, dịch vụ này cũng được khuyến cáo thiết lập bắt đầu tự động (set to start automatically). SQL Server Agent Service phụ thuộc vào SQL Server Service, nghĩa là chỉ có thể khởi động khi SQL Server đang ở trạng thái "Running".

- SQL Browser Service: dịch vụ này chịu trách nhiệm lắng nghe các yêu cầu từ client để cung cấp tài nguyên và kết nối đến SQL Server (thường port 1434). Hoạt động của dịch vụ này cũng có thể được hiểu như DNS Server. Chỉ có một SQL Browser Service để cung cấp thông tin kết nối cần thiết cho chỉ một máy chủ SQL Server mà nó đang chạy trên máy đó, nó không thể cung cấp thông tin của những máy chủ SQL Server khác mà chạy trên các máy khác trong cùng một mạng.

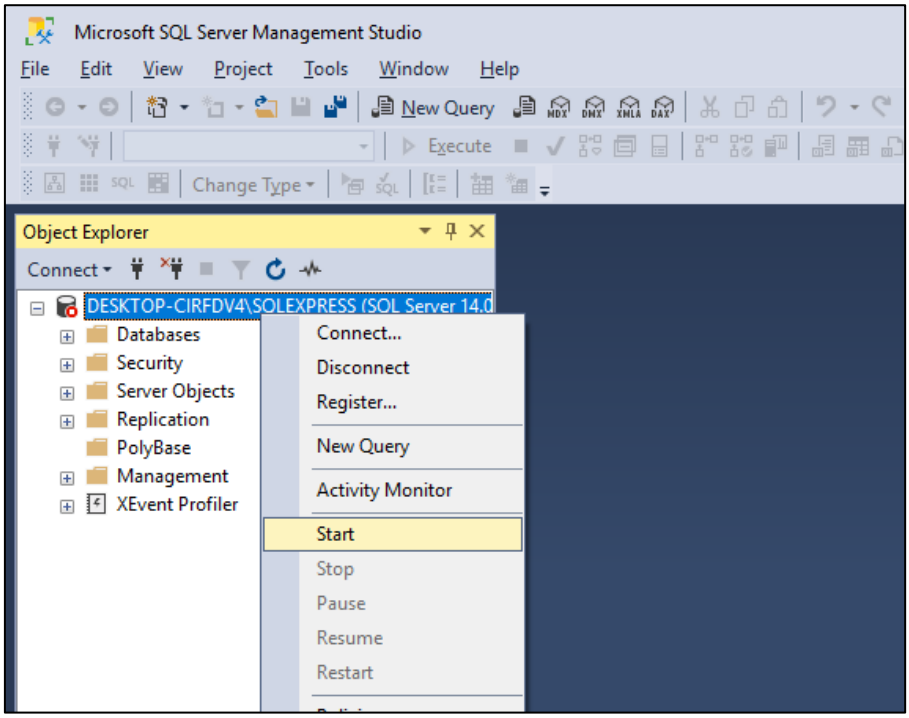
Ba cách khác nhau cho phép thay đổi trạng thái Status các SQL Service như sau:

i. Sử dụng SQL Server Configuration Manager:



Hình 1.7: Cấu hình SQL Services với SQL Service Configuration Manager

ii. Sử dụng Microsoft SQL Server Management Studio (cửa sổ Object Explorer) – xem nội dung "Cài đặt…" ở mục 1.3.



Hình 1.8: Cấu hình SQL Server Service với Microsoft SQL Server Management Studio

iii. Sử dụng Command Prompt: khởi động Command Prompt với quyền Local Administrator

+ Khởi động Command Prompt với lệnh cmd
+ Gõ lệnh: start "DESKTOP-CIRFDV4\SQLEXPRESS" (tên Server cụ thể)

b) Microsoft Distributed Transaction Coordinator Service (MSDTC)

Dịch vụ MSDTC điều phối các giao tác phân tán (Distributed Transaction) giữa hai hoặc nhiều máy chủ cơ sở dữ liệu. Các ứng dụng client sử dụng dịch vụ này để làm việc với dữ liệu từ nhiều nguồn trong một transaction. Luôn luôn chỉ có một phiên bản MSDTC chạy trên máy tính bất kể có bao nhiêu phiên bản máy chủ cơ sở dữ liệu được cài đặt. Dịch vụ này không bắt buộc, nghĩa là nó có thể ở trạng thái bị tắt (Disabled), ví dụ khi máy tính chạy local database. Service này có thể được thao tác sau khi cài đặt Window Server trong mục "Troubleshooting SQL Server Distributed Transactions".

1.2.3. SQL Server Business Intelligence (BI)

BI là một loạt các công cụ để chuyển đổi dữ liệu thô thành thông tin doanh nghiệp có thể sử dụng để đưa ra quyết định. Trước đây, nó đã bao gồm IDE (Integrated Development Environment) Studio Business Intelligence Development Studio trong các phiên bản trước của Microsoft SQL Server, nhưng đã được thay thế bởi SQL Server Data Tools - Business Intelligence kể từ khi phát hành Visual Studio 2010.

BI cung cấp các dịch vụ về phân tích, báo cáo và tích hợp của Microsoft gồm 3 dịch vụ sau SQL Server Analysis Services, Reporting Services and Integration Services:

i. SQL Server Integration Services (SSIS)

SSIS cung cấp cơ chế để di chuyển dữ liệu từ các nguồn dữ liệu khác nhau, và cơ chế thay đổi dữ liệu nếu cần thiết.

Ba thành phần cơ bản để thực hiện nhiệm vụ của SSIS gồm: Import and Export Wizard, SSIS Designer, và SSIS API Programming:

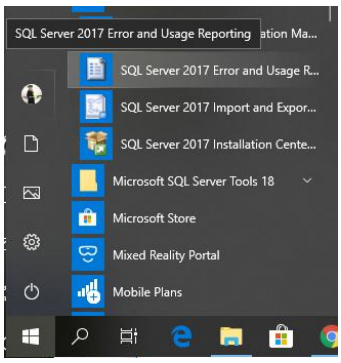
+ Import and Export Wizard: chuyển dữ liệu từ Source (nguồn) đến Destination (đích).
+ SSIS Designer: được sử dụng để phát triển và bảo trì các gói dịch vụ tích hợp.
+ SSIS API Programming: cho phép mã hóa các gói SSIS bởi bất kỳ ngôn ngữ lập trình nào.

ii. SQL Server Reporting Services (SSRS)

SSRS là một framework thực hiện cơ chế báo cáo (Report), gồm Report Builder, Report Designer, Report Manager and Report Server thông qua giao diện Web (Web Interface)

+ Report Builder: là một giải pháp đơn giản cho nhân viên thông tin hoặc người dùng doanh nghiệp (Worker or Business User) để tạo các báo cáo nhanh mà không cần phải hiểu cấu trúc cốt lõi của dữ liệu
+ Report Designer: là một công cụ dành cho nhà phát triển (Developer) vì nó tăng thêm độ phức tạp cho việc phát triển báo cáo tùy chỉnh
+ Report Manager: là một công cụ quản trị kiểm soát các dịch vụ báo cáo thông qua giao diện Web.

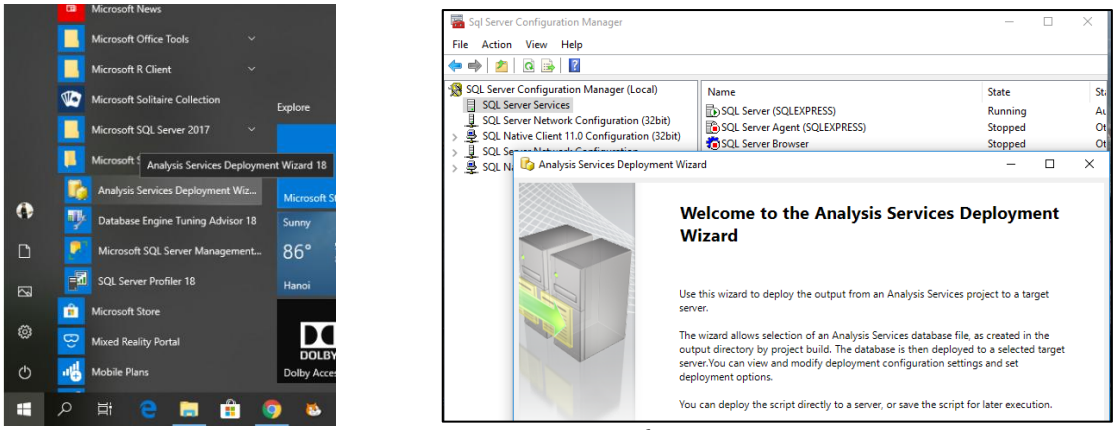
+ Report Server: báo cáo về quá trình xử lý của các cơ sở dữ liệu và thành phần chính. Report Server là công cụ xử lý cốt lõi trong SSRS quản lý việc xử lý và phân phối báo cáo.



Hình 1.9: Giao diện khởi động để làm việc với SSRS

iii. SQL Server Analysis Services (SSAS)

SSAS là một công cụ phân tích đa chiều có tính năng xử lý phân tích trực tuyến (Online Analytical Processing), có khả năng khai thác dữ liệu mạnh mẽ và sâu hơn trong cơ sở dữ liệu quan hệ. Phân tích đa chiều là một kỹ thuật tạo ra khả năng phân tích lượng lớn dữ liệu bằng cách lưu trữ dữ liệu theo trục và ô (Axes and Cells) thay vì chế độ xem hai chiều truyền thống trong các hàng và cột (Column and Row).



Hình 1.10: Giao diện khởi động để làm việc với SSAS

1.3. Các phiên bản và cài đặt

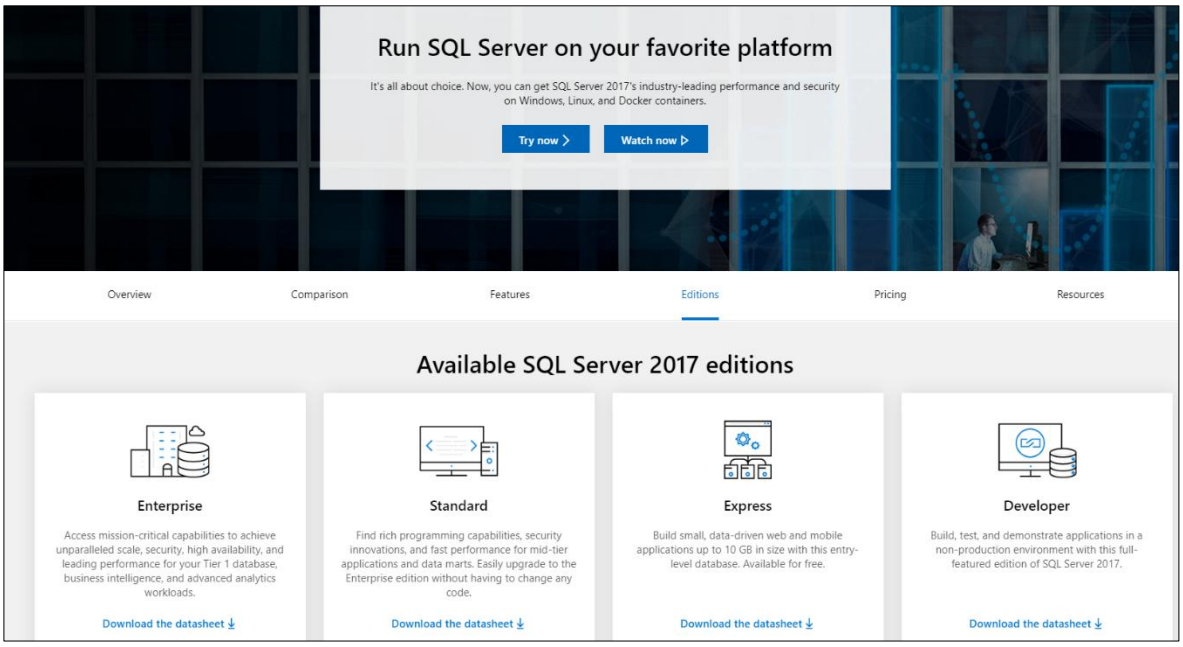
Các thuật ngữ:

- Build − executable or a library created by compiling source code.
- Version − a software build. New version is a different build.
- Release − (public release) a version intended for use by general population.
- Edition – is a unique variant of the software product that is usually identified by the specific segment of end users it is designed to serve (ex: Express, Developer, Enterprise, Standard)

➢ Version 2.1.5 means it's the second edition with at least 1 major difference from its 2.0.~ predecessors and 5 minor feature updates or bug fixes.

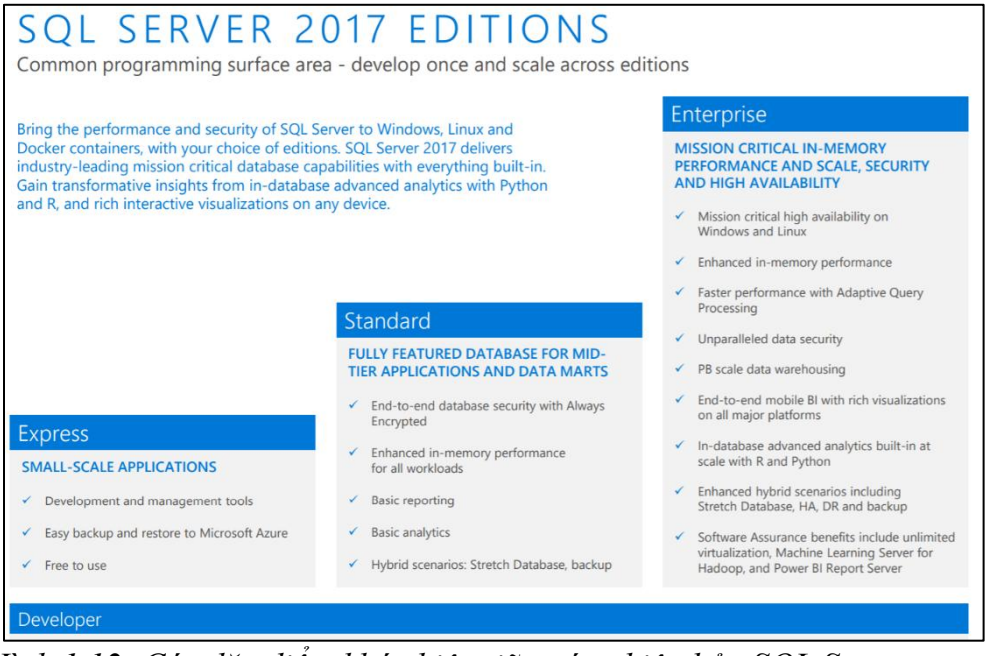
SQL Server hỗ trợ 4 phiên bản chính gồm (ví dụ với SQL Server 2017 - https://www.microsoft.com/en-us/sql-server/sql-server-2017-editions):

+ Bản Enterprise
+ Bản Standard
+ Bản Express
+ Bản Developer



Hình 1.11: SQL Server 2017 Editions

Các đặc điểm khác biệt giữa các phiên bản (ngoại trừ phiên bản Developer dành cho nhà phát triển, tài liệu SQL_Server_2017_Editions_Datasheet.pdf) được thể hiện ở hình dưới:

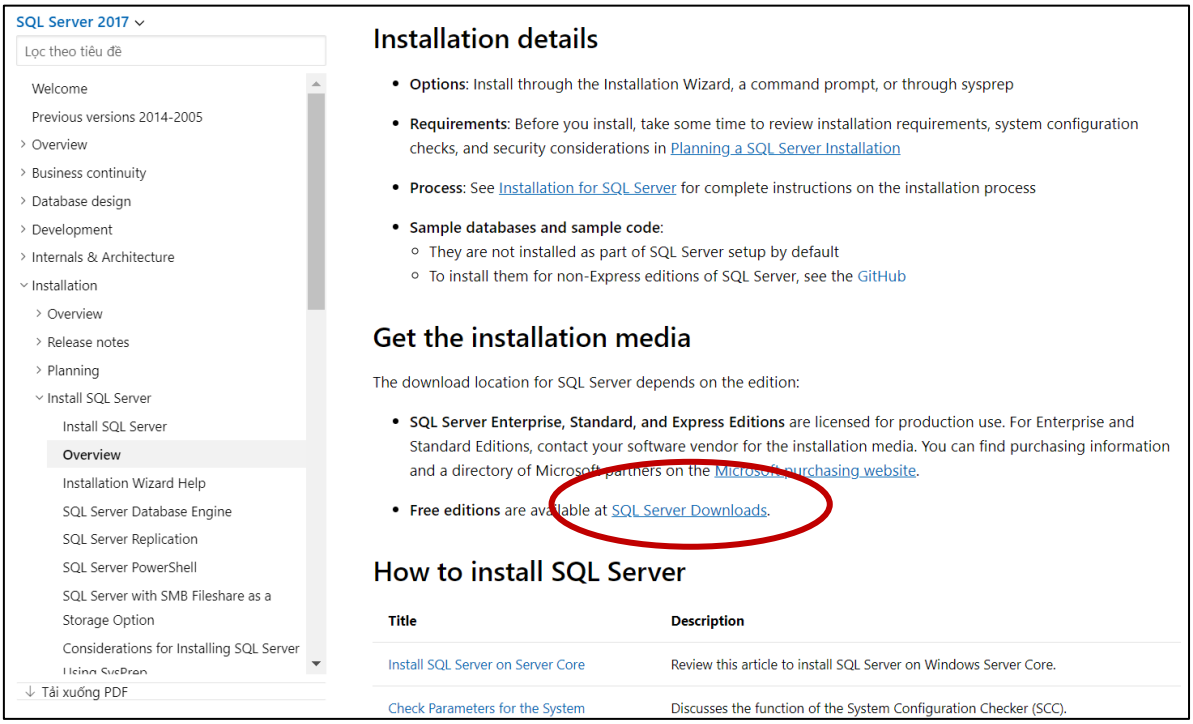


Hình 1.12: Các đặc điểm khác biệt giữa các phiên bản SQL Server

Giáo trình này hướng đến sử dụng và cài đặt phiên bản Express, phù hợp cho việc học, phát triển cũng như xây dựng các ứng dụng nhỏ, và được hỗ trợ Free.

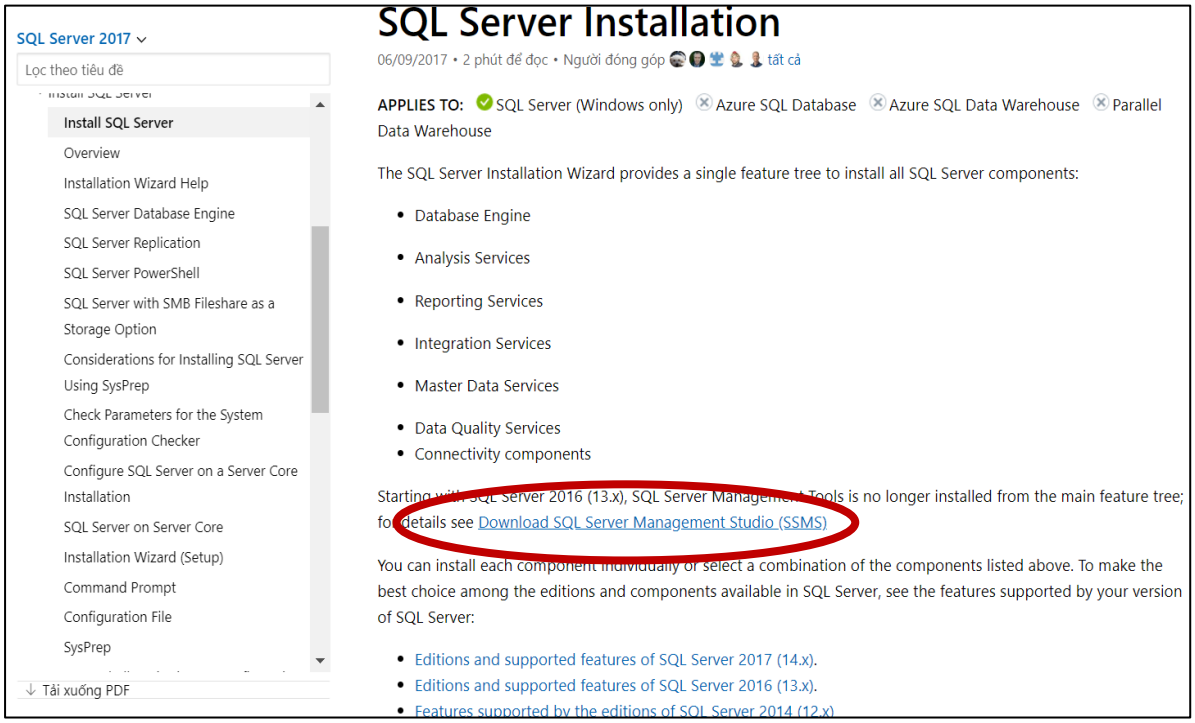
Cài đặt đầy đủ gồm 2 phần riêng biệt:

i. SQL Server:



Hình 1.13: Resources Download SQL Server (Bản 2017)

ii. SQL Server Management Studio (SSMS): là môi trường cho phép người dùng truy vấn, thiết kế và quản trị cơ sở dữ liệu.



Hình 1.14: Resources Download SQL Server Management Studio (SSMS – Bản 2017)

BÀI TẬP

CHƯƠNG 2. THAO TÁC VỚI DỮ LIỆU TRONG SQL SERVER

2.1. Cơ sở dữ liệu quan hệ

2.1.1. Cơ sở dữ liệu (Database)

Một cơ sở dữ liệu (CSDL) là một kho dữ liệu được tổ chức theo một nguyên tắc nào đó. Đó là một tập hợp các tập tin có liên quan với nhau, được thiết kế nhằm làm giảm thiểu sự dư thừa dữ liệu và đảm bảo tính tin cậy khi truy xuất dữ liệu. Các tập tin này chứa các thông tin biểu diễn các đối tượng trong một ứng dụng thế giới thực.

Có thể triển khai đồng thời nhiều ứng dụng trên cùng một CSDL, điều này có nghĩa là các ứng dụng không chỉ chia sẻ chung tài nguyên dữ liệu mà còn có thể triển khai đồng thời nhiều ứng dụng khác nhau tại các thiết bị đầu cuối khác nhau.

CSDL sẽ được quản lý tập trung bởi quản trị viên về cơ sở dữ liệu (một người hay một nhóm người) thông qua các hệ quản trị cơ sở dữ liệu (DataBase Management System – DBMS):

- Quản trị viên CSDL có thể áp dụng thống nhất các tiêu chuẩn, quy định, thủ tục chung như quy định thống nhất về mẫu biểu báo cáo, thời gian bổ sung, cập nhật dữ liệu. Điều này làm dễ dàng cho công việc bảo trì dữ liệu.
- Quản trị viên CSDL còn có thể bảo đảm việc truy nhập tới CSDL, có thể kiểm tra, kiểm soát các quyền truy nhập của người sử dụng. Ngăn chặn các truy nhập trái phép, sai quy định từ trong ra hoặc từ ngoài vào.

DBMS là một hệ thống phần mềm cho phép người dùng quản lý và tương tác với CSDL, cụ thể người dùng có thể định nghĩa, thao tác, và xử lý dữ liệu của một CSDL để xuất ra những thông tin cần thiết có nghĩa.

Một số DBMS phổ biến hiện nay: MS SQL Server, Oracle, MySQL, Microsoft Access, MongoDB, DB2, PostgreSQL,.. Giáo trình này sử dụng MS SQL Server (xem chương 1).

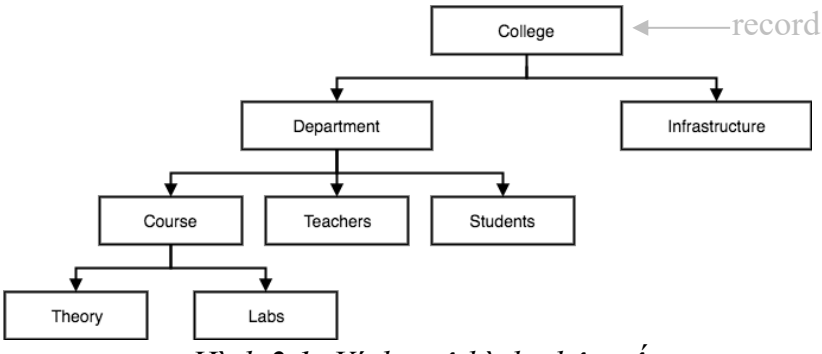
2.1.2. Các mô hình dữ liệu

2.1.2.1. Mô hình dữ liệu phân cấp và mô hình dữ liệu mạng

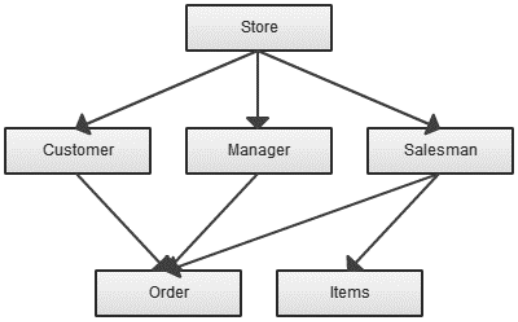
Mô hình dữ liệu phân cấp (Hierachical Model) được phát triển vào đầu những năm 1960, được sử dụng để mô hình cho những dự án với số lượng dữ liệu rất lớn. Mô hình phân cấp này được biểu diễn theo dạng cây phân cấp (tree), mỗi nhánh (segment) biểu diễn một số record có quan hệ với nhau, mỗi record chỉ được có duy nhất một parent record. Quan hệ được thể hiện trong mô hình này là quan hệ one-to-many.

Mô hình dữ liệu mạng (Network Model) được sử dụng để biểu thị cho những dữ liệu có mối quan hệ phức tạp giữa các record với nhau cũng theo quan hệ one-to-many.

Khác biệt lớn nhất giữa mô hình phân cấp và mô hình mạng là, các record trong mô hình mạng được phép có nhiều hơn một parent record.



Hình 2.1: Ví dụ mô hình phân cấp



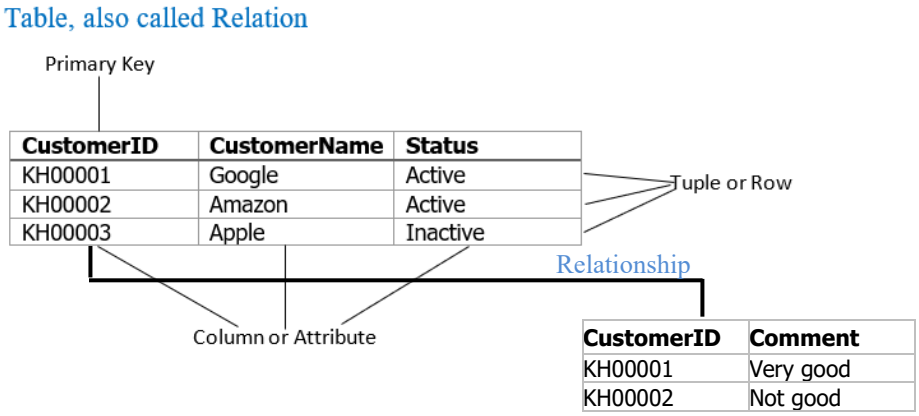
Hình 2.2: Ví dụ mô hình mạng

Lưu ý, mô hình phân cấp và mô hình mạng thường chỉ được sử dụng trong các hệ cơ sở dữ liệu trước đây, và không hỗ trợ để biểu diễn trong các cơ sở dữ liệu quan hệ.

2.1.2.2. Mô hình dữ liệu quan hệ

Mô hình dữ liệu quan hệ (Relational Model) được giới thiệu vào những những năm 1970, được dùng để trao đổi về mặt dữ liệu giữa designers và end users. Nền tảng của mô hình dữ liệu chính là relation (đôi khi còn được gọi là table), là một cấu trúc 2 chiều gồm row và column. Mỗi row được gọi là một tuple, mỗi column được gọi là một attribute.

Mô hình quan hệ cung cấp mức thấp nhất để kiểm soát dư thừa dữ liệu để có thể giảm bớt dữ thừa dữ liệu, và được hiện thực thông qua hệ quản trị cơ sở dữ liệu quan hệ (RDBMS).



Hình 2.3: Ví dụ mô hình quan hệ

2.1.2.3. Mô hình dữ liệu thực thể quan hệ

Các khái niệm ở mô hình quan hệ dẫn đến nhu cầu tạo ra những công cụ thiết kế cơ sở dữ liệu hiệu quả hơn trước khi database được xây dựng trong hệ quản trị cơ sở dữ liệu. Mặc

dù mô hình quan hệ có cải thiện so với mô hình mạng và mô hình phân cấp, tuy nhiên vẫn chưa có nhiều công cụ hỗ trợ việc thiết kế database hiệu quả cho mô hình quan hệ.

Lý do là cấu trúc về mặt đồ họa (graphical) thì cần thiết và thể hiện được nhiều thông tin hơn so với hiển thị bằng text. Mô hình thực thể quan hệ (Entity Relationship Model – ERM) đáp ứng được nhu cầu thể hiện Entity và Relationship bằng hình ảnh (picture).

Peter Chen lần đầu tiên giới thiệu mô hình ERM vào năm 1976, và dần được phổ biến cho đến ngày nay. Mô hình ERM được thể hiện bởi sơ đồ thực thể quan hệ ERD (Entity Relationship Diagram).

Có nhiều hệ thống ký hiệu để vẽ sơ đồ ERD, bao gồm:

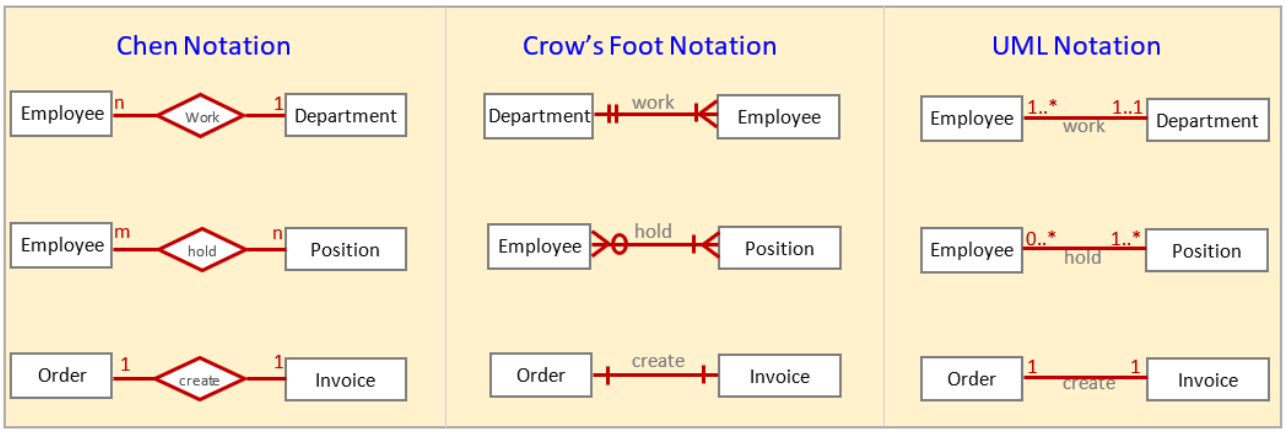
- Chen Notation: entity được thể hiện bởi hình chữ nhật (box), relationship được thể hiện bởi hình thoi (diamond) – nằm giữa 2 thực thể, dùng số 1 hoặc M để thể hiện sự phụ thuộc giữa thực thể này so với thực thể kia.

  Công cụ hỗ trợ vẽ ERD: SmartDraw, Microsoft Visio, Edraw

- Crow's Foot Notation: relation ship được thể hiện bởi ký hiệu nét gạch (được dùng phổ biến hơn).

  Công cụ hỗ trợ vẽ ERD: Star UML, Visual Paradigm, draw.io, Microsoft Visio

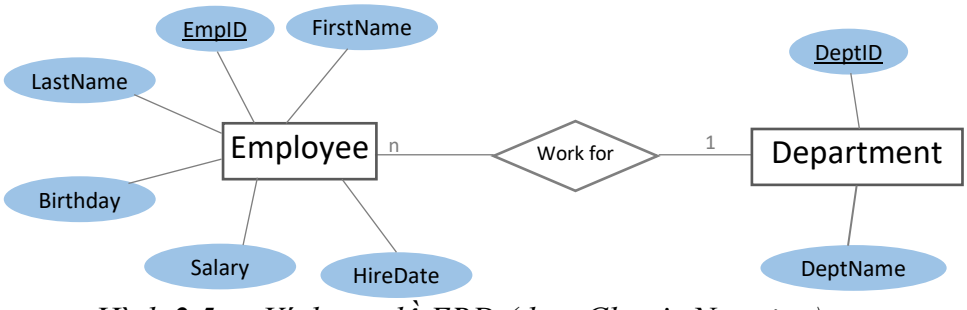
- UML Notation: relationship được thể hiện bởi ký hiệu 1..1 hoặc 1..*. Hiện nay, UML Notation phiên bản mới đã bổ sung các ký hiệu để vẽ theo hệ thống Crow's Foot Notaion.



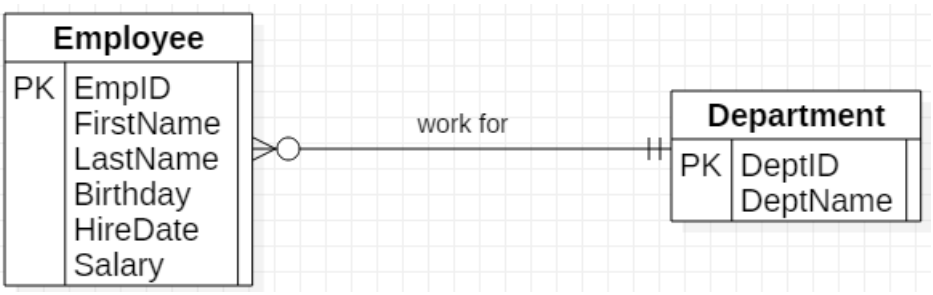
Hình 2.4: Hệ thống các ký hiệu theo Chen, Crow's Foot và UML

Giải thích:

- *Mỗi Employee chỉ thuộc về (làm việc cho) 1 Department, và mỗi Department có thể có nhiều nhiều Employee ➔ dạng quan hệ one-to-many (1:n)*
- *Mỗi Employee có thể giữ nhiều vị trí, và một vị trí cũng có nhiều Employee đảm nhận ➔ dạng quan hệ many-to-may (m:n)*
- *Mỗi Order phát sinh ra một Invoice, và mỗi Invoice chỉ xác định cho một Order ➔ dạng quan hệ on-to-one (1:1)*



Hình 2.5.a: Ví dụ sơ đồ ERD (theo Chen's Notation)



Hình 2.5.b: Ví dụ sơ đồ ERD (theo Crow's Foot Notation)

2.1.3. Các giai đoạn thiết kế cơ sở dữ liệu

Các cơ sở dữ liệu khác nhau có thể khác nhau dựa vào mô hình và chức năng của dữ liệu. Một mô hình cơ sở dữ liệu (Database Model) định nghĩa thiết kế logic, kiến trúc của một cơ sở dữ liệu.

Các giai đoạn thiết kế cơ sở dữ liệu gồm 3 giai đoạn sau:

a) Thiết kế cơ sở dữ liệu khái niệm (Conceptual Database Design)

Đầu tiên, ta cần thiết kế mức khái niệm cho cơ sở dữ liệu. Giai đoạn này cần xác định 2 thành phần gồm: khái niệm cho việc biểu diễn dữ liệu (theo cách mà người dùng nhận thức về dữ liệu), mối quan hệ giữa các khái niệm này. Đây được xem là cầu nối giữa các đối tượng trong thế giới thực với cơ sở dữ liệu trong máy tính, và phải đảm hợp nhất về mặt dữ liệu giữa designers, programmers và end users.

Thiết kế cơ sở dữ liệu khái niệm cần tạo ra mô hình dữ liệu khái niệm (Conceptual Data Model). Mô hình này là phương tiện để những người phân tích thiết kế giao tiếp với người sử dụng, nhằm thu thập thông tin, xác định đúng đắn và đầy đủ các đặc tả yêu cầu của hệ thống.

Như đã trình bày ở mục 2.1.2.3, sơ đồ ERD được sử dụng để biểu diễn cơ sở dữ liệu khái niệm, các công việc ở giai đoạn này gồm:

- Xác định các Entity, các thuộc tính của thực thể: thuộc tính đơn hay thuộc tính phức hợp, hay là thuộc tính dẫn xuất (từ một thuộc tính của thực thể khác)
- Xác định các Relationship
- Xác định các Attribute và các miền xác định của thuộc tính (Data Type)
- Xác định các Constraints: các ràng buộc về mặt dữ liệu (ví dụ ProductID gồm 7 ký tự, 2 ký tự 'KH' là bắt buộc, mỗi ký tự trong 5 ký tự còn lại là các con số từ 0..9)

- Xác định thuộc tính là khóa chính
- Xác định các quan hệ như quan hệ kế thừa, quan hệ kết tập hay quan hệ kết hợp giữa các thực thể
- Kiểm tra tính dư thừa dữ liệu, nếu quan hệ 1:1 thì có thể gộp (merge) hai thực thể thành một thực thể mới

b) Thiết kế cơ sở dữ liệu logic (Logical Database Design)

Thiết kế cơ sở dữ liệu logic bao gồm những hoạt động liên quan đến việc ánh xạ (map) và kiểm tra một lần nữa tính hợp lệ (validate) của mô hình dữ liệu khái niệm đã có ở giai đoạn trước đó (giai đoạn thiết kế cơ sở dữ liệu khái niệm).

Có thể hiểu giai đoạn này thực hiện các bước:

- Xác định thực thể mạnh và thực thể yếu: nếu quan hệ m:n thì bổ sung thực thể yếu (weak entity) để thành 2 quan hệ 1:n
- Chuẩn hóa sơ đồ ERD đã có có giai đoạn thiết kế cơ sở dữ liệu khái niệm (ít nhất là phải đạt chuẩn 3NF)
- Xác định Constraint một lần nữa: những thuộc tính không được phép NULL, miền giá trị của thuộc tính, quan hệ khóa chính và khóa ngoại

Sơ đồ ERD được sử dụng ở giai đoạn này. Ngoài ra, có thể bổ sung thêm các mô hình như: mô hình mạng, mô hình phân cấp, hay mô hình quan hệ để biểu thị những thông tin về tập các record, việc ánh xạ.

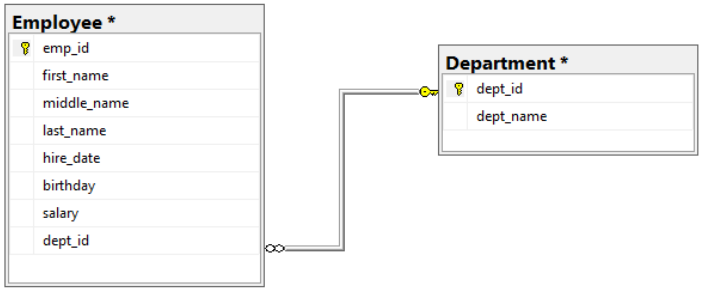
c) Thiết kế cơ sở dữ liệu vật lý (Physical Database Design)

Đây là giai đoạn cuối cùng, sử dụng một hệ quản trị cơ sở dữ liệu cụ thể cho thiết kế và lưu trữ cơ sở dữ liệu.

Nếu ở giai đoạn Conceptual Database Design, sử dụng mô hình ER khi thiết kế cơ sở dữ liệu ➔ giai đoạn Logical Database Design cũng nên sử dụng mô hình quan hệ ER để tiếp tục chuẩn hóa và tối ưu lược đồ thiết kế cơ sở dữ liệu ➔ giai đoạn Physical Database Design sẽ dựa vào sơ đồ ERD sau khi đã tối ưu trong DBMS theo nguyên tắc:

- Entity ↔ Table (hay Relation)
- Attribute ↔ Column
- Relationship ↔ Relationship

Ví dụ 1:



Hình 2.6: Relationship Diagram trong DBMS

Từ ERD ở hình 2.5.b, khi tạo cơ sở dữ liệu trong hệ quản trị cơ sở dữ liệu SQL Server, thực thể Employee sẽ trở thành một bảng có tên Employee, các thuộc tính tương ứng trong thực thể Employee sẽ trở thành các cột trong bảng Employee; thực thể Department sẽ trở thành bảng có tên Department, các thuộc tính trong thực thể Department sẽ trở thành các cột trong bảng Department.

2.1.4. Mô hình thực thể quan hệ (ERM)

Hình dưới là sơ đồ ERD cho hệ thống quản lý bán hàng, và sẽ được sử dụng xuyên suốt giáo trình để minh họa các câu lệnh T-SQL thao tác với cơ sở dữ liệu SaleMangement.

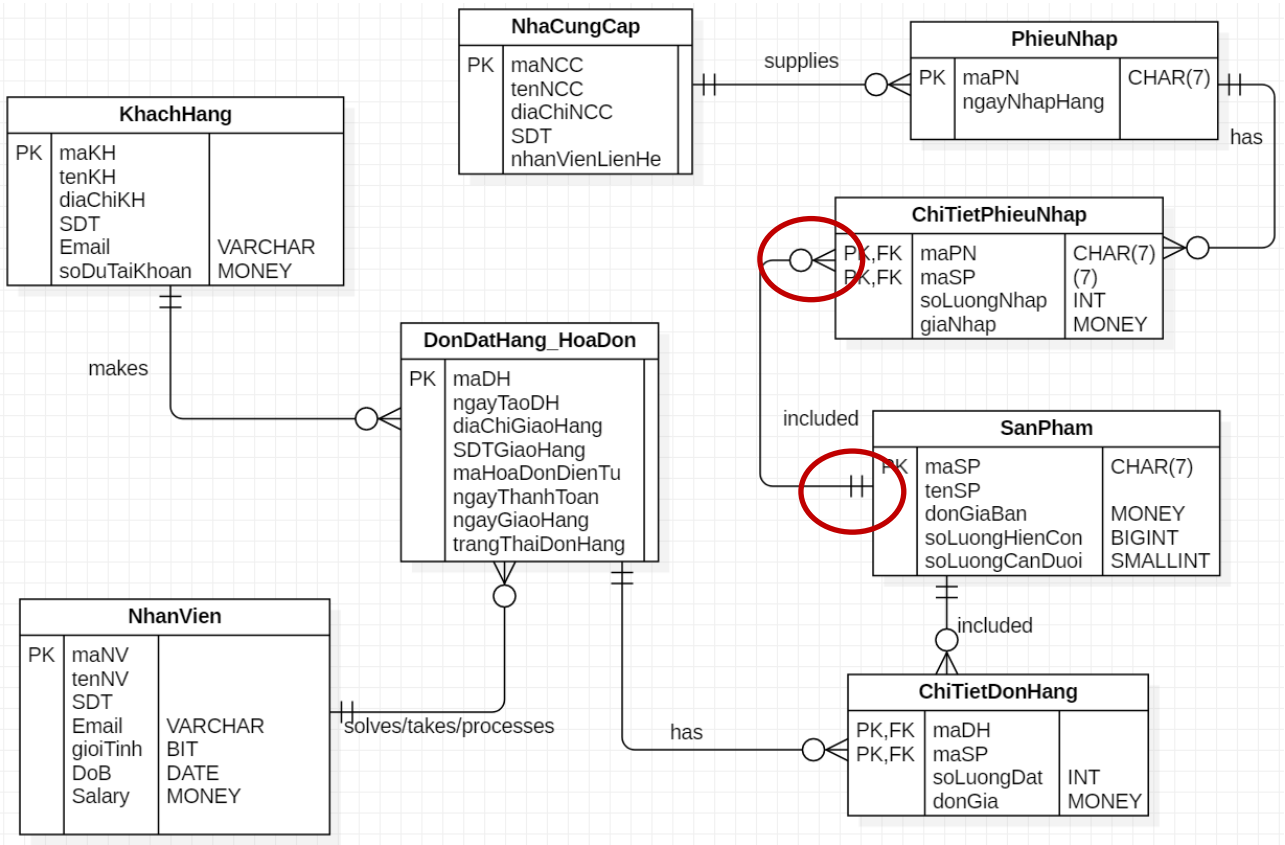
Ví dụ:

Với bài toán quản lý bán hàng, cho phép quản lý các đơn hàng bán ra và các phiếu nhập hàng về từ nhà cung cấp.

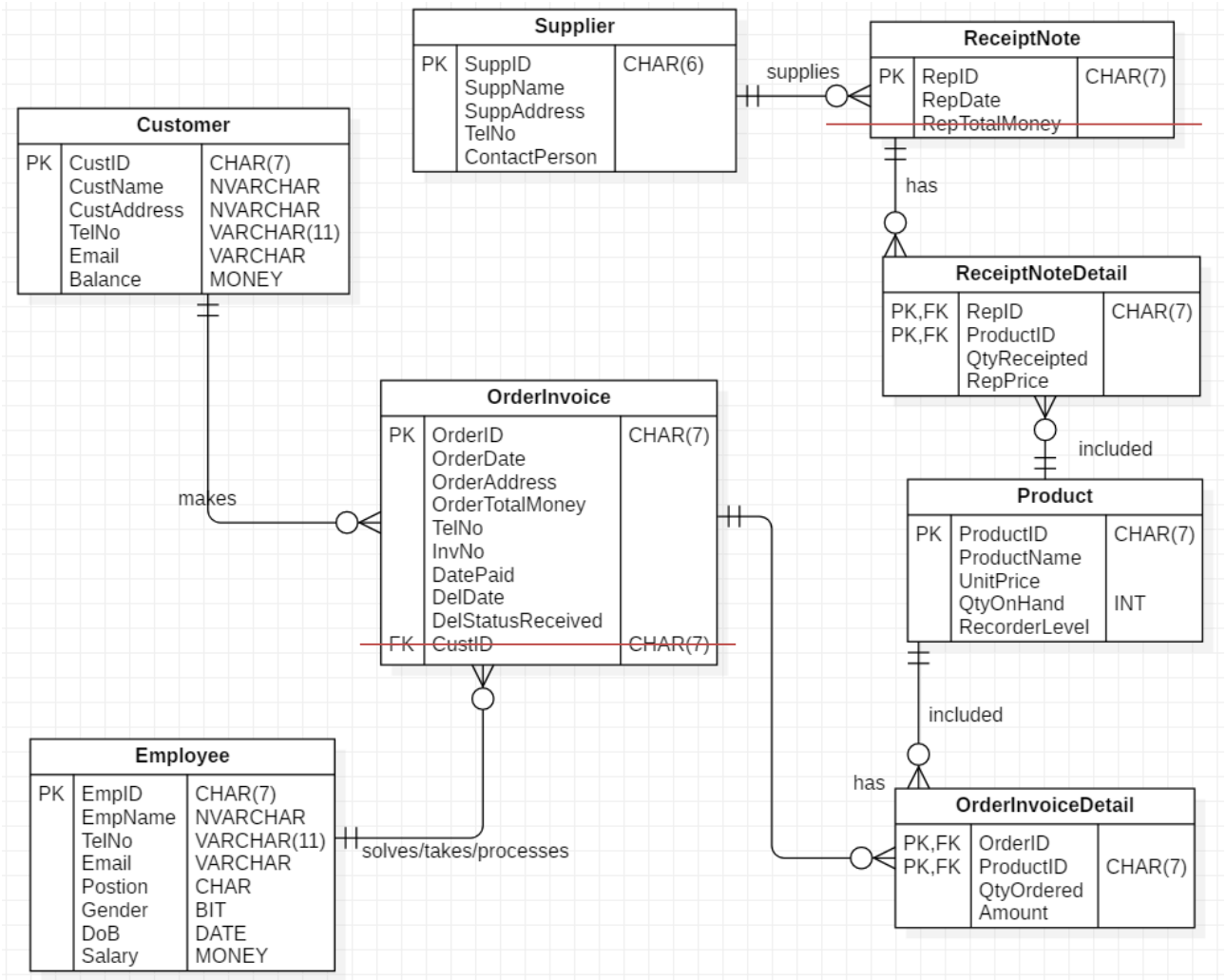
Để có được 2 ERD dưới đây (bản tiếng Anh và bản tiếng Việt), cần thiết kế ERD bao gồm các thực thể lưu các cột thông tin liên quan trong từng thực thể, và mối quan hệ khóa chính – khóa ngoại giữa các cặp thực thể.

ERD ở hình 2.7 được thiết kế với tiêu chí đặt tên Entity và tên Column/Attribute theo tiếng Việt không dấu.

ERD ở hình 2.8 được thiết kế với tiêu chí đặt tên Entity và tên Column/Attribute theo tên tiếng Anh (nhằm tránh trường hợp hiểu sai ý nghĩa với tên tiếng Việt không dấu ở hình 2.7) → các dòng lệnh ở những nội dung sau sẽ được viết dựa trên ERD bản tiếng Anh.



Hình 2.7: Sơ đồ ERD cho bài toán Quản lý bán hàng (Sale Management)



Hình 2.8: Sơ đồ ERD cho bài toán Sale Management

2.1.4.1. Thực thể (Entity) ≡ Table

Một thực thể là một đối tượng, một địa điểm, con người…có thể xác định được và tồn tại trong thế giới thực. Ví dụ trong sơ đồ ERD cho bài toán quản lý bán hàng (hình 2.6) trên, các thực thể có thể là: Customer (Khách_Hàng), Product (Sản_Phẩm), Employee (Nhân_Viên),..

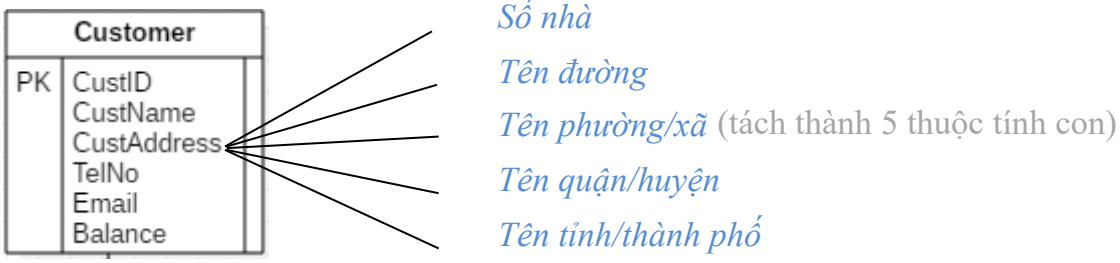
Thực thể yếu là một thực thể phải được định nghĩa bởi một khóa ngoại với thực thể khác vì nó không thể được định nghĩa duy nhất, nghĩa là nó phụ thuộc vào thực thể khác đó. Ví dụ trong sơ đồ ERD cho bài toán quản lý bán hàng trên, ReceiptNoteDetail (Chi_Tiết_Phiếu_Nhập_Hàng) là thực thể yếu vì nó phục thuộc vào giá trị của thuộc tính ProductID trong thực thể Product.

2.1.4.2. Thuộc tính (Attribute) ≡ Column

Các thực thể được đặc trưng bởi các giá trị/nội dung của những thuộc tính (Attribute). Ví dụ, thực thể Product sẽ có các thuộc tính gồm ProductID (mã sản phẩm), ProductName (tên sản phẩm), UnitPrice (đơn giá), QtyOnHand (số lượng còn),…

Mỗi thuộc tính sẽ có kiểu dữ liệu và miền giá trị xác định. Ví dụ, thuộc tính UnitPrice của thực thể Product phải là số nguyên dương và phải luôn lớn hơn hoặc bằng 0. Thuộc tính có thể rơi vào một trong các kiểu sau:

- Thuộc tính đơn trị (Single-valued Attribute): hay còn gọi là Simple Attribute, là thuộc tính chỉ nhận một giá trị cụ thể. Ví dụ như trong thực thể Customer, thuộc tính CustName = "Lê Thị Thu Hà" (không thể vừa nhận giá trị là "Đinh Văn Đông" được)
- Thuộc tính phức hợp (Composite Attribute): là thuộc tính mà giá trị của nó có thể được phân tách thành các attribute nhỏ hơn. Ví dụ như trong thực thể Customer, thuộc tính CustAddress (địa chỉ) có giá trị gồm thông tin về số nhà – tên đường – tên phường/xã – tên quận/huyện – tên tỉnh/thành phố.



- Thuộc tính đa trị (Multivalued Attribute): là thuộc tính mà giá trị của nó gồm một tập các giá trị khác nhau. Ví dụ như trong thực thể Customer, thuộc tính TelNo có thể nhận nhiều hơn 1 giá trị gồm '0907666111' và '0236 3816995'.
- Thuộc tính dẫn xuất (Derived Attribute): là thuộc tính mà giá trị được tính toán dựa vào giá trị của thuộc tính khác. Ví dụ, trong thực thể Employee có thuộc tính DoB (Date of Birth) – nếu bổ sung thêm thuộc tính Age cho thực thể Employee thì giá trị của thuộc tính Age được tính từ giá trị của thuộc tính DoB bằng cách "lấy giá trị năm hiện tại trừ đi thành phần năm của giá trị DoB).

Trong trường hợp không xác định được giá trị của thuộc tính, lúc đó thuộc tính được gọi là nhận giá trị NULL.

2.1.4.3. Khóa (Key)

Một ràng buộc quan trọng đối với một thực thể là khóa. Khóa được tạo bởi một thuộc tính hoặc một nhóm các thuộc tính, có giá trị có mà thể được sử dụng để phân biệt các thể hiện (bộ) của thực thể.

Các loại khóa:

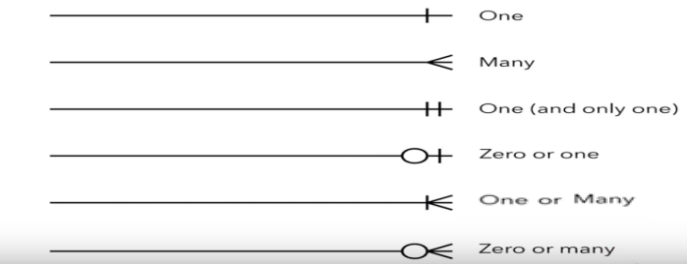
- Candidate key (khóa dự tuyển/khóa ứng viên): khóa dự tuyển phải thỏa mãn tính duy nhất và không tối thiểu. Tính duy nhất nghĩa là không có 2 hàng dữ liệu trong một bảng có cùng giá trị, và tính tối thiểu nghĩa là tất cả các cột đều cần thiết để đảm bảo tính duy nhất (khi ánh xạ Entity ↔ Table). Vì vậy, nếu xóa bất kỳ 1 thuộc tính nào của khóa, đều dẫn đến sự phá hủy tính xác định duy nhất của khóa. Khóa dự tuyển có thể là khóa đơn hay khóa phức hợp.

- Simple key (khóa đơn): là khóa được tạo từ chỉ một cột, ví dụ thuộc tính CustID trong thực thể Customer.
- Composite key (khóa kết hợp): là khóa được tạo thành từ hơn 1 thuộc tính, và phải đảm bảo tính tối thiểu. Ví dụ thuộc tính RepID và ProductID trong thực thể ReceiptNoteDetail.
- Primary key (khóa chính): thường được viết tắt là PK, là khóa dự tuyển được lựa chọn bởi nhà thiết kế dữ liệu để xác định tính duy nhất của bộ dữ liệu (tuples – tập nội dung của các thuộc tính nằm trên một hàng) trong bảng và không thể nhận giá trị rỗng (NULL). Ví dụ thuộc tính CustID làm PK trong thực thể Customer.
- Foreign key (Khóa ngoại): thường được viết tắt là FK, là thuộc tính mà giá trị của nó tham chiếu đến các giá trị của khóa chính trong một bảng khác, và có thể nhận giá trị NULL.

2.1.4.4. Mối quan hệ (Relationship)

Mối quan hệ, đôi khi còn được gọi là quan hệ, dùng để liên kết các thực thể lại với nhau. Các loại quan hệ giữa các thực thể gồm:

- Quan hệ 1-1 (One – to – one): quan hệ 1-1 là quan hệ giữa một thực thể này với chỉ một thực thể khác, và ngược lại. Quan hệ này thường rất hiếm trong các bản thiết kế cơ sở dữ liệu quan hệ. Trong thực tế, quan hệ này chỉ ra rằng 2 thực thể nên thuộc về cùng một table (trong DBMD).
- Quan hệ 1-n (one – to – many): quan hệ 1-n là quan hệ phổ biến trong hầu hết các thiết kế cơ sở dữ liệu quan hệ, là quan hệ giữa một thực thể này với nhiều thể hiện khác nhau của thực thể khác (có thể hiểu là nhiều row trong table khác).
- Quan hệ n-n (many – to – many): quan hệ n-n là quan hệ giữa hai thực thể mà trong đó một thực thể này có thể liên kết với 0,1 hoặc nhiều thể hiện của thực thể và ngược lại.



Hình 2.9: Ký hiệu và ý nghĩa quan hệ theo Crow's Foot Notation

2.2. Ngôn ngữ T-SQL

2.2.1. Tổng quan ngôn ngữ T-SQL

SQL (Structured Query Language) là ngôn ngữ cho phép người dùng tạo cơ sở dữ liệu, thực hiện các thao tác với dữ liệu cũng như quản trị dữ liệu, và truy vấn dữ liệu theo nhu cầu của người dùng.

T-SQL (Transact-SQL) là tập các mở rộng về lập trình cho ngôn ngữ SQL bằng cách bổ sung vài tính năng thao tác với cơ sở dữ liệu gồm: điều khiển giao tác (transaction control), xử lý ngoại lệ và lỗi (exception and error handling), lập trình hướng thủ tục (procedural programming), xử lý dòng dữ liệu (rows) và các biến (variables).

Các ứng dụng giao tiếp với SQL Server bằng cách gửi các yêu cầu (request) dạng câu lệnh T-SQL đến SQL Server ➔ thành phần Relational Engine trong SQL Server sẽ thực hiện xử lý yêu cầu và thao tác với dữ liệu thông qua thành phần Storage Engine.

2.2.1.1. Các kiểu dữ liệu (Data Type)

T-SQL cho phép thao tác với nhiều kiểu dữ liệu được phân nhóm theo bảng sau:

Tên nhóm	Tên kiểu dữ liệu	Mô tả	Minimum - Maximum
Character String (Chuỗi ký tự)	char	chuỗi có độ dài cố định	8000 ký tự
	varchar	chuỗi có độ dài thay đổi	8000 ký tự
	varchar(max)	chuỗi có độ dài thay đổi	2^{31} ký tự (2GB)
	text	chuỗi có độ dài thay đổi	2^{31} -1 = 2,147,483,647 ký tự
Unicode Character String (Chuỗi ký tự có hỗ trợ Unicode)	nchar	chuỗi Unicode có độ dài cố định	4000 ký tự
	nvarchar	chuỗi Unicode có độ dài thay đổi	4000 ký tự
	ntext	chuỗi Unicode có độ dài thay đổi	1,073,741,823 ký tự
Binary (Nhị phân)	binary	nhị phân có độ dài cố định	8000 bytes
	varbinary	nhị phân có độ dài thay đổi	8000 bytes
	image	nhị phân có độ dài thay đổi	2,147,483,647 bytes
Exact Numeric (Số chính xác – không có sai số)	bigint	số nguyên từ -2^{63} đến $(2^{63}-1)$	-9,223,372,036,854,775,808 đến 9,223,372,036,854,775,807
	int	số nguyên từ -2^{31} đến $(2^{31}-1)$	-2,147,483,648 đến 2,147,483,647
	smallint	số nguyên từ -2^{15} đến $2^{15}-1$	-32,768 đến 32767
	tinyint	số nguyên từ 0 đến $2^{8}-1$	0 đến 255
	bit	nhận trong 3 giá trị 0,1,NULL	0 đến 1
	decimal(p,s)	p là tổng số lượng các chữ số (mặc định p=18), s là số lượng các chữ số nằm bên phải của dấu phân cách phần nguyên và phần phân (mặc định s =0)	$(-10^{38}+1)$ đến $(10^{38}-1)$
	numeric(p,s)	tương tự như kiểu decimal	
	money	dữ liệu tiền tệ	-922,337,203, 685,477.5808 đến 922,337,203, 685,477.5807

Tên nhóm	Tên kiểu dữ liệu	Mô tả	Minimum - Maximum
	smallmoney	dữ liệu tiền tệ	-214,748.3648 đến 217,748.3647
Approximate Numeric (Số gần đúng – có sai số)	float(n)	dữ liệu dấu chấm động, kích thước bộ nhớ tùy thuộc vào n	-(1.79E + 308) đến (1.79E+308)
	real	dữ liệu dấu chấm động, kích thước bộ nhớ 4 bytes	(-3.40E+38) đến (3.40E+38)
Date & Time	datetime	dữ liệu ngày và giờ: - ngày lấy từ 1753-01-01 đến 9999-12-31 - giờ lấy từ 00:00:00 đến 23:59:59.997 (có độ chính xác 3.33 milliseconds)	
	datetime2	dữ liệu ngày và giờ: - ngày lấy từ 0001-01-01 đến 9999-12-31 - giờ lấy từ 00:00:00 đến 23:59:59.9999999	
	smalldatetime	dữ liệu ngày và giờ: - ngày lấy từ 1900-01-01 đến 2079-6-6 - giờ lấy từ 00:00:00 đến 23:59:59 (độ chính xác phút)	
	date	dữ liệu ngày từ 0001-01-01 đến 9999-12-31	
	time	dữ liệu giờ: thời gian của một ngày với độ chính xác 100 nano giây, giá trị từ 00:00: 00.0000000 đến 23:59:59.9999999.	
Kiểu dữ liệu khác	sql_variant	lưu giữ các giá trị của các kiểu dữ liệu đa dạng được hỗ trợ bởi SQL Serverv, ngoại trừ text, ntext, và timestamp	
	timestamp	lưu giữ số duy nhất mà được cập nhật mỗi khi một hàng được cập nhật	
	uniqueidentifier	lưu giữ một định danh chung (Globally Unique Identifier - GUID)	

Tên nhóm	Tên kiểu dữ liệu	Mô tả	Minimum - Maximum
	cursor	tham chiếu tới một đối tượng con trỏ (Cursor)	
	xml	lưu giữ dữ liệu XML	

2.2.1.2. Toán tử (Operator)

Toán tử xuất hiện trong biểu thức, và độ ưu tiên của toán tử sẽ quyết định thứ tự thực hiện của các phép tính trong biểu thức.

Các nhóm toán tử:

- Toán tử số học: +, - , *, / , % (chia lấy dư)
- Toán tử so sánh: =, !=, <>, > , <, >=, <=, !< (không nhỏ hơn), !> (không lớn hơn)
- Toán tử logic: AND, ANY, ALL, BETWEEN, EXISTS, IN, LIKE, NOT, OR, IS NULL, UNIQUE

Bảng dưới đây mô tả các toán tử và mức độ ưu tiên của các toán tử:

Mức ưu tiên	Toán tử
1	* (Multiply), / (Division), % (Modulo)
2	+ (Positive), - (Negative), + (Add), (+ Concatenate), - (Subtract),
3	= , >, <, >=, <=, <>, !=, !>, !< (Comparison operators)
4	NOT
5	AND
6	ALL, ANY, BETWEEN, IN, LIKE, OR, SOME…
7	= (Assignment)

2.2.1.3. Chú thích

T-SQL dùng kí hiệu -- để chú thích cho một dòng đơn và kí hiệu /*…*/ để chú thích cho một nhóm dòng. (Xem Ví dụ ở mục 2.2.1.4)

2.2.1.4. Khai báo và sử dụng biến (Variable)

Biến là một đối tượng dùng để lưu trữ giá trị trong vùng nhớ, biến cần được khai báo kiểu dữ liệu trước khi sử dụng.

Có 2 loại biến:

- Biến cục bộ (local variable): do người sử dụng khai báo, bắt đầu bắt kí tự @
- Biến toàn cục (global variable): do hệ thống sử dụng, bắt đầu @@

Cú pháp khai báo biến:

```
DECLARE  { @biến [AS] kiểuDL  [ = giá trị ] }
```

Lưu ý:

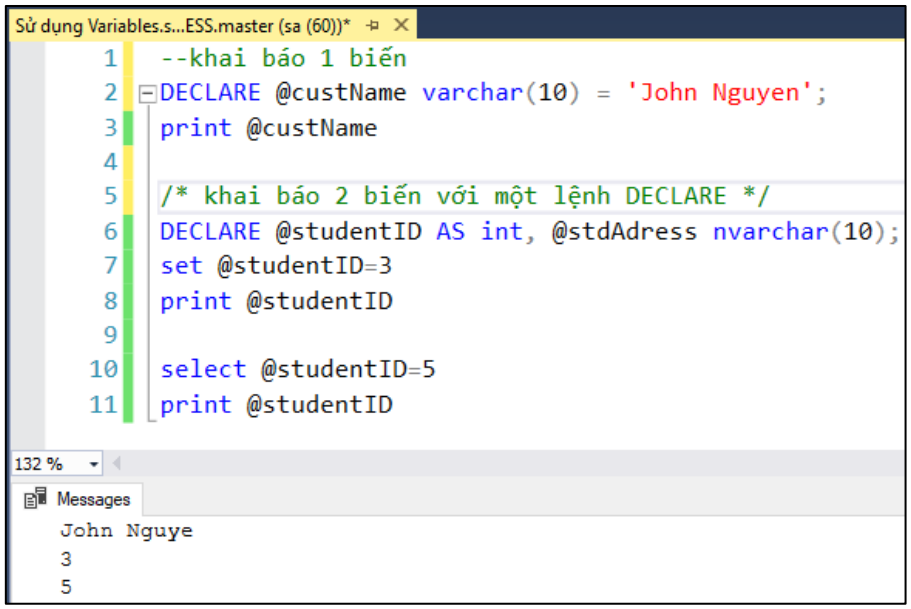
- Giá trị: tùy chọn, mặc định sẽ gán giá trị NULL cho biến.
- Để khai báo nhiều hơn 1 biến cục bộ, thì sử dụng dấu phẩy “,” phân cách các biến.

- Biến kiểu table: lưu giữ nhiều giá trị tạm thời trong quá trình xử lý

Gán giá trị cho biến có thể sử dụng một trong 3 cách sau:

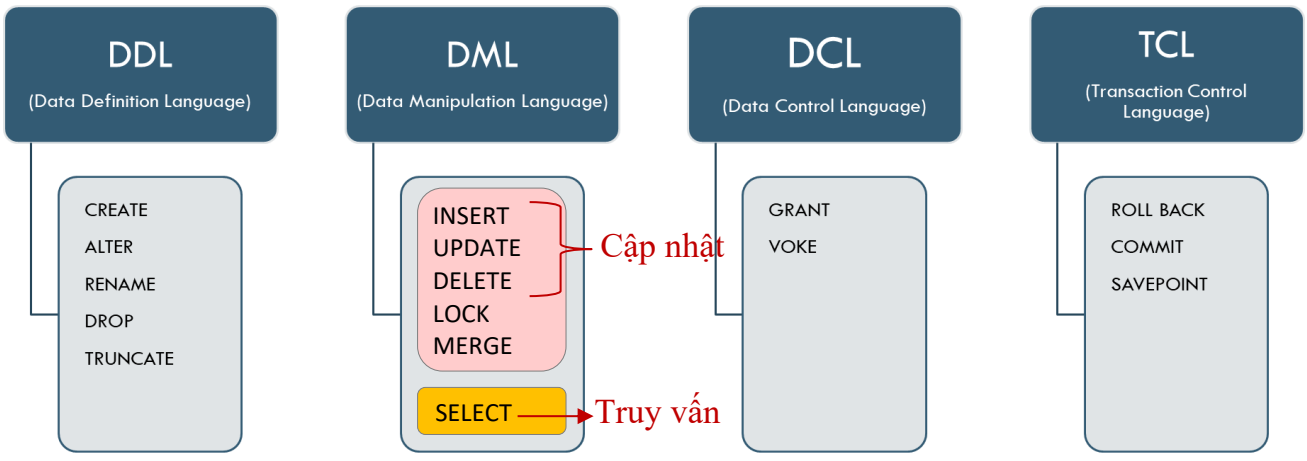
- Trong khi khai báo biến (DECLARE @varName type=…)
- Dùng lệnh SET
- Dùng lệnh SELECT

Ví dụ: Khai báo và gán giá trị cho biến



2.2.1.5. Các nhóm câu lệnh T-SQL

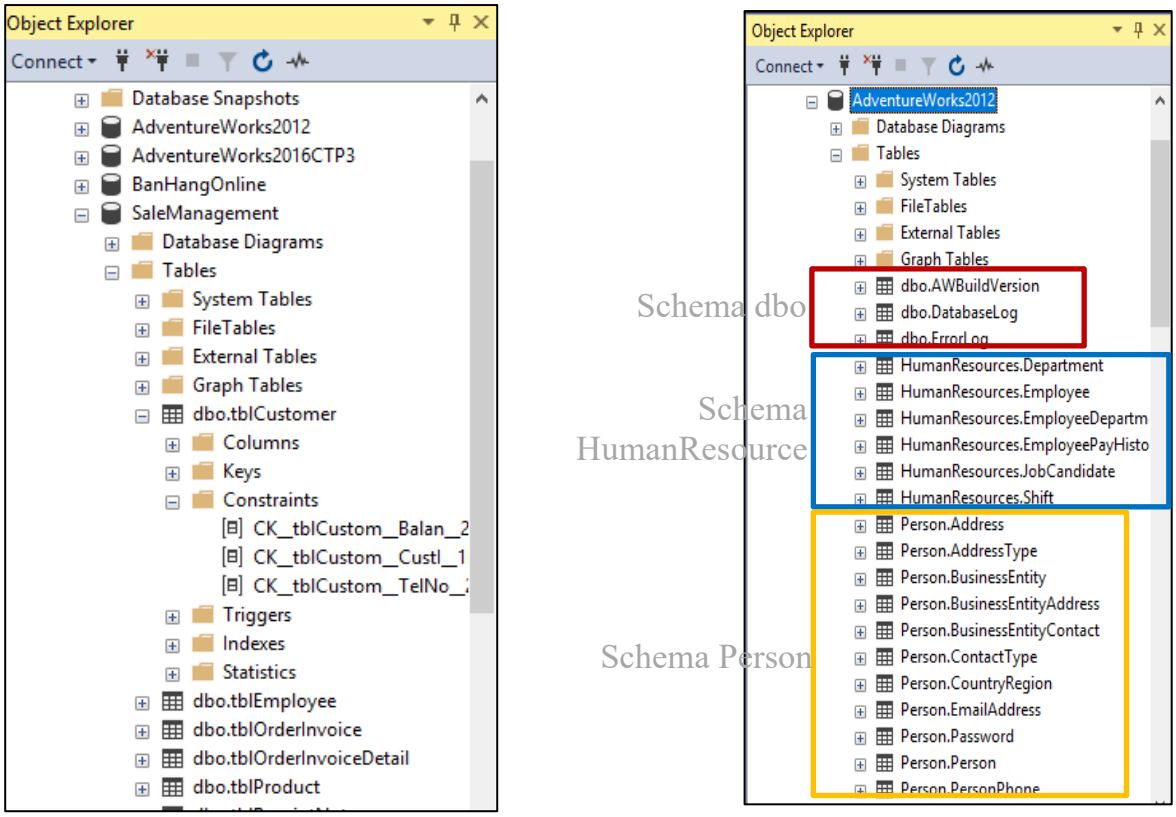
T-SQL gồm rất nhiều câu lệnh (Statement/Command) khác nhau, và thường được phân thành 4 nhóm lệnh sau:



Hình 2.10: Danh mục nhóm các câu lệnh T-SQL

2.2.1.6. Khái niệm Schema

Schema là khái niệm được sử dụng cho phiên bản SQL Server 2005 trở lên. Schema định nghĩa một ranh giới mà trong đó tất cả các tên (của object) là duy nhất. Như vậy, Schema có thể được hiểu là một namespace cho các object trong một database như table, function…, và mặc định là ***dbo***.



Hình 2.11: Minh họa các Schema (dbo mặc định và người dùng tạo mới)

Lợi ích lớn nhất của Schema đó là nhóm các Database Object trong một cơ sở dữ liệu lại với nhau thành nhóm riêng, và việc này giúp quản lý cơ sở dữ liệu dễ dàng hơn. Ví dụ trong cơ sở dữ liệu quản lý bán hàng, nếu muốn có thể tách thành 2 nhánh object riêng gồm: nhánh các object về nhân sự, và nhánh các object về marketing.

Lúc này, ta có thể tạo 2 schema:

- Marketing Schema: chứa các table, Function và Procedure liên quan đến marketing
- HumanResoure Schema: chứa các đối tượng liên quan đến nhân sự

Nếu một cơ sở dữ liệu có nhiều hơn một Schema thì ta có thể tạo 2 hoặc nhiều table có tên giống nhau – với điều kiện chúng phải thuộc 2 hoặc nhiều Schema khác nhau (lưu ý, khi tạo bảng phải chỉ rõ bảng thuộc Schema nào – theo cú pháp <tên_Schema>.<tên_Table>; nếu không mặc định là Schema dbo).

Cú pháp lệnh tạo và xóa Schema:

```
--lệnh tạo Schema
CREATE SCHEMA <tên_Schema>
--lệnh xóa Schema
DROP SCHEMA <tên_Schema>
```

2.2.1.7. Quy tắc sử dụng tên

Trong hệ quản trị cơ sở dữ liệu SQL Server, chúng ta sẽ làm việc thường xuyên với các đối tượng như: colum, table, view, procedure, function, trigger, constraint, transaction. Để xác định rõ table, thông thường tên table thường được thêm cụm từ "tbl" (ví dụ tblEmployee).

Vì vậy, các đối tượng trong cơ sở dữ liệu phải được xác định thông qua tên của nó, và tên phải là duy nhất trong mỗi cơ sở dữ liệu.

Cú pháp đầy đủ để xác định tên trong SQL Server là sử dụng dấu ".":

```
<tên_Schema>.<tên_Table>.<tên_Column>
```

2.2.2. Ngôn ngữ định nghĩa dữ liệu (DDL)

Data Definition Language (DDL), được dùng để định nghĩa (xây dựng), thay đổi hoặc xóa cấu trúc của các đối tượng CSDL.

2.2.2.1. Lệnh CREATE

a) Cú pháp lệnh tạo Table:

```
CREATE TABLE table_name
| database_name.schema_name.table_name | schema_name.table_name
(
   column_name datatype[(size)] [ NULL | NOT NULL ],
   column_name datatype[(size)] [ NULL | NOT NULL ],
   [
      PRIMARY KEY ( col1, col2, ... ) |
      FOREIGN KEY ( col1, col2, ... ) REFERENCES table_2 [ ( col1, col2, ... )
         [ ON DELETE / ON UPDATE]
         [NO ACTION | SET NULL | SET DEFAULT | CASCADE ]
         ]
      ] |
      UNIQUE ( col1, col2, ... ) |
      CHECK ( expression )
   ] ...
```

Ví dụ: (Code SQL cho ERD ở phụ lục - trang 109)

Tạo bảng Supplier và bảng ReceiptNote trong SQL Server dựa trên sơ đồ ERD hình 2.6

```
--tạo bảng Nhà cung cấp
create table tblSupplier
(
   SuppID          char(6) not null primary key,
   SuppName        nvarchar(50) not null,
   SuppAddress     nvarchar(50) null default N'Đà Nẵng',
   TelNo           varchar(11) null unique,
   ContactPerson   nvarchar(50) null
)
--hoặc
create table tblSupplier
(
```

```
    SuppID          char(6) not null,
    SuppName        nvarchar(50) not null,
    SuppAddress     nvarchar(50) null default N'Đà Nẵng',
    TelNo           varchar(11) null unique,
    ContactPerson   nvarchar(50) null
    primary key (SuppID)
 )
--tạo bảng Phiếu nhập
create table tblReceiptNote
(
    RepID           char(7) not null primary key (RepID),
    RepDate         datetime null default getdate()
            check (RepDate <= getdate()),
    RepTotalMoney   money null ,
    SuppNo          char(6) not null foreign key references tblSupplier(SuppID)
 )
--hoặc
create table tblReceiptNote
(
    RepID           char(7) not null primary key (RepID),
    RepDate         datetime null default getdate()
            check (RepDate <= getdate()),
    RepTotalMoney   money null ,
    SuppNo          char(6) not null foreign key (SuppNo) references tblSupplier(SuppID)
 )
--tạo bảng Chi tiết phiếu nhập
create table tblReceiptNoteDetail
(
    RepID           char(7) not null foreign key references tblReceiptNote(RepID),
    ProductID       char(7) not null ,
    primary key (RepID,ProductID),
    QtyReceipted    int not null,
    RepPrice        money not null,
    foreign key (ProductID) references tblProduct(ProductID)
)
```

*Lưu ý: sử dụng lệnh **GO** để chỉ ra sự kết thúc của một đoạn lệnh, và cho phép chạy một bó lệnh (batch) tiếp theo (thường dùng để tạo nhiều table cho một lần ấn F5/Exceute).*

b) Thiết lập ràng buộc quan hệ khi trong lệnh tạo bảng

+ NULL: cho phép NULL (xem 2.3.2)
+ NOT NULL: chỉ ra rằng giá trị của cột không được phép là NULL
+ PRIMARY KEY: trường hợp Simpke Key (xem đoạn lệnh tạo bảng tblSupplier), trường hợp Composite Key (xem đoạn lệnh tạo bảng tblReceiptNoteDetail)
+ UNIQUE: ràng buộc UNIQUE khi muốn cột (hoặc nhiều cột) chứa giá trị duy nhất nhưng không phải là khóa chính, cột UNIQUE có thể chứa giá trị NULL (xem đoạn lệnh tạo bảng tblSupplier, giá trị column "TelNo" không được trùng nhau)

+ DEFAULT: giá trị mặc định được gán cho column khi thêm mới row, trong trường hợp dữ liệu ở column này không được nhập thì giá trị ở column này tự động nhận giá trị DEFAULT này (xem đoạn lệnh tạo bảng tblSupplier)
+ FOREIGN KEY: (xem đoạn lệnh tạo bảng tblReceiptNote)
    - ON DELETE: áp dụng cho trường hợp xóa dữ liệu ở bảng chứa khóa chính
        - Cascade: các dòng dữ liệu ở bảng chứa khóa ngoại bị xóa khi dữ liệu ở bảng chứa khóa chính bị xóa
        - No Action: dữ liệu ở bảng chứa khóa ngoại không bị ảnh hưởng
        - Set NULL: thiết lập giá trị NULL cho cột dữ liệu ở bảng chứa khóa ngoại nếu dữ liệu ở bảng chứa khóa chính bị xóa
        - Set Default: thiết lập giá trị DEFAUT cho cột dữ liệu ở bảng chứa khóa ngoại nếu dữ liệu ở bảng chứa khóa chính bị xóa
    - ON UPDATE: tương tự ON DELETE, áp dụng cho trường hợp cập nhật dữ liệu ở bảng chứa khóa chính
        - Cascade: các dòng dữ liệu ở bảng chứa khóa ngoại bị cập nhật khi dữ liệu ở bảng chứa khóa chính bị cập nhật
        - No Action: dữ liệu ở bảng chứa khóa ngoại không bị ảnh hưởng
        - Set NULL: thiết lập giá trị NULL cho cột dữ liệu ở bảng chứa khóa ngoại nếu dữ liệu ở bảng chứa khóa chính bị cập nhật
        - Set Default: thiết lập giá trị DEFAUT cho cột dữ liệu ở bảng chứa khóa ngoại nếu dữ liệu ở bảng chứa khóa chính bị cập nhật
+ CHECK: kiểm tra tính hợp lệ của dữ liệu khi được nhập (xem đoạn lệnh tạo bảng tblReceiptNote, giá trị column "RepDate" không được vượt quá ngày hiện tại)
+ IDENTITY: dùng để tạo ra cột tự động tăng, phụ thuộc vào giá trị đầu tiên và giá trị được thêm theo cú pháp `IDENTITY[(seed,increment)]` Trong đó: seed là giá trị của dòng đầu tiên được nạp vào bảng, increment giá trị được thêm vào so với giá trị của dòng trước đó. Mặc định seed và increment là 1. Tuy nhiên, hầu hết các PK đều là kiểu dữ liệu string với các ký tự có ý nghĩa của riêng nó, ví dụ như mã sinh viên, mã số tài khoản ngân hàng…

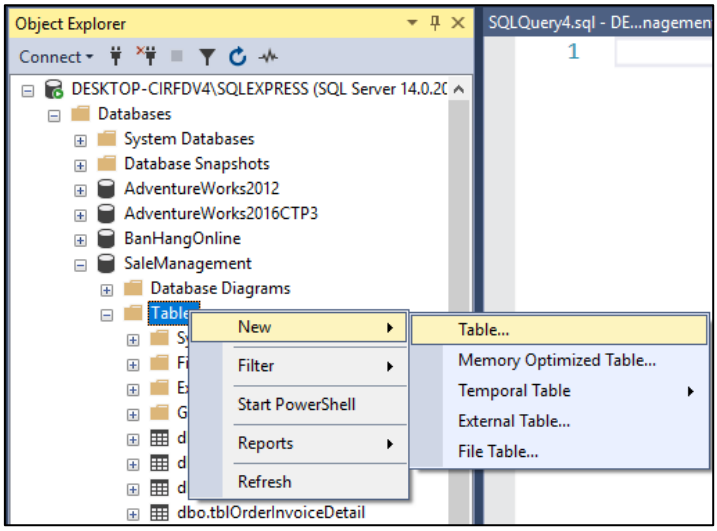
  Cột dữ liệu được khai báo dữ liệu với identity – mặc định sẽ không cho phép người dùng nhập dữ liệu tường minh cho cột này, nếu người dùng vẫn muốn nhập dữ liệu tường minh cho cột này thì phải bật chế độ nhập dữ liệu với câu lệnh sau:

  SET IDENTITY_INSERT talbe_name ON

c) Lưu ý

+ Bảng dữ liệu có chứa Foreign Key – phụ thuộc vào Primary Key ở bảng khác, vì vậy bảng này bắt buộc phải được tạo sau đoạn lệnh tạo bảng có chứa Primary Key đó (ví dụ, bảng tblReceiptNote phải được tạo sau bảng tblSupplier).

+ Ngoài cách tạo bảng bằng câu lệnh T-SQL, bảng còn có thể được tạo bằng design view theo hình dưới:



Hình 2.12: Tạo bảng bằng design view

2.2.2.2. Lệnh ALTER

Dùng để thay đổi cấu trúc của bảng đã tồn tại bao gồm các lệnh: thêm/xóa cột hoặc ràng buộc, thay đổi định nghĩa của cột có sẵn.

Lưu ý, nếu bảng đã tồn tại dữ liệu thì cấu trúc mới của bảng phải phù hợp với dữ liệu có sẵn đó, nếu không buộc phải xóa các dòng dữ liệu không tương thích trước khi sử dụng lệnh ALTER.

a) Cú pháp

```
ALTER TABLE table_name
[{ database_name.schema_name.table_name | schema_name.table_name}]
   ADD new_column_name data_type [column_attribute] |
   DROP COLUMN column_name |
   ALTER COLUMN column_name new_data_type [NULL|NOT NULL] |
   ADD [CONSTRAINT] new_constraint_name |
   DROP [CONSTRAINT] constraint_name
```

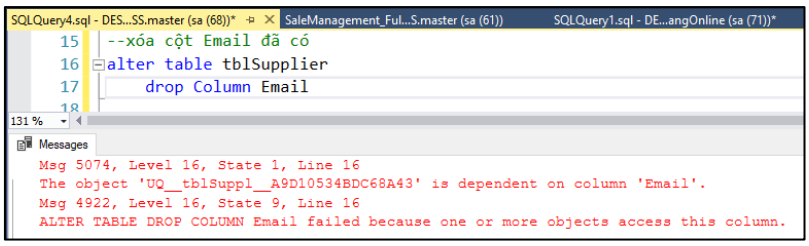
Trong đó, CONSTRAINT là các ràng buộc gồm Primary Key, Foreign Key, Check, Unique, Default.

b) Ví dụ

```
--thêm mới cột Email
alter table tblSupplier
   add Email varchar(30) null unique
--thay đổi cột đã có về kiểu dữ liệu, NULL hoặc NOT NULL
alter table tblSupplier
   alter column Email varchar(50) null
--xóa cột Email đã có
alter table tblSupplier
   drop Column Email
```

```
--thêm mới constraint
alter table tblSupplier
    add constraint email_format check (Email like '[A-Za-z]%@gmail.com')
--thêm mới constraint
alter table tblSupplier
    add constraint email_default default 'quyennth@gmail.com' for Email with values
--thêm mới constraint unique
alter table tblSupplier
    add constraint email_unique unique(Email)
```

c) Lưu ý



Ví dụ trên không thực hiện được bởi vì bảng tblReceiptNote có quan hệ khóa ngoại với bảng tblSupplier.

Với lệnh Drop Column column_name, trước cần phải xóa hết tất cả các ràng buộc liên quan đến column đó.

2.2.2.3. Lệnh DROP

Dùng để xóa bảng ra khỏi cơ sở dữ liệu, khi thực hiện lệnh xóa bảng, nó sẽ xóa tất cả các dữ liệu, triggers, ràng buộc của bảng đó.

a) Cú pháp

```
DROP TABLE table_name
```

b) Ví dụ

```
--xóa bảng tblSupplier
    drop table tblSupplier
```

2.2.2.4. Lệnh TRUNCATE

Lệnh TRUNCATE thực hiện xóa tất cả các rows trong một table hoặc một phân vùng của table, và việc xóa dữ liệu với lệnh TRUNCATE không được ghi lại trong Temporal Table.

Lưu ý:

- Không hỗ trợ xóa theo điều kiện với mệnh đề WHERE
- Nếu table có cột là identity thì identity tự động reset lại giá trị sau khi xóa các row
- Không thể xóa nếu trong table có Foreign Key
- Không hỗ trợ xóa với các VIEW có index

Cú pháp:

```
TRUNCATE TABLE table_name
   [{ database_name.schema_name.table_name | schema_name.table_name}]
[WITH(PARTITIONS { <partition_number_expression> | <range> }
```

```
[ , ...n ] ) ) ];
```

Ví dụ:

```
truncate table tblReceiptNoteDetail
```

Partition Table là chia/tách các table lớn thành nhiều phần nhỏ hơn (không tạo ra table riêng nào). Partition Table có thể chia theo Partition Column (áp dụng với column) hoặc Partition Function (áp dụng với row).

Cách thực hiện:

- Với SSMS:
  - + Tạo File Group: Properties của DB muốn Partition ➔ File Groups (trong Select a page) ➔ Rows ➔ Add "File Groups" – Enter Namev➔ File (trong Select a page) ➔ Add – Enter Name
  - + Tạo Partitioned Table: Right Click Table muốn partition ➔ Storage ➔ Create Partition ➔ thực hiện theo Wizard
- Với T-SQL

```
CREATE PARTITION FUNCTION func_name (variable_list)
   AS RANGE LEFT FOR VALUES (number…) ;
CREATE PARTITION SCHEMA schem_name
   AS PARTITION func_name
   TO (groupfile_name) ;
CREATE TABLE PartitionTable (column_name type [constraint])
ON func_name (parameter_list) ;
```

2.2.3. Ngôn ngữ thao tác dữ liệu (DML)

Data Manipulation Language (DML), được dùng đọc và cập nhật dữ liệu.

2.2.3.1. Cập nhật dữ liệu

2.2.3.1.1. *Lệnh INSERT*

Để thêm một hoặc nhiều dòng vào một bảng, ta sử dụng câu lệnh INSERT.

a) Cú pháp:

```
INSERT INTO table_name [(column_list)]
VALUES (value_list);
```

Hoặc (khi muốn chèn nhiều dòng với một lệnh INSERT INTO

```
INSERT INTO table_name [(column_list)]
VALUES (value_list_1),
       (value_list_2),
       (value_list_3),
       ...
       (value_list_n);
```

Hoặc (khi muốn chèn vào table_2 với kết quả select từ table_1)

```
INSERT INTO table_2 (column1, column2, column3,...)
SELECT column1, column2, column3,...
FROM table_1
WHERE condition;
```

b) Ví dụ

```
--chèn row có chỉ định column_list
insert into tblSupplier(SuppID,SuppName)
values('NCC001','Vietgap Bean')
--hoặc
insert into tblSupplier (SuppID,SuppName)
select 'NCC006',N'Thế giới di động'
--chèn row không chỉ định column_list
insert into tblSupplier
values('NCC002',N'Sư Phạm Kỹ Thuật',N'Đà Nẵng','02363822571',N'N.T. Hà Quyên')
--chèn từ kết quả câu select
insert into tblSupplier (SuppID,SuppName)
select EmpID,EmpName
from tblEmployee
where Gender='F'
```

c) Lưu ý

+ Danh sách cột (column_list) không nhất thiết phải chỉ định, khi đó giá trị các trường của bản ghi mới phải được chỉ định đầy đủ trong danh sách giá trị (values_list). Trong trường hợp này, thứ tự các giá trị trong danh sách values_list phải tương ứng theo đúng thứ tự của các trường như khi bảng được định nghĩa.
+ Trong trường hợp chỉ nhập giá trị cho một số cột trong bảng, ta phải chỉ định danh sách các cột cần nhập dữ liệu ngay sau tên bảng. Khi đó, các cột không xuất hiện trong column_list sẽ nhận giá trị:
  - Giá trị NULL nếu cột cho phép NULL
  - Giá trị tăng tiếp theo nếu cột chứa thuộc tính IDENTITY
  - Giá trị mặc định nếu cột có thuộc tính DEFAULT
  - Giá trị đã tính toán nếu cột là một cột được tính toán.
  - Giá trị timestamp hiện tại nếu kiểu giá trị của cột là timestamp (xem 2.4.1.1)
+ Ngược lại, nếu các cột không thuộc các trường hợp trên mà không được nhập dữ liệu thì câu lệnh INSERT sẽ gây lỗi.

2.2.3.1.2. *Lệnh UPDATE*

Để cập nhật dữ liệu đang có trong bảng, ta sử dụng lệnh UPDATE.

a) Cú pháp

```
UPDATE table_name
SET column_name1 = expression1 [,column_name2 = expression2, ...]
[FROM table_source [[AS] table_alias] ]
```

```
[WHERE condition]
```

b) Ví dụ

```
--cập nhật cột SuppAddress thành 'Bình Sơn, Quảng Ngãi' cho dòng có SuppID='NCC002'
update tblSupplier
set SuppAddress=N'Bình Sơn, Quảng Ngãi'
   where SuppID='NCC002'
```

c) Lưu ý

+ Mệnh đề WHERE dùng để chỉ định điều kiện mà các dòng phải thỏa mãn mới được cập nhật, nếu bỏ qua mệnh đề WHERE, tất cả các dòng trong bảng sẽ được cập nhật.
+ Mệnh đề FROM được sử dụng khi cần chỉ định các điều kiện liên quan đến các bảng khác với bảng cần cập nhật dữ liệu. Trong trường hợp này, trong mệnh đề WHERE thường có điều kiện nối giữa các bảng.

2.2.3.1.3. *Lệnh DELETE*

Để xoá dữ liệu trong một bảng, ta sử dụng câu lệnh DELETE.

a) Cú pháp

```
DELETE [TOP (expression) [percent]] [FROM] table_alias
[FROM table_source]
[WHERE condition]
```

b) Ví dụ

```
--xóa dòng có SuppID='NCC002'
delete  tblSupplier
where SuppID='NCC002'
--hoặc
delete  from tblSupplier
where SuppID='NCC002'
--xóa những row trong tblReceiptNoteDetail mà phụ thuộc vào điều kiện ở tblReceiptNote
delete tblReceiptNoteDetail
from tblReceiptNote
   where RepTotalMoney<=800000 and tblReceiptNoteDetail.RepID=tblReceiptNote.RepID
```

c) Lưu ý

+ Mệnh đề WHERE trong câu lệnh được sử dụng để chỉ định điều kiện đối với các dòng dữ liệu cần xoá. Nếu câu lệnh DELETE mà không có mệnh đề WHERE thì toàn bộ các dòng dữ liệu trong bảng đều bị xoá.
+ Nếu điều kiện trong câu lệnh DELETE liên quan đến các bảng không phải là bảng cần xóa dữ liệu, ta phải sử dụng thêm mệnh đề FROM và sau đó là danh sách tên các bảng đó. Trong trường hợp này, trong mệnh đề WHERE ta chỉ định thêm điều kiện nối giữa các bảng.

2.2.3.2. Truy vấn dữ liệu

2.2.3.2.1. *Lệnh SELECT*

Câu lệnh SELECT được sử dụng để truy xuất dữ liệu từ các dòng và các cột của một hay nhiều bảng, khung nhìn.

a) Cú pháp

```
SELECT [ALL|DISTINCT] [TOP n] column_list
FROM table_list
[WHERE conditional_list]
[GROUP BY column_list]
[HAVING conditional_list]
[ORDER BY column_list];
```

b) Ví dụ

```
--hiển thị tất cả các phiếu nhập
select * from tblReceiptNote
--hiển thị các phiếu nhập của NCC001 – đặt bí danh cho cột RepTotalMoney
select RepID, RepDate,RepTotalMoney as "Tổng tiền"
from tblReceiptNote
where SuppNo='NCC001'
--sử dụng bí danh cho bảng
select pn.RepID, ct.ProductID, ct.RepPrice, ct.RepPrice, pn.RepDate
from tblReceiptNote as pn, tblReceiptNoteDetail as ct
where pn.RepID=ct.RepID
--sử dụng TOP
select top 3 *
from tblReceiptNote
--sử dụng DISTINCT
select distinct SuppNo
from tblReceiptNote
```

c) Lưu ý

+ **Mệnh đề SELECT:**
  - Chỉ định rõ tên các column cần lấy dữ liệu, hoặc dùng ký tự '*' ~~hoặc ALL~~ trong trường hợp muốn lấy tất cả các column trong bảng
  - Loại bỏ dữ liệu trùng nhau trong kết quả truy vấn bằng cách dùng từ khóa DISTINCT ngay sau từ khóa SELECT
  - Select Top N mặc định chỉ lấy đúng N dòng đầu tiên. Nếu có nhiều dòng cùng giá trị thì cần dùng sub-query để lấy N mức giá trị thỏa mãn. Nếu chỉ lấy 1 mức đầu tiên thỏa mãn, thì dùng "WITH TIES" để thay cho subquery
  - Giới hạn số lượng dòng trong kết quả truy vấn bằng cách thêm từ khóa TOP ngay trước danh sách cột của câu lệnh SELECT. Thường dùng kết hợp với Order by (xem 2.4.3.2.1.d)
  - Sử dụng các hàm tính toán như SUM, MAX, MIN, AVG, COUNT…
+ **Mệnh đề FROM** được sử dụng nhằm chỉ định các bảng và khung nhìn cần truy xuất dữ liệu, danh sách này được cách nhau bởi dấu phẩy.

+ **Mệnh đề WHERE:**
  - Toán tử số học: +, - , *, / , %
  - Toán tử so sánh: =, !=, <>, > , <, >=, <=, !<, !>
  - Toán tử logic: ANY, ALL, AND, OR, BETWEEN, EXISTS, IN, LIKE, NOT, IS NULL, UNIQUE
    - ANY, ALL: ANY trả về giá trị TRUE nếu có bất kỳ giá trị truy vấn phụ nào đáp ứng điều kiện; ALL trả về giá trị TRUE nếu tất cả các giá trị truy vấn con đáp ứng điều kiện.
    - AND, OR, NOT: dùng khi cần kết hợp nhiều điều kiện
    - IN, NOT IN, CONTAINS, NOT CONTAINS: quan hệ bao hàm
    - LIKE, NOT LIKE: so sánh giá trị, sử dụng ký tự đại diện (wildcard). Dấu "%" để đại diện cho một cụm từ bất kỳ; dấu "_" để đại diện cho một ký tự bất kỳ
    - BETWEEN, NOT BETWEEN: kiểm tra giới hạn miền dữ liệu
    - EXISTS, NOT EXITS: kiểm tra điều kiện tồn tại
    - UNION (hợp), INTERSECT (Giao), EXCEPT (Hiệu): các toán tử tập hợp
    - NULL, NOT NULL: kiểm tra NULL hay không
  - Biểu thức sau WHERE không được sử dụng các hàm tính toán SUM, MAX, MIN, AVG, COUNT… trừ phi các hàm tính toán này nằm trong truy vấn con ở mệnh đề HAVING

+ Có thể sử dụng bí danh (Alias, hay còn gọi là tên tạm thời và chỉ có ý nghĩa trong một câu lệnh select này) cho các cột, bảng hoặc khung nhìn trong câu lệnh SELECT. Nếu bí danh chứa khoảng trắng thì phải đặt trong cặp dấu trích dẫn "…"

d) Sắp xếp kết quả truy vấn với ORDER BY

Mặc định, các dòng dữ liệu trong kết quả của câu truy vấn được hiển thị theo thứ tự của index (Primary Key column) trong bảng dữ liệu.

Để dữ liệu được sắp xếp theo chiều tăng hoặc giảm của giá trị của một hoặc nhiều trường, ta sử dụng thêm mệnh đề ORDER BY trong câu lệnh SELECT

Sau ORDER BY là danh sách các cột cần sắp xếp (tối đa là 16 cột). Dữ liệu được sắp xếp có thể theo chiều tăng (ASC – Ascending, mặc định) hoặc giảm (DESC – Descending).

Có thể dùng tên cột hoặc số thứ tự của cột trong bảng để sắp xếp. Thứ tự sắp xếp được tính từ trái sang phải nếu có nhiều cột cần sắp xếp.

Ví dụ:

```
--sử dụng TOP và ORDER BY
select top 3 *
from tblReceiptNote
```

```
order by RepTotalMoney desc,
```

e) Nhóm dữ liệu với GROUP BY

Nhóm dữ liệu là quá trình tổ hợp các cột có các giá trị giống nhau theo một thứ tự logic, nhóm dữ liệu được thực hiện thông qua mệnh đề GROUP BY trong câu lệnh SELECT.

Mệnh đề GROUP BY đứng sau mệnh đề WHERE trong câu lệnh SELECT và đứng trước mệnh đề ORDER BY

Ví dụ:

```
--sử dụng group by
select SuppNo as "Mã nhà cung cấp", sum(RepTotalMoney)
from tblReceiptNote
group by SuppNo
```

f) Mệnh đề HAVING

Mệnh đề HAVING được sử dụng khi câu lệnh truy vấn SELECT có sử dụng nhóm dữ liệu với GROUP BY. Mệnh đề HAVING thường đi với GROUP BY và đứng trước ORDER BY (nếu có) trong một câu truy vấn.

Mệnh đề HAVING cho phép xác định các điều kiện để lọc nhóm kết quả nào sẽ xuất hiện trong kết quả cuối cùng. Mệnh đề HAVING đặt các điều kiện (conditional_list) trên các nhóm đã được tạo bởi mệnh đề GROUP BY (trong khi mệnh đề WHERE đặt các điều kiện trên các cột đã lựa chọn).

Như vậy, HAVING đối với GROUP BY cũng giống như WHERE đối với SELECT.

Ví dụ:

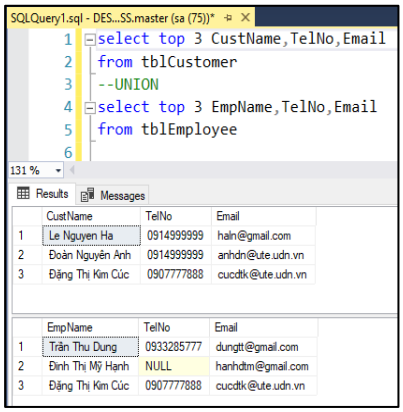
```
--sum(QtyReceipted) có được sau khi đã group by theo RepID
select RepID, sum(RepPrice)
from tblReceiptNoteDetail
group by RepID
having sum(QtyReceipted) < 20

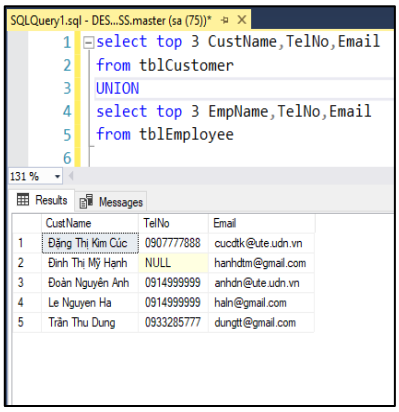
--hiển thị những loại vị trí nhân viên, là nữ có lương trung bình hơn 5.000.000
select Position,AVG(Salary)
from tblEmployee
where Gender='F'
group by Position
having AVG(Salary)>5000000
```

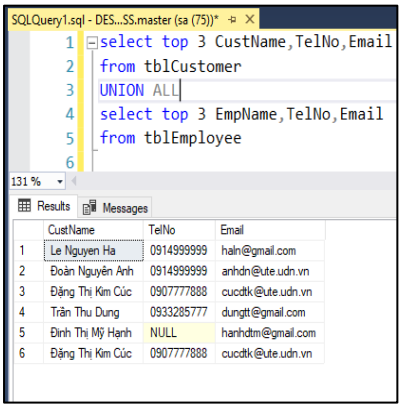
2.2.3.2.2. *Phép hợp*

a) Phép hợp UNION

3 kết quả minh họa với câu lệnh select riêng, sử dụng phép hợp UNION và UNION ALL.







Phép hợp UNION được sử dụng trong trường hợp ta cần gộp kết quả của hai hay nhiều truy vấn thành một tập kết quả duy nhất. Điều kiện để thực hiện hợp dữ liệu từ hai truy vấn trở lên:

- Colum_list trong các truy vấn SELECT phải có cùng số lượng và có thứ tự.
- Các cột phải có cùng kiểu dữ liệu.

Sử dụng từ khóa UNION để kết hợp tập kết quả từ hai hay nhiều câu lệnh SELECT.

Cú pháp:

```
SELECT column_list FROM table1
UNION [ALL]
SELECT column_list FROM table2;
```

Lưu ý:

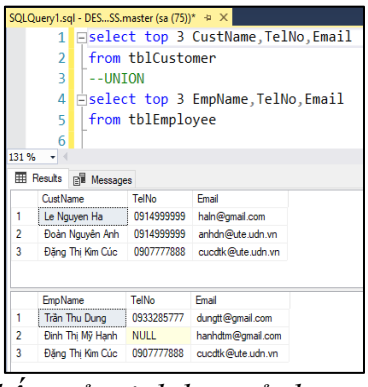
Mặc định, từ khóa UNION chỉ lấy các giá trị khác nhau, để cho phép lấy các giá trị trùng nhau thì sử dụng từ khóa UNION ALL.

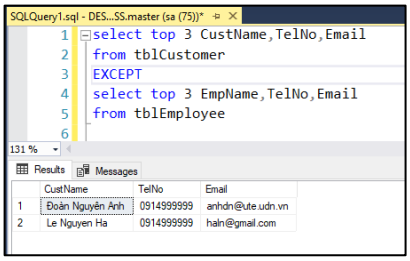
Ví dụ:

```
--lấy tất cả tên, số điện thoại và email của tblCustomer và tblEmployee
select CustName,TelNo,Email
from tblCustomer
UNION
select EmpName,TelNo,Email
from tblEmployee
```

b) Phép hợp EXCEPT (MINUS)

EXCEPT là phép hợp được sử dụng giữa 2 kết quả truy vấn, trong đó chỉ trả về các bản ghi ở của truy_vấn_1, loại trừ đi những bản ghi ở truy_vấn_2.

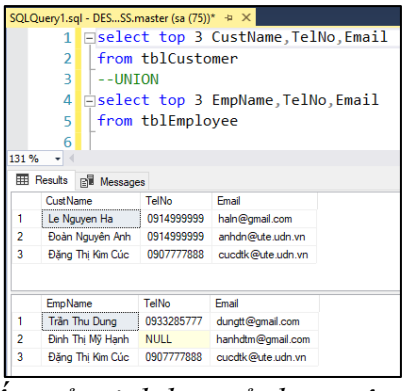


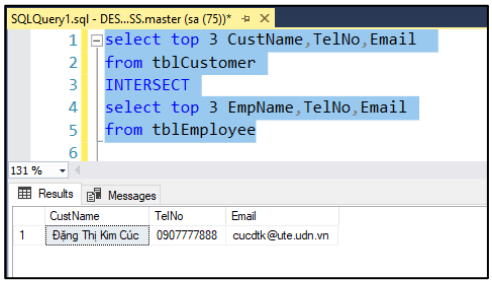


2 kết quả minh họa sử dụng câu select riêng và sử dụng phép EXCEPT.

c) Phép hợp INTERSECT

INTERSECT là phép hợp trả về những bản ghi chung giữa 2 tập kết quả truy vấn.





2 kết quả minh họa sử dụng câu select riêng và sử dụng phép INTERSECT.

2.2.3.2.3. *Phép nối (JOIN)*

Phép nối được dùng để kết hợp các dòng dữ liệu giữa các bảng, bằng cách kết hợp các dòng thoả mãn điều kiện (conditional_list) thành những dòng trong kết quả truy vấn.

Như vậy, khi cần thực hiện truy vấn SELECT từ nhiều bảng dữ liệu có liên quan với nhau ➔ ta cần dùng phép nối.

Khi thực hiện lệnh nối bảng, ta phải xác định rõ 3 yếu tố sau:

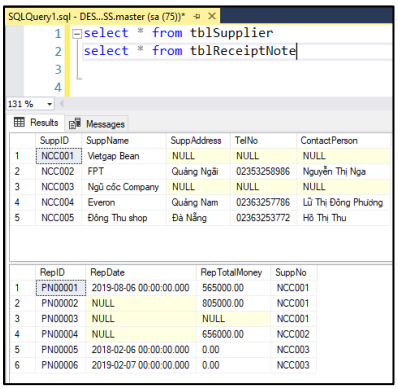
+ Những cột nào cần hiển thị trong kết quả truy vấn? (***SELECT what?***)
+ Những bảng nào có tham gia vào truy vấn? (***FROM tables_name?)***
+ Conditional_list để thực hiện phép nối giữa các bảng dữ liệu? (***WHERE/ON conditional_list***)

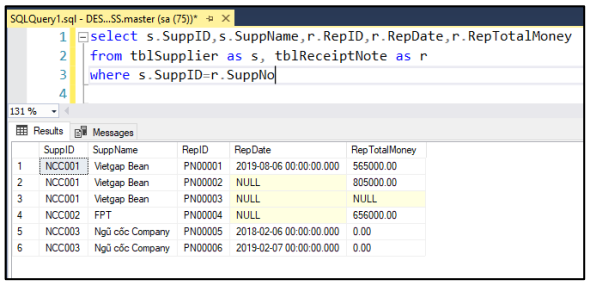
Các toán tử có thể sử dụng dùng để nối các bảng gồm: =, < , >, < >, <=, >=, !=, BETWEEN, LIKE, NOT (thông dụng nhất là toán tử =).

Các loại phép nối:

i. Nối bằng mệnh đề WHERE

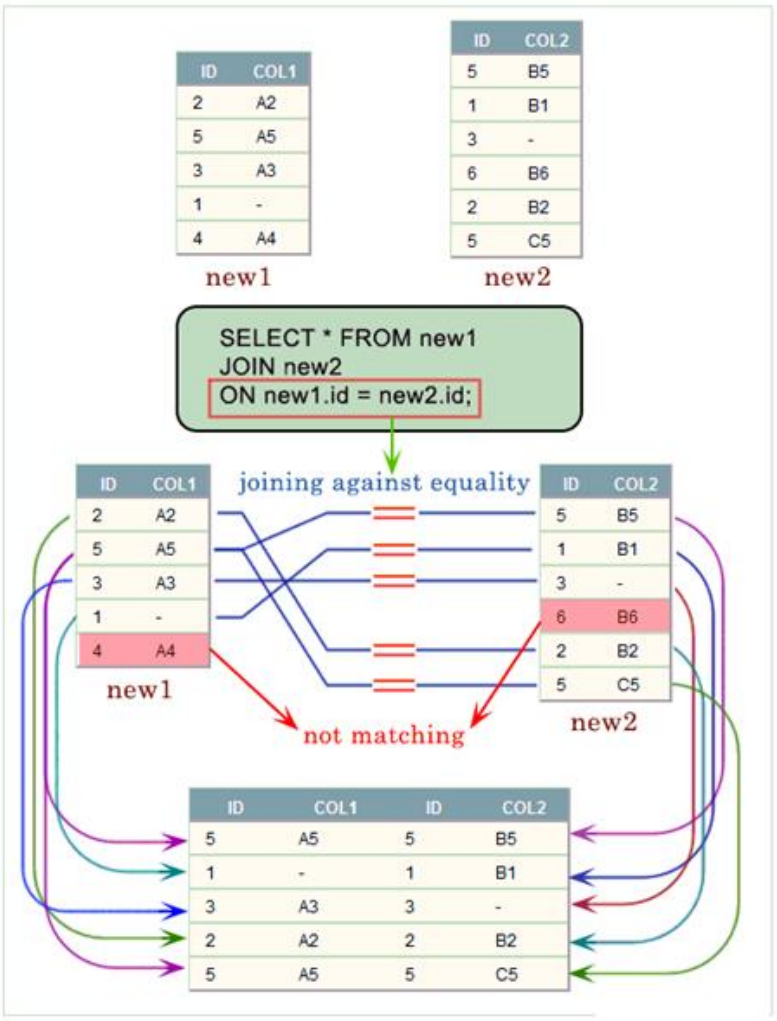
2 kết quả minh họa câu select riêng, và phép nối (Join) giữa tblSupplier và tblReceiptNote





+ Phép nối bằng (Equijoin): sử dụng toán tử '='
+ Phép nối không bằng (Non Equijoin): sử dụng các toán tử khác toán tử '='
+ Phép tự nối (SELF JOIN) là phép nối có các conditional_list chỉ liên quan đến các cột của cùng một bảng.

ii. Nối bằng mệnh đề JOIN



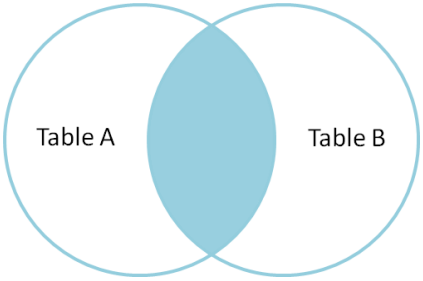
Hình 2.13: Minh họa thực hiện phép nối bằng

+ INNER JOIN: trả về kết quả là tập các bản ghi được join theo các trường (field) ở hai bảng khớp nhau, các bản ghi chỉ xuất hiện ở một trong hai bảng sẽ bị loại.
+ OUTER JOIN:
  - LEFT/RIGHT JOIN: các bản ghi chỉ xuất hiện ở một trong hai bảng
  - FULL JOIN: trả về kết quả là các bản ghi có mặt trong cả hai bảng
+ CROSS JOIN: trả về tích Đề-các giữa hai bảng (số bản ghi trả về bằng tích của số bản ghi trong hai bảng)

Cú pháp câu lệnh SELECT có phép nối JOIN vẫn có thể gồm đầy đủ các mệnh đề WHERE, HAVING, GROUP BY, ORDER BY.

a) Phép nối INNER JOIN

Đây là phép nối đơn giản nhất hay còn gọi là phép nối bằng, sử dụng từ khóa INNER JOIN, phép nối này dùng để lấy tất cả các dòng từ cả 2 bảng thỏa mãn điều kiện.



Hình 2.14: Minh họa tập các bản ghi kết quả của phép InnerJoin

Phép nối INNER JOIN có tính đối xứng, nghĩa là A Join B cũng có kết quả tương tự như B Join A.

Cú pháp:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
[INNER] JOIN table2
  ON table1.matching_column = table2.matching_column;
```

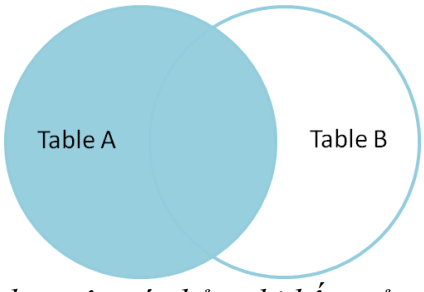
Ví dụ:

```
select r.RepID,r.RepDate,r.RepTotalMoney,s.SuppID,s.SuppAddress
from tblReceiptNote as r
   inner join tblSupplier s
         on s.SuppID=r.SuppNo

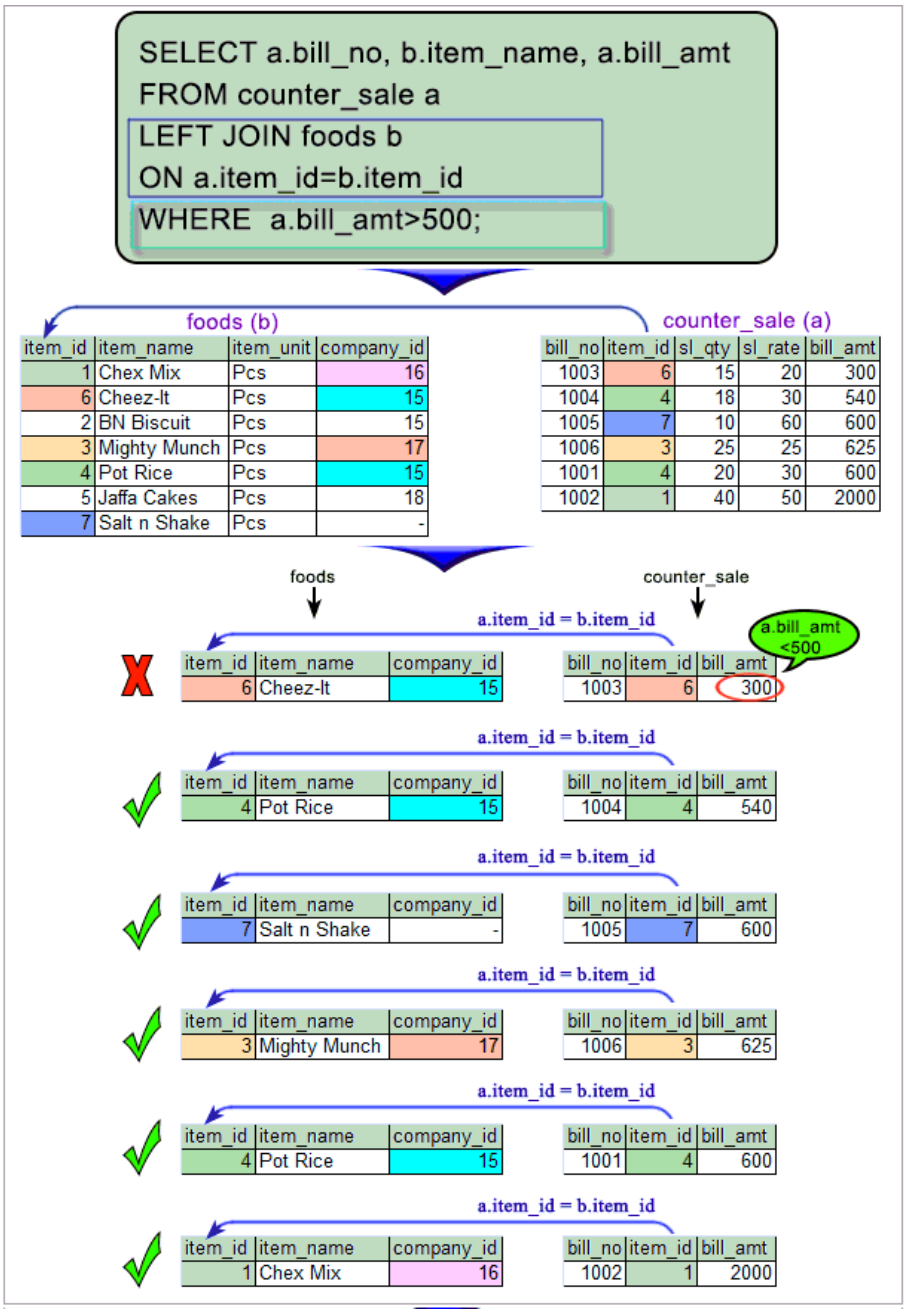
--kết quả select giống đoạn lệnh ngay trên
select r.RepID,r.RepDate,r.RepTotalMoney,s.SuppID,s.SuppAddress
from tblSupplier as s
   inner join tblReceiptNote r
         on s.SuppID=r.SuppNo
```

b) Phép nối LEFT JOIN

Trả về tất cả bản ghi của bảng bên trái, và những bản ghi khớp với bản ghi của bảng bên phải, nếu không có dữ liệu khớp sẽ nhận NULL



Hình 2.15: Minh họa tập các bản ghi kết quả của phép Left Join

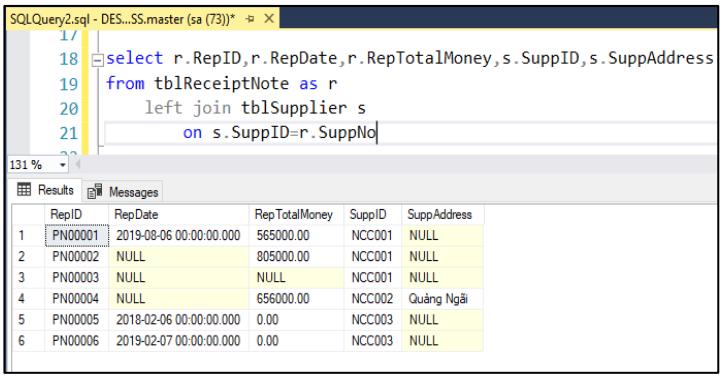


Hình 2.16: Minh họa thực hiện phép nối JOIN (LEFT JOIN)

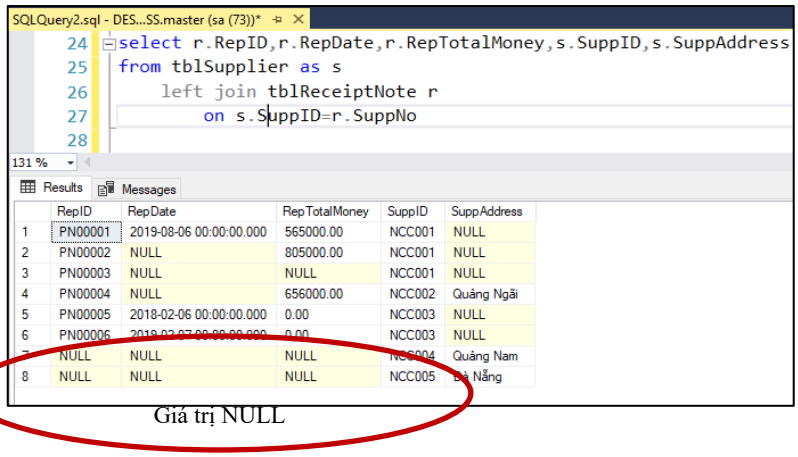
Cú pháp:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT [OUTER] JOIN table2
ON table1.matching_column = table2.matching_column;
```

Ví dụ minh họa kết quả khác nhau cho 2 đoạn lệnh sử dụng phép Left Join khi hoán trật tự table.



	RepID	RepDate	RepTotalMoney	SuppID	SuppAddress
1	PN00001	2019-08-06 00:00:00.000	565000.00	NCC001	NULL
2	PN00002	NULL	805000.00	NCC001	NULL
3	PN00003	NULL	NULL	NCC001	NULL
4	PN00004	NULL	656000.00	NCC002	Quảng Ngãi
5	PN00005	2018-02-06 00:00:00.000	0.00	NCC003	NULL
6	PN00006	2019-02-07 00:00:00.000	0.00	NCC003	NULL

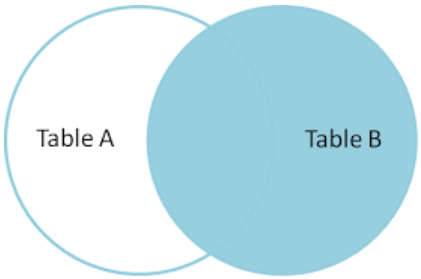


	RepID	RepDate	RepTotalMoney	SuppID	SuppAddress
1	PN00001	2019-08-06 00:00:00.000	565000.00	NCC001	NULL
2	PN00002	NULL	805000.00	NCC001	NULL
3	PN00003	NULL	NULL	NCC001	NULL
4	PN00004	NULL	656000.00	NCC002	Quảng Ngãi
5	PN00005	2018-02-06 00:00:00.000	0.00	NCC003	NULL
6	PN00006	2019-02-07 00:00:00.000	0.00	NCC003	NULL
7	NULL	NULL	NULL	NCC004	Quảng Nam
8	NULL	NULL	NULL	NCC005	Đà Nẵng

Giá trị NULL

c) Phép nối RIGHT JOIN

Trả về tất cả bản ghi của bảng bên phải, và những bản ghi khớp với bản ghi của bảng ghi bên trái, nếu không có dữ liệu khớp sẽ nhận NULL.



Hình 2.17: Minh họa tập các bản ghi kết quả của phép Right Join

Cú pháp:

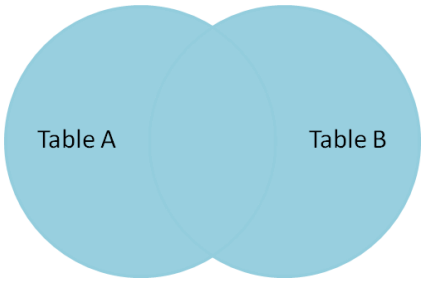
```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
RIGHT [OUTER] JOIN table2
ON table1.matching_column = table2.matching_column;
```

Ví dụ:

```
select r.RepID,r.RepDate,r.RepTotalMoney,s.SuppID,s.SuppAddress
from tblReceiptNote as r
    right join tblSupplier s
            on s.SuppID=r.SuppNo
-- A Left Join B, có kết quả giống B Right Join A
select r.RepID,r.RepDate,r.RepTotalMoney,s.SuppID,s.SuppAddress
from tblSupplier as s
    left join tblReceiptNote r
            on s.SuppID=r.SuppN
```

d) Phép nối FULL JOIN

Phép nối này tạo ra tập kết quả bằng cách kết hợp kết quả của 2 lệnh LEFT JOIN và RIGHT JOIN, trả về tất cả bản ghi của bảng bên trái và bảng bên phải, những bản ghi nào không khớp sẽ nhận giá trị NULL.



Hình 2.18: Minh họa tập các bản ghi kết quả của phép Full Join

Cú pháp:

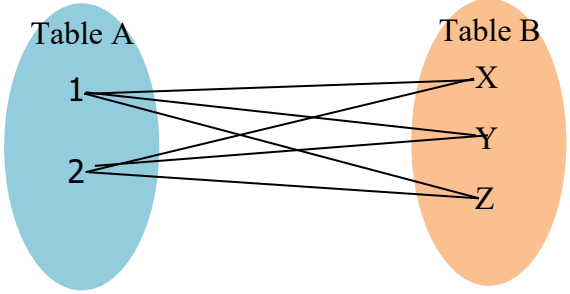
```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
FULL [OUTER] JOIN table2
ON table1.matching_column = table2.matching_column;
```

Ví dụ:

```
-- c có 5 row, e có 15 row ➔ kết quả phép full join mặc định tạo ra 20 row
-- tuy nhiên, c và e có 1 row trùng nhau nên kết quả chỉ còn 19 row
select c.CustID,c.CustName,c.CustAddress,e.EmpID,e.EmpName,e.Position
from tblCustomer as c
    full join tblEmployee e
            on c.CustName=e.EmpName
```

e) Phép nối CROSS JOIN

Phép nối này tạo ra tập kết quả là tích Đề-các (Descartes), mỗi bản ghi của bảng này được kết hợp với tất cả các bản ghi của bảng kia và ngược lại.



Hình 2.19: Minh họa tập các bản ghi kết quả của tích Đề-các

Cú pháp:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
CROSS JOIN table2
```

Ví dụ:

```
-- c có 5 row, e có 15 row ➔ kết quả phép cross join tạo ra 75 row
select c.CustID,c.CustName,c.CustAddress,e.EmpID,e.EmpName,e.Position
from tblCustomer as c
    cross join tblEmployee e
```

2.2.3.2.4. *Truy vấn con (Subquery/ Nesting Query)*

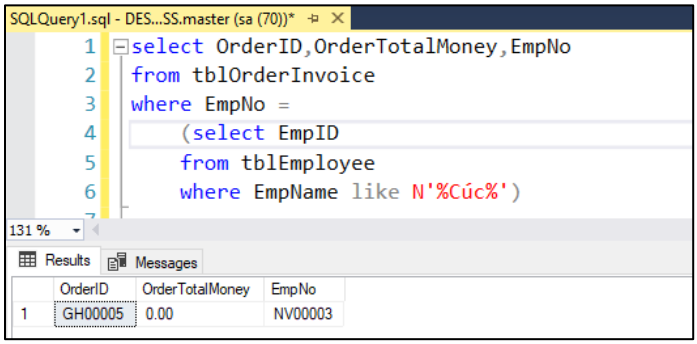
Truy vấn con, còn được gọi là truy vấn lồng, là một câu lệnh SELECT được lồng vào bên trong một câu lệnh SELECT, INSERT, UPDATE, hoặc DELETE.

Ví dụ:

```
--hiển thị các nhà cung cấp mà có tổng tiền các phiếu nhập > 600000
select SuppID,SuppName,SuppAddress
from tblSupplier
where SuppID in
    (select SuppNo
    from tblReceiptNote
    where RepTotalMoney > 600000)
--hiển thị các nhà cung cấp mà có tổng tiền các phiếu nhập > 600000 và tên có chứa ký tự 'B'
select SuppID,SuppName,SuppAddress
from tblSupplier
where SuppName like '%B%' and exists
    (select SuppNo
    from tblReceiptNote
    where RepTotalMoney > 600000)
```

Truy vấn con thường được sử dụng khi cần truy vấn mà trong đó điều kiện truy vấn dữ liệu cần phải sử dụng đến kết quả của một truy vấn khác, với 2 khái niệm sau:

- Outer query: là truy vấn chính cần hiển thị
- Subquery/Inner query: là truy vấn trả về tập kết quả cho outer query sử dụng. Subquery có thể trả về một trong các giá trị thuộc 3 loại sau
    i. Single value: chỉ trả về giá trị vô hướng, 1 giá trị duy nhất hoặc giá trị NULL nếu subquery không trả về giá trị nào. Subquery này còn được gọi là scalar subquery.

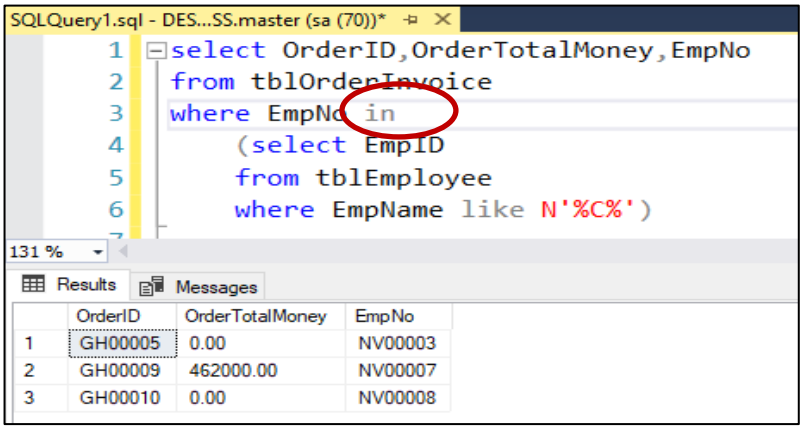


Ví dụ minh họa Scalar Subquery, kết quả lỗi vì subquery chỉ trả về hơn 1 giá trị.

Scalar Subquery đi sau các toán tử gồm: =, !=, <, <=, >, >=, EXISTS, LIKE hoặc BETWEEN (nếu kết quả chỉ trả về 1 giá trị)

ii. Multiple value: trả về nhiều giá trị dạng single_column hoặc table. Subquery này được gọi là multivalued subquery.

Multivalued Subquery đi sau các toán tử: IN, NOT IN, SOME, ANY, ALL, LIKE hoặc BETWEEN (trong trường hợp trả về nhiều giá trị)



Ví dụ minh họa Multivalued Subquery bởi toán tử IN.

Lưu ý:

+ Subquery thường được sử dụng trong mệnh đề WHERE, hoặc mệnh đề HAVING của outer subquery.

Cú pháp subquery trong mệnh đề WHERE:

```
SELECT column_name [,column_name2,...]
FROM table [,table 2]
WHERE column_name|OPERATOR
        (SELECT column_name
         FROM table
         WHERE conditions);
```

Cú pháp subquery trong mệnh đề HAVING:

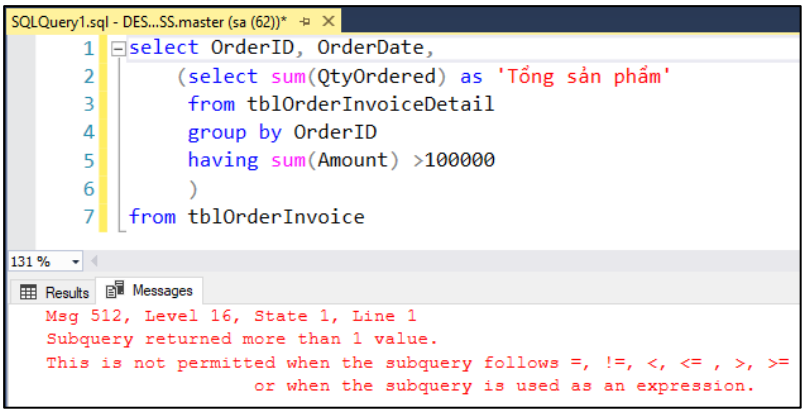
```
SELECT column_name [,column_name2,...]
FROM table [,table 2]
[WHERE conditional_list]
HAVING column_name OPERATOR
        (SELECT column_name
         FROM table
         WHERE conditions);
```

+ Subquery có thể được sử dụng trong mệnh đề SELECT của outer query, tuy nhiên subquery bắt buộc phải là dạng Scalar Query – thường dùng với các hàm SUM, COUNT, MIN, MAX…

Cú pháp subquery trong mệnh đề SELECT:

```
SELECT column_name [,column_name2,...],
        [coumn_name=](SELECT column_name|expression(column)
         FROM table_list2
         WHERE conditional_list)
FROM table_list1
WHERE condition_list;
```

Ví dụ minh họa subquery trong mệnh đề SELECT phải là scalar query.



+ Không cần sử dụng đặc tả DISTINCT trong subquery vì mặc định Database Engine đã tự loại bỏ những dữ liệu duplicate.
+ Toán tử BETWEEN không được dùng cùng với câu truy vấn con nhưng có thể được dùng bên trong câu vấn tin con

2.3. Khung nhìn (VIEW)

Một khung nhìn có thể được xem như là một bảng ảo (virtual table) trong cơ sở dữ liệu, có nội dung được định nghĩa thông qua một truy vấn (câu lệnh SELECT). Như vậy, ta có thể sử dụng view_name như table_name trong câu lệnh SQL.

Điểm khác biệt giữa khung nhìn và bảng là, khung nhìn không được xem là một cấu trúc lưu trữ dữ liệu tồn tại trong cơ sở dữ liệu. Thực chất dữ liệu quan sát được trong khung nhìn được lấy từ các bảng thông qua câu lệnh truy vấn dữ liệu.

View có thể được sử dụng như một cơ chế tăng tính bảo mật của database, nghĩa là ngăn người dùng truy cập trực tiếp vào row và column của các table mà view dựa vào bằng cách cho phép cấp phát các quyền trên View.

a) Cú pháp

Cú pháp tạo VIEW:

```
CREATE VIEW  view_name
  [WITH SCHEMABINDING]
AS
SELECT * | column1 [,column2,...]
FROM table_name
[WHERE expression1 [,expression2]...]
[WITH CHECK OPTION]
[GROUP BY]
```

Cú pháp thay đổi VIEW:

```
ALTER VIEW view_name [(column_commalist)] AS
Query_statement [WITH READ ONLY]| [WITH [level] CHECK OPTION]
```

Cú pháp xóa VIEW:

```
DROP VIEW view_name
```

Ý nghĩa các tùy chọn:

- WITH SCHEMABINDING: tùy chọn này cho phép tạo khung nhìn ở một Schema khác so với Schema mà bảng cơ sở dựa vào
- Tùy chọn WITH CHECK OPTION: là ràng buộc cho phép thực hiện INSERT, UPDATE hay DELETE thông qua View trên table cơ sở mà View dựa vào có kiểm tra hợp lệ theo ràng buộc dữ liệu.
- Tùy chọn WITH READ ONLY: là ràng buộc chỉ cho phép đọc dữ liệu trên view, không cho phép chỉnh sửa hay xóa.
- Thu hồi quyền cập nhật và xóa dữ liệu trên view với lệnh REVOKE UPDATE, DELETE ON view_name from user_name;

Ví dụ:

```
--tạo view dựa trên tblEmployee
create view vEmp
as
   select EmpID,EmpName,Email,Position
   from tblEmployee
   where Position is not null
   with check option
--tạo View từ 2 table trở lên
create view vSupReceipNote
as
   select s.SuppID,s.SuppName,r.RepID,r.RepDate,r.RepTotalMoney
   from tblSupplier as s
         inner join tblReceiptNote r
         on s.SuppID=r.SuppNo
   where r.RepTotalMoney > 200000
```

b) Dữ liệu trong VIEW

Dữ liệu (row và column) trong View được tham chiếu từ dữ liệu của các bảng mà View dựa vào ➔ dữ liệu trong View sẽ tự động được cập nhật mỗi khi có thay đổi dữ liệu trên các bảng mà View dựa vào. Chính vì vậy, View thường được dùng cho các báo cáo (report).

c) Các thao tác với VIEW

i. Truy vấn view (SELECT): tương tự với SELECT trong table

ii. Cập nhật view (INSERT – DELETE – UPDATE)

- Cập nhật dữ liệu trong view cũng sử dụng cú pháp như cập nhật trong table.

- Trong trường hợp cập nhật dữ liệu trong view thành công (successful), dữ liệu trong table mà view dựa vào cũng sẽ tự động được cập nhật theo

Ví dụ:

```
--insert vào vEmp thành công (row tự động insert vào tblEmployee)
insert into vEmp
values('NV00018',N'Quyên Nguyễn','0917258657','spktquyennth@gmail.com',N'Nhân viên kho')
--xóa dữ liệu trong vEmp thành công (row trong tblEmployee cũng bị xóa theo)
delete from vEmp
where EmpID='NV00015'
```

- Cập nhật dữ liệu trong view chỉ có thể được thực hiện khi thỏa mãn cả 2 trường hợp a và b dưới đây:

a. Câu lệnh "SELECT …statement" khi tạo view thỏa mãn các điều kiện sau:

- ✓ Mệnh đề FROM không được chứa nhiều tables
- ✓ Mệnh đề SELECT phải có chứa column Primary Key của table mà view dựa vào
- ✓ Mệnh đề SELECT không được chứa các toán tử DISTINCT, hàm (aggregate functions như MIN, MAX, AVG, COUNT, SUM), toán tử (operator)
- ✓ Không được chứa mệnh đề ORDER BY, GROUP BY hoặc HAVING
- ✓ Mệnh đề WHERE và SELECT không được chứa subquery.

b. Ràng buộc khi tạo bảng mà view dựa vào phải thỏa mãn các điều kiện sau:

- ✓ Không chứa giá trị DEFAULT

Ví dụ:

```
--xóa dữ liệu trong vSumReceiptNote thất bại vì FROM từ 2 table
delete from vSupReceipNote
where SuppID='NCC002'
```

2.4. Bảng thời gian (Temporal Table)

Bảng thời gian (Temporal Table), hay còn được gọi là "System-versioned Temporal Table", là tính năng mới của SQL Server 2016.

Khi cập nhật dữ liệu trong các table, thông thường trạng thái trước đó của các row dữ liệu sẽ bị mất, và chúng ta chỉ có thể truy cập vào dữ liệu hiện tại trên các table đó.

Trong trường hợp chúng ta cần truy cập vào trạng thái cũ trước đó (historical state) của table, ví dụ đã lỡ xóa những dòng dữ liệu quan trọng ➔ bảng thời gian sẽ giúp lấy được những thông tin về dữ liệu trong table tại bất kỳ thời điểm nào chứ không chỉ là dữ liệu table hiện tại.

Bảng thời gian cũng tương tự với cú pháp tạo bảng thông thường, tuy nhiên trong lệnh tạo table thông thường cần lưu ý các điều kiện sau:

- Có khóa chính Primary Key
- Bổ sung thêm 2 column (ví dụ đặt tên startcol và endcol) có kiểu dữ liệu DATETIME2, NOT NULL và chứa dữ liệu start/end về khoảng thời gian hợp lệ trong UTC (Coordinated Universal Time – giờ phối hợp quốc tế). Cả 2 column này nên được thiết lập với tùy chọn GENERATED ALWAYS AS ROW START.
- Column chứa dữ liệu end nên được thiết lập với tùy chọn PERIOD FOR SYSTEM_TIME (<startcol>, <endcol>)
- Thiết lập giá trị ON cho tùy chọn SYSTEM_VERSIONING
- Bảng lịch sử liên kết (Historical Table) để lưu các trạng thái quá khứ của table chính

Ví dụ:

```
-- Tạo bảng Employees table có bảng thời gian Temporal table
CREATE TABLE dbo.Employees_TemporalTable
(
   empid INT NOT NULL
         CONSTRAINT PK_Employees PRIMARY KEY NONCLUSTERED,
   empname VARCHAR(25) NOT NULL,
   department VARCHAR(50) NOT NULL,
   salary NUMERIC(10, 2) NOT NULL,
   sysstart DATETIME2(0)
         GENERATED ALWAYS AS ROW START HIDDEN NOT NULL,
   sysend DATETIME2(0)
         GENERATED ALWAYS AS ROW END HIDDEN NOT NULL,
   PERIOD FOR SYSTEM_TIME (sysstart, sysend),
   INDEX ix_Employees CLUSTERED(empid, sysstart, sysend)
)
WITH ( SYSTEM_VERSIONING = ON ( HISTORY_TABLE = dbo.EmployeesHistory ) );
--select dữ liệu từ bảng chính và bảng thời gian
SELECT empid, empname, department, salary, sysstart, sysend
FROM dbo.Employees_TemporalTable;

SELECT empid, empname, department, salary, sysstart, sysend
FROM dbo.EmployeesHistory
```

Câu Select trên bảng chính trả về dữ liệu trong khi câu select trên bảng thời gian không trả về giá trị nào (Empty), lý do là vì chưa có một cập nhật nào trên dữ liệu của bảng chính (chỉ có dữ liệu khi update hoặc delete dữ liệu ở bảng chính).

2.5. Tối ưu hóa câu truy vấn (Query Optimization)

Truy vấn dữ liệu (SELECT) là một trong những thao tác quan trọng và được sử dụng nhiều đối với các cơ sở dữ liệu.

Với một cơ sở dữ liệu đủ lớn về mặt số lượng bảng dữ liệu cũng như số dòng dữ liệu trong bảng dữ liệu, nếu truy vấn không được tối ưu tốt thì việc ứng dụng chạy ỳ ạch là khó tránh khỏi. Vì vậy, tuân thủ một số nguyên tắc để đảm bảo tối ưu hóa thời gian thực thi câu lệnh và chi phí tài nguyên là rất cần thiết cho người lập trình.

2.5.1. Execution Plan

Execution Plan (kế hoạch thực thi), hay còn gọi là trình phân tích thực thi câu truy vấn, cho biết cách thành phần Optimizer trong SQL Server (xem 1.2.1) lựa chọn để thực thi query. Cụ thể, Execution Plan cho ta biết các thông tin gồm: tables nào được truy cập và truy cập như thế nào, phép nối nếu có được kết hợp như thế nào, có sử dụng aggregate function hay không, …. Từ đó, đánh giá CPU cost và I/O cost trước và sau khi thực thi query (dạng text; hoặc graphical ở hình 2.24).

Cú pháp hiển thị phân tích thực thi câu truy vấn:

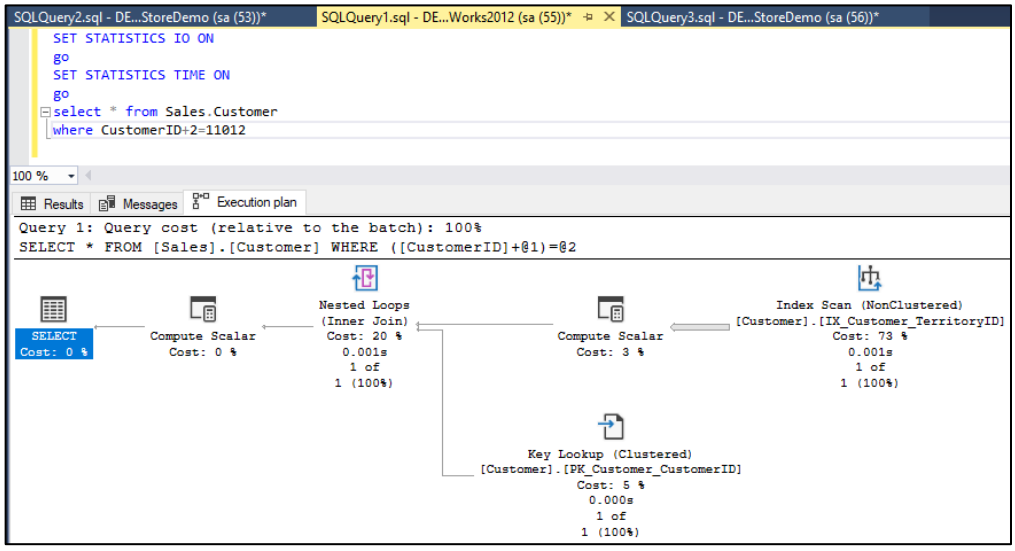
```
SET STATISTICS IO|TIME ON
SET SHOWPLAN_TEXT ON
SET SHOWPLAN_ALL ON|OFF
```

Ý nghĩa:

- SET SHOWPLAN_ALL: nếu là ON thì SQL Server trả về báo cáo thông tin kế hoạch thực thi ước đoán, ngược lại nếu là OFF thì SQL Server sẽ thực thi câu lệnh mà không tạo ra báo cáo thực thi
- SET STATISTICS IO|TIME: nếu là ON nghĩa hiển thị thống kê hoạt động đọc/ghi từ đĩa hoặc thời gian thực thi câu lệnh

SSMS cung cấp 2 tùy chọn cho Execution Plan gồm:

- Estimated Execution Plan: tạo ra ước đoán về kế hoạch thực thi mà không cần thực hiện câu lệnh T-SQL nào cả.
- Actual Execution Plan: tạo báo cáo kế hoạch thực thi sau khi câu lệnh T-SQL được thực hiện.



Hình 2.20: Minh họa Execution Plan

2.5.2. Các cơ chế truy vấn dữ liệu

SQL Server hỗ trợ các kiểu chỉ mục gồm: Hash Index, Memory-Optimized Nonclustered Index, Clustered Index, Nonclustered Index, Unique Index, Columnstore Index, Index with Included Columns, Index on Computed Columns, Filtered, Spatial, XML, Full-text.

Trong các loại chỉ mục nêu trên, cơ chế truy vấn dữ liệu phụ thuộc vào 2 loại chỉ mục (Index) sau: Clustered Indexs và Nonclustered Indexs

Clustered Indexs	Nonclustered Indexs
Chứa toàn bộ data của row tại node lá ➔ dữ liệu sẽ được sắp xếp một cách vật lý trên table theo giá trị của cột được thiết lập Clustered Index đó.	Chỉ chứa con trỏ đến data của row – không thực sự chứa data
Chỉ có thể có duy nhất 1 index này cho 1 table	Phát sinh một bảng tham chiếu – gồm 2 trường (cột): một trường chứa giá trị của cột được index, một trường chứa địa chỉ đến row mà chứa giá trị đó
Khi 1 record được insert hoặc delete khỏi table thì dữ liệu đó phải được lưu trữ lại và như vậy index phải được refresh lại	
Ví dụ: Primary Key Unique (nếu chưa có Clustered Index)	Ví dụ: Foreign Key Unique (mặc định)

SSMS hỗ trợ thao tác với chỉ mục thông qua Object Explorer (Indexes trong mỗi table).

2.5.2.1. Truy vấn dạng Scan

a) Full Table Scan

Truy vấn dạng Full Table Scan là truy vấn thực hiện tìm kiếm bằng cách quét qua từng dòng (row) của table một cách tuần tự, và kiểm tra xem nội dung các cột có thỏa mãn điều kiện tìm kiếm hay không – nếu có thì trả kết quả hoặc lưu kết quả và tiếp tục tìm cho đến row cuối cùng.

Chính vì tìm kiếm theo cơ chế quét tất cả các row dữ liệu của table, Full Table Scan đòi hỏi đọc/ghi (I/O) đĩa liên tục dẫn đến IO cost rất cao.

Ví dụ cho dạng Full Table Scan là truy vấn "SELECT * FROM tableName", mà trong đó tableName không được đánh chỉ mục.

b) Cluster Index Scan

Truy vấn dạng Clustered Scan là truy vấn hoạt động theo cơ chế gần giống với Full Table Scan, bởi vì dữ liệu được lưu trữ trong Index nên khi tìm kiếm dạng scan theo Index cũng có nghĩa là tìm kiếm với cả các column dữ liệu.

Ví dụ cho dạng Cluster Index Scan là truy vấn "SELECT * FROM tableName", trong đó tableName được đánh chỉ mục dạng Cluster.

2.5.2.2. Truy vấn dạng Seek

Seek hoạt động theo cơ chế lựa chọn các dòng dữ liệu cần từ table dựa trên cách tổ chức chỉ mục ở table và điều kiện tìm kiếm (ví dụ trong mệnh đề WHERE). Nghĩa là thực hiện tìm kiếm trên cây Index và nhảy tới bản ghi tương ứng trong table để lấy dữ liệu trả về.

a) Cluster Index Seek

b) Index Seek (Non-Clustered): đây là dạng truy vấn phổ biến trong SQL Server

2.5.3. Các kỹ thuật tối ưu hóa câu truy vấn

- Xác định rõ các column_list trong mệnh đề SELECT (hạn chế SELECT *)
- Hạn chế từ khóa DISTINCT trong mệnh đề SELECT, bằng cách chỉ rõ nhiều column_list nhất có thể
- Sử dụng từ khóa TOP hoặc ROWCOUNT để hạn chế số kết quả trả về
- Khi sử dụng ký tự đại diện wildcard, nên đặt cụm từ cần tìm ở đầu hoặc cuối (ví dụ '%Ha' hoặc 'Ha%' thay vì '%Ha%' – không phân biệt chữ hoa và thường)
- Hạn chế các truy vấn dạng SCAN
  + Thay vì sử dụng operation trong mệnh đề HAVING (thực hiện nối theo tích Đề-các), hãy sử dụng operation trong mệnh đề WHERE
  + Không áp dụng phép toán trên cột

    Ví dụ, thay vì sử dụng biểu thức tìm kiếm "tblProduct.QtyOnHand + 2 =10", nên sử dụng biểu thức "tblProduct.QtyOnHand = 8"
  + Tránh sử dụng hàm ở vế trái của toán tử

    Ví dụ, thay vì sử dụng "datediff(day,tblEmployee.DoB,getdate()) > 0", nên sử dụng " tblEmployee.DoB < getdate()"
  + Sử dụng phép nối bằng mệnh đề JOIN (ngoại trừ CROSS JOIN) thay cho phép nối bằng mệnh đề WHERE
  + Mệnh đề WHERE nên kết hợp từ nhiều điều kiện lọc (sử dụng toán tử AND) và xem xét thứ tự các biểu thức conditional_list
  + Sử dụng toán tử EXIST hay cho toán tử IN nếu có thể

    Ví dụ:

```
--cách query tối ưu
select *
from tblEmployee,tblOrderInvoice
where tblEmployee.EmpName like N'%Thảo'
        and  tblEmployee.EmpID=tblOrderInvoice.EmpNo
--cách query không tối ưu
select *
from tblEmployee,tblOrderInvoice
where tblEmployee.EmpID=tblOrderInvoice.EmpNo
        and tblEmployee.EmpName like N'%Thảo'
```

  + Sử dụng Index trong mệnh đề WHERE

    Nếu table_lists đã có Index, nếu không nên tạo mới Index cho các table_lists và việc tạo/xóa Index không ảnh hưởng đến dữ liệu của table. Tuy nhiên, vì Index có thể chiếm nhiều không gian của ổ cứng, do đó không nên triển khai quá nhiều Index nếu như chúng không thực sự cần thiết, hoặc nếu table là các bảng nhỏ.

Cú pháp tạo Index:

```
CREATE [UNIQUE] [CLUSTERED | NONCLUSTERED] INDEX index_name
ON table_name (column, [column1],[column2]…)
```

Cú pháp xóa Index:

```
DROP INDEX table_name.index_name
```

+ Đối với Clustered Index, hãy cố gắng giữ cho độ dài của các cột được lập Index càng ngắn càng tốt. Lý tưởng nhất là tạo Clustered Index trên cột có thuộc tính Unique và không cho phép giá trị NULL. Đây là lý do tại sao các cột khóa chính thường được sử dụng cho Clustered Index của bảng, và không cho phép giá trị NULL.
+ Đối với Composite Index, cần phải xem xét thứ tự của các cột trong lệnh định nghĩa của Index. Cột nào thường được sử dụng trong các biểu thức so sánh ở mệnh đề WHERE sẽ được liệt kê đầu tiên. Thứ tự của các cột tiếp theo sẽ được liệt kê dựa trên tính duy nhất của các giá trị trong cột, trong đó tính duy nhất của giá trị trong cột càng cao thì càng được liệt kê trước.

CHƯƠNG 3. LẬP TRÌNH TRONG SQL SERVER

3.1. Khối lệnh (Statement Block)

Cấu trúc này dùng để khai báo một số lệnh hoặc khối lệnh khác bên trong, điều này giúp tạo ra một chương trình có tính thẩm mỹ và rõ ràng.

Cú pháp:

```
BEGIN
   { sql_statement |
    statement_block}
END
```

Lưu ý:

- Khối lệnh BEGIN…END thường dùng bắt buộc đối với các câu lệnh IF…ELSE, WHILE khi cần dùng để bao bọc một khối lệnh.
- Khối lệnh BEGIN…END có thể được sử dụng lồng bên trong một khối lệnh BEGIN…END khác.

3.2. Các lệnh điều khiển

3.2.1. Lệnh BREAK

Lệnh BREAK dùng để thoát khỏi vòng lặp trong lệnh WHILE.

Nếu vòng lặp WHILE có WHILE LOOP lồng nhau thì BREAK chỉ thoát khỏi vòng lặp gần nhất.

3.2.2. Lệnh CONTINUE

Lệnh CONTINUE được sử dụng trong lệnh lặp WHILE, nghĩa là bỏ qua các lệnh sau CONTINUE và lộn ngược lên đầu vòng lặp.

3.2.3. Lệnh GOTO

Lệnh GOTO là lệnh nhảy, cho phép luồng điều khiển nhảy vô điều kiện từ vị trí GOTO tới vị trí gán nhãn (LABEL).

Cú pháp lệnh nhảy:

```
GOTO label_name;
```

LABEL gồm có 2 phần: tên nhãn label_name, và câu lệnh để thực hiện tiếp theo

Cú pháp gán nhãn:

```
LABEL_NAME: {sql_statement}
```

3.3. Cấu trúc rẽ nhánh

3.3.1. Lệnh IF…ELSE

Cấu trúc rẽ nhánh IF…ELSE cho phép thực hiện lệnh khi điều kiện được thỏa mãn.

Cú pháp:

```
IF Boolean_expression
    { sql_statement | statement_block }
[ ELSE
    { sql_statement | statement_block } ]
```

Lưu ý:

- Nếu biểu thức điều kiện Boolean_expression trả về TRUE thì lệnh/khối lệnh được thực thi, khối lệnh ELSE là thành phần tùy chọn, nếu biểu thức điều kiện Boolean_expression trả về FALSE thì lệnh/khối lệnh trong ELSE sẽ được thực thi.
- Nếu nhiều hơn 1 lệnh được thực thi trong IF hoặc ELSE thì phải đặt trong khối BEGIN…END.
- Cấu trúc IF…ELSE có thể được lồng bên trong khối IF…ELSE khác.

Ví dụ:Tính tổng tiền nhập hàng trong năm 2019, nhiều hay ít hơn 100000VNĐ

```
BEGIN
    DECLARE @total money;
    SELECT @total = SUM(QtyReceipted * RepPrice)
    FROM tblReceiptNoteDetail as rd
       INNER JOIN tblReceiptNote n
          ON YEAR(n.RepDate)=2019 and n.RepID = rd.RepID
    --hiển thị giá trị @total
    print @total;
  --kiểm tra điều kiện
    IF @total > 10000
          PRINT 'Tổng tiền nhập hàng năm 2019 trên 100000 VNĐ';
    ELSE
         PRINT 'Tổng tiền nhập hàng năm 2019 không quá 100000 VNĐ';
END
```

3.3.2. Lệnh CASE

Cấu trúc này dùng để duyệt qua các điều kiện và trả về 1 giá trị khi điều kiện đầu tiên thỏa mãn, khi 1 điều kiện đúng, nó sẽ dừng lệnh đọc và trả về kết quả. Nếu không có điều kiện nào đúng, nó trả về giá trị trong mệnh đề ELSE.

Lưu ý lệnh Case phải lồng trong lệnh Select, cho dù không cần select bất kỳ cột nào từ bảng nào.

Cú pháp:

```
SELECT
  CASE
        WHEN condition1 THEN result1
        WHEN condition2 THEN result2
        WHEN conditionN THEN resultN
        ELSE result
  END;
```

Ví dụ:

Hiển thị thông tin của sản phẩm gồm ProductID, Name, QtyOnHand và Result (giá trị của Result là kết quả so sánh QtyOnHand với 20):

```
SELECT ProductID,Name,QtyOnHand,
    CASE
        WHEN QtyOnHand > 20 THEN N'Số lượng lớn hơn 20'
        WHEN QtyOnHand = 20 THEN 'Số lượng bằng 20'
        ELSE 'Số lượng nhỏ hơn 20'
            END AS Result
FROM tblProduct;
```

Hiển thị thông tin của sản phẩm gồm ProductID, Name và Price (Price là UnitPrice, nếu UnitPrice là NULL thì Price là giá thấp nhất của sản phẩm):

```
SELECT ProductID,Name,
    CASE
        WHEN UnitPrice IS NULL
                THEN (SELECT MIN(UnitPrice)
                        FROM tblProduct)
        ELSE UnitPrice
    END AS Price
FROM tblProduct
```

3.4. Cấu trúc lặp (WHILE)

Lệnh lặp WHILE dùng để lặp lại việc thực thi các câu lệnh hoặc khối lệnh trong khi điều kiện đúng. Việc thực thi các lệnh trong WHILE có thể được điều khiển với từ khóa BREAK và CONTINUE.

Không tồn tại lệnh lặp FOR trong SQL Server (như trong các ngôn ngữ lập trình thông dụng).

Cú pháp:

```
WHILE Boolean_expression
    { sql_statement | statement_block | BREAK | CONTINUE }
```

Ý nghĩa:

- Boolean_expression: là biểu thức logic trả về TRUE hoặc FALSE. Nếu biểu chứa logic chứa câu lệnh SELECT thì câu lệnh SELECT phải được đóng bởi cặp dấu ngoặc đơn.
- sql_statement hoặc statement_block: là các câu lệnh T-SQL, hoặc nhóm các lệnh được chứa trong khối lệnh BEGIN và END.
- BREAK: thoát khỏi lệnh WHILE
- CONTINUE: bỏ qua các dòng lệnh sau CONTINUE và bắt đầu vòng lặp mới.

Ví dụ:

Hãy in các số từ 1 đến 10 với lệnh lặp WHILE:

```
DECLARE @counter int
SET @counter =1
WHILE @counter <=10
BEGIN
```

```
    PRINT @counter
    SET @counter = @counter+1
END
```

3.5. Thủ tục (Stored Procedure)

3.5.1. Định nghĩa

Stored Procedure, một tập nhiều câu lệnh SQL được nhóm lại với nhau thành một nhóm, và được lưu trữ như là một đối tượng trong cơ sở dữ liệu.

Khi Stored Procedure lần đầu tiên được gọi (call), SQL Server sẽ tạo ra một Execution Plan và được lưu vào Plan Cache. Kể từ đó, những lần gọi Stored Procedure sau đó sẽ tái sử dụng Execution Plan giúp việc thực thi Stored Procedure nhanh hơn.

3.5.2. Tạo/Cập nhật Stored Procedure

Cú pháp:

```
CREATE [OR ALTER] PROCEDURE | PROC  [schema_name] procedure_name
 [ @parameter datatype
 [ VARYING ] [ = default ] [ OUT | OUTPUT | READONLY ]
 , @parameter datatype
 [ VARYING ] [ = default ] [ OUT | OUTPUT | READONLY ] ]
[ WITH  ENCRYPTION | RECOMPILE ]
AS
BEGIN
  SQL_Statements
END;
```

Ý nghĩa:

- procedure_name: tên thủ tục
- @parameter: tham số, trong đó tên tham số phải đặt sau dấu @ - chỉ rõ kiểu dữ liệu datatype. Tham số được đặt ngay sau tên thủ tục, nếu nhiều tham số thì các tham được cách nhau bởi dấu phẩy. Mặc định là tham số đầu vào
- default: giá trị mặc định gán cho tham số
- VARYING: chỉ định rằng tập kết quả được hỗ trợ như là tham số đầu ra. Tham số này được tạo tự động khi thực thi thủ tục, và VARYING chỉ áp dụng đối với tham số cursor.
- OUT | OUT PUT: chỉ định rằng tham số là tham số đầu ra. Thiết lập OUTPUT cho tham số khi cần lấy giá trị trả về khi gọi thủ tục. Lưu ý, các kiểu dữ liệu gồm text, ntext và image không được sử dụng cho tham số đầu ra (trừ phi thủ tục thuộc loại CLR Procedure - Common Language Runtime Procedure)
- READONLY: chỉ ra rằng tham số không thể được thay đổi giá trị trong thân thủ tục, nếu tham số thuộc kiểu table-value thì bắt buộc phải thiết lập READONLY

- ENCRYPTION: thủ tục sẽ được mã hóa sang một định dạng khác so với plain text khi có tùy chọn WITH ENCRYPTION, và chỉ những privileged users mới được phép truy cập nội dung của thủ tục ở dạng mã hóa mới (chỉ áp dụng từ phiên bản SQL Server 2008)
- RECOMPILE: chỉ ra rằng thủ tục cần phải được biên dịch, tức là tạo một Execution Plan mỗi khi thủ tục được gọi.

Ví dụ: 2 cách viết dưới đây khác biệt ở chỗ cách viết thứ 2 cho phép thống kê 1,2,3,…,n sản phẩm bán chạy chỉ với một thủ tục; trong khi cách viết thứ phải được alter khi cần thay đổi số sản phẩm bán chạy.

Tạo thủ tục thống kê 5 sản phẩm bán được nhiều nhất trong năm 2019:

```
create proc prBestSell_Top5
as
BEGIN
   Select top 5 ProductID, sum(QtyOrdered) as 'SumOfProduct'
   from tblOrderInvoiceDetail as od
   inner join tblOrderInvoice odd
         on year(odd.OrderDate)=2019 and odd.OrderID=od.OrderID
   group by ProductID
   order by SumOfProduct desc
END
```

3.5.3. Xóa Stored Procedure

Khi xoá một thủ tục, tất cả các quyền đã cấp cho người sử dụng trên thủ tục đó cũng đồng thời bị xoá bỏ. Do đó, nếu tạo lại thủ tục, ta phải tiến hành cấp phát lại các quyền trên thủ tục đó.

Cú pháp:

```
DROP PROCEDURE | PROC procedure_name
```

Ví dụ:

```
drop proc prBestSell_Top
```

3.5.4. Thực thi Stored Procedure

Cú pháp:

```
EXEC|EXECUTE procedure_name  {value|@variable [OUTPUT] } [,value|@variable [OUTPUT]}
```

Ví dụ:

```
exec prBestSell_Top5
execute prBestSell_Top   3
```

Lưu ý:

- Số lượng các đối số cũng như thứ tự của chúng phải phù hợp với số lượng và thứ tự của các tham số khi định nghĩa thủ tục.
- Thứ tự của các đối số được truyền cho thủ tục có thể không cần phải tuân theo thứ tự của các tham số như khi định nghĩa thủ tục nếu tất cả các đối số được viết dưới dạng:

  EXEC procedure_name @argument = value,...

3.6. Hàm (FUNCTION)

3.6.1. Định nghĩa

Hàm là đối tượng cơ sở dữ liệu tương tự như thủ tục. Điểm khác biệt giữa hàm và thủ tục là hàm trả về một giá trị thông qua tên hàm còn thủ tục thì không. Điều này cho phép ta sử dụng hàm như là một thành phần của một biểu thức (chẳng hạn trong danh sách chọn của câu lệnh SELECT).

Ngoài những hàm do hệ quản trị cơ sở dữ liệu cung cấp sẵn, người sử dụng có thể định nghĩa thêm các hàm nhằm phục vụ cho mục đích riêng của mình và gọi là UDF (User-Defined Function)

Các loại UDF function:

- Scalar Function: trả về giá trị vô hướng
- Table-valued Function: trả về kết quả dạng table
- Aggregated Function: là những hàm mà được định nghĩa trong một class ở .NET Framework

3.6.2. Tạo/Cập nhật FUNCTION

Cú pháp:

```
CREATE | ALTER FUNCTION [ schema_name. ] function_name
( [ { @parameter_name [ AS ][ type_schema_name. ] parameter_data_type
   [ = default ] [ READONLY ] }
   [ ,...n ]
 ]
)
RETURNS data_type
[AS]
BEGIN
   Statement_list
   RETURN returned_value
END
```

Ý nghĩa:

- schema_name: nếu để trống, mặc định function thuộc Schema dbo
- function_name: tên hàm
- default: giá trị mặc định gán cho tham số
- READONLY: chỉ ra rằng tham số không thể được thay đổi giá trị trong thân thủ tục, nếu tham số thuộc kiểu table-value thì bắt buộc phải thiết lập READONLY
- data_type: kiểu dữ liệu mà function sẽ trả về
- value: giá trị mà function sẽ nhận
- lưu ý sự khác biệt RETURNS và RETURN

Ví dụ:

```
Hàm tính giá của sản phẩm sau khi đã áp dụng giảm giá
create function fnCalPriceDiscount
(
   @qty int,
   @unitPrice money,
   @discount dec(4,2)
)
returns money
BEGIN
   return @qty * @unitPrice * (1 - @discount)
END
```

3.6.3. Xóa Function

Cú pháp:

```
DROP FUNCTION function_name;
```

Ví dụ:

```
drop function fnCalPriceDiscount
```

3.6.4. Gọi Function

Gọi hàm với tên hàm và danh sách đối số truyền vào theo đúng thứ tự và kiểu dữ liệu của parameter_list, và danh sách đối số phải được đặt trong cặp dấu ngoặc đơn, phân cách bởi dấu phẩy ‘,’.

Gọi hàm có thể được thực hiện thông qua các câu lệnh như SELECT, toán hạng trong toán tử so sánh…

Cú pháp:

```
function_name  [value] [,value]
```

Ví dụ:

```
select dbo.fnCalPriceDiscount (2, 100000, 0.15)
```

3.6.5. Các hàm hệ thống thông dụng

- Hàm chuyển đổi: (Conversion Functions)
- Hàm logic (logic Functions)
- Hàm toán học (Math Functions)
- Hàm chuỗi (String Funtions)
- Hàm ngày tháng (Date Functions)

Ỹ nghĩa và cách dùng các hàm hệ thống được trình bày ở phần phụ lục (trang …).

3.7. Trigger

3.7.1. Định nghĩa

Trigger là một loại Stored Procedure đặc biệt, đó là tự động chạy (automatically run) khi có một sự kiện (event) xảy ra trên cơ sở dữ liệu.

- DDL trigger: là trigger tự động chạy khi có các sự kiện do các câu lệnh CREATE, ALTER hoặc DROP xảy ra
- LOGON trigger: là trigger liên quan đến các sự kiện logon khi có một phiên làm việc của người dùng được thiết lập
- DML trigger: là trigger tự động chạy khi có sự kiện cập nhật dữ liệu bởi các câu lệnh INSERT, UPDATE hoặc DELETE xảy ra. DML trigger sử dụng 2 bảng logic là deleted và inserted, 2 bảng này có cấu trúc giống với cấu trúc của table mà ta muốn tạo trigger

Sự kiện	Bảng DELETED	Bảng INSERTED
INSERT	Rỗng	Chứa dữ liệu của dòng vừa được thêm vào
UPDATE	Dữ liệu cũ của dòng vừa cập nhật	Chứa dữ liệu mới của dòng được cập nhật
DELETE	Dữ liệu của dòng bị xóa	Rỗng

Giáo trình này trình bày nội dung và ví dụ về loại DML trigger trên các table của cơ sở dữ liệu, cụ thể là trigger do một trong các sự kiện sau:

- INSERT: trigger sẽ được kích hoạt tự động khi có thao tác thêm dữ liệu
- UPDATE: trigger sẽ được kích hoạt tự động khi có thao tác cập nhật dữ liệu
- DELETE: trigger sẽ được kích hoạt tự động khi có thao tác xóa dữ liệu

3.7.2. Tạo/Cập nhật Trigger

Cú pháp:

```
CREATE [ OR ALTER ] TRIGGER [ schema_name . ]trigger_name
ON { table | view }
 [ WITH <dml_trigger_option> [ ,...n ] ]
{ FOR | AFTER | INSTEAD OF }
{ [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] }
AS
BEGIN
  { sql_statement  [ ; ] [ ,...n ] [ ; ] > }

<dml_trigger_option> ::=
      [ EXECUTE AS Clause ]
END
```

Ý nghĩa:

- schema_name: tên của Schema
- trigger_name: tên của trigger
- table: tên của table mà trigger sẽ được tự động chạy khi có sự kiện insert, hoặc update, hoặc delete trên table đó xảy ra

- FOR | AFTER | INSTEAD OF: liên quan đến thời điểm kích hoạt trigger
  + AFTER/ FOR được sử dụng khi muốn DML trigger được kích hoạt sau khi sự kiện cập nhật dữ liệu diễn ra, nghĩa là tập các câu lệnh trong thân trigger sẽ được thực thi sau sự kiện cập nhật
  + INSTEAD OF được sử dụng khi muốn tập các câu lệnh trong thân trigger sẽ được thực thi thay cho sự kiện mà trigger dựa vào, tức là sự kiện DML trigger không được thực thi. Lưu ý, chỉ được định nghĩa duy nhất một INSTEAD OF trigger cho mỗi sự kiện insert/delete/update trên một table
- EXECUTE AS: chỉ định ngữ cảnh liên quan đến bảo mật khi trigger được kích hoạt, cho phép kiểm soát quyền của những tài khoản người dùng trên các đối tượng của database mà trigger có tham chiếu đến.

Ví dụ:

Tạo trigger cho sự kiện Insert, tự động cập nhật giá trị cột Amount trong bảng tblReceiptNoteDetail dựa vào giá trị QtyOrdered và RepPrice mà người dùng chèn vào theo công thức:

*Amount = QtyOrdered * RepPrice*

(Lưu ý, chỉ cập nhật cho row vừa mới insert vào – không cập nhật cho cả table)

```
--bổ sung thêm cột thành tiền (Amount) trên bảng tblReceiptNoteDetail
alter table tblReceiptNoteDetail
   add Amount money null default 0
--trigger tự động cập nhật giá trị cột Amount trong tblReceiptNoteDetail
create trigger trgCalculateAmount
on tblReceiptNoteDetail
after insert
AS
BEGIN
   declare @id char(7), @qty int, @price money
   declare @amount money
   select @id=RepID from inserted
   select @qty=QtyReceipted from inserted
   select @price=RepPrice from inserted
   set @amount=@qty * @price
   update tblReceiptNoteDetail
   set Amount=@amount
   where RepID=@id
END
```

Lưu ý:

- Mỗi table có thể có một hoặc nhiều trigger
- Trigger cũng có thể được thực thi như là một phần của transaction. Trong trường hợp muốn huỷ bỏ tác dụng của câu lệnh làm kích hoạt trigger ta sử dụng lệnh ROLLBACK TRANSACTION.

3.7.3. Xóa Trigger

Cú pháp:

```
DROP TRIGGER trigger_name
```

3.7.4. Disable/Enable Trigger

Mặc định, trigger ở trạng thái 'Enabled' khi nó được tạo ra.

Cú pháp Enable trigger:

```
ENABLE TRIGGER { [ schema_name . ] trigger_name [ ,...n ] | ALL }
ON { object_name | DATABASE | ALL SERVER } [ ; ]
```

Disable không có nghĩa là xóa bỏ trigge – tức là trigger vẫn tồn tại trong database, tuy nhiên trigger sẽ không được kích hoạt khi có sự kiện mà được lập trình cho trigger đó xảy ra.

Cú pháp Disable trigger:

```
DISABLE TRIGGER { [ schema_name . ] trigger_name [ ,...n ] | ALL }
ON { object_name | DATABASE | ALL SERVER } [ ; ]
```

Ý nghĩa:

- ALL: là tất cả các trigger trong phạm vi của đối tượng sau ON

3.8. Con trỏ (Cursor)

Con trỏ là một cấu trúc đặc biệt được sử dụng trong SQL để lưu giữ dữ liệu của các dòng được trả về bởi câu truy vấn, nôm na là mảng các row và column.

Hai loại cursor gồm:

- Implicit cursor: được tạo tự động khi câu lệnh SQL chỉ trả về 1 giá trị duy nhất
- Explicit cursor: được tạo để lưu giữ giá trị đầu ra của câu lệnh SQL mà trả về từ 2 row dữ liệu trở lên

3.8.1. Cú pháp khai báo cursor

```
DECLARE cursor_name CURSOR [ LOCAL | GLOBAL ]
    [ FORWARD_ONLY | SCROLL ]
    [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
    [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
    [ TYPE_WARNING ]
    FOR
          select_statement
    [ FOR UPDATE [ OF column_name [ ,...n ] ] ]
[;]
[CLOSE cursor_name    ;]
[DEALLOCATE cursor_name ;]
```

Ý nghĩa:

- LOCAL: phạm vi của cursor chỉ ở trong bó lệnh, hay thủ tục, hay trigger mà cursor được tạo. Mặc định là LOCAL
- GLOBAL: trigger có thể được tham chiếu trong bất kỳ bó lệnh, hay thủ tục, hay trigger mà có kết nối

- SCROLL: chỉ định rằng các tùy chọn tìm kiếm (FIRST, LAST, PRIOR, NEXT, RELATIVE, ABSOLUTE) được hỗ trợ. Nếu SCROLL không được chỉ định, cursor chỉ hỗ trợ tùy chọn tìm kiếm NEXT và được gọi là FORWARD_ONLY
- STATIC: chỉ định rằng cursor luôn hiển thị tập kết quả truy vấn khi cursor lần đầu tiên được opened và tạo bảng sao chép dữ liệu tạm cho cursor sử dụng. Tất cả những request đến cursor đều được lấy từ bảng tạm này trong tempdb. Lưu ý, những dữ liệu do các câu lệnh insert, update hay delete sẽ không được nhận ra bởi lệnh fetch.
- DYNAMIC: định nghĩa một cursor cho phép phản chiếu tất cả các thay đổi dữ liệu khi SCROLL và FETCH
- KEYSET: chỉ định rằng các mối quan hệ và trật tự các row dữ liệu là không thay đổi kể từ khi cursor được opened
- FAST_FORWARD: chỉ định rằng FORWARD_ONLY hoặc READ_ONLY với sự tối ưu hóa thực hiện là được phép. FAST_FORWARD không đi với SCROLL hoặc FOR UPDATE.
- READ_ONLY: ngăn chặn cập nhật thông qua cursor này
- SCROLL_LOCKS: chỉ định rằng việc cập nhật hoặc xóa vị trí thông qua con trỏ là được chấp nhận. SCROLL_LOCKS không đi với FAST_FORWARD hoặc STATIC
- OPTIMISTIC: ngược với SCROLL_LOCKS
- TYPE_WARNING: chỉ định rằng message cảnh báo được gửi đến client khi cursor được chuyển một cách không tường minh từ một kiểu khác
- select_statement: là câu truy vấn trả kết quả cho cursor. Các từ khóa COMPUTE, COMPUTE BY, FOR BROWSE, INTO không được xuất hiện trong câu truy vấn

3.8.2. Các lệnh thông dụng cho cursor

- OPEN: mở con trỏ để chứa dữ liệu của câu lệnh truy vấn

  Cú pháp:

```
OPEN cursor_name
```

- FETCH: sau khi con trỏ được mở, lệnh FETCH để truy xuất dữ liệu từ cursor ➔ có thể gán vào biến cho việc xử lý

  Cú pháp:

```
FETCH FROM cursor_name INTO variable1[,variable2…]
```

  + FETCH FIRST: truy xuất hàng đầu tiên (ko được dùng với FORWARD_ONLY cursor)
  + FETCH NEXT: truy xuất hàng tiếp theo
  + FETCH PRIOR: truy xuất hàng trước hàng hiện tại
  + FETCH LAST: truy xuất hàng cuối cùng
  + FETCH ABSOLUTE n: truy xuất hàng thứ n trong cursor

- CLOSE: đóng con trỏ

  Cú pháp:

```
CLOSE cursor_name
```

- DEALLOCATE: xóa tham chiếu đến con trỏ

  Cú pháp:

```
DEALLOCATE cursor_name
```

3.8.3. Các biến thông dụng với cursor

- @@FETCH_STATUS: trả về số nguyên – cho biết kết quả của lệnh truy xuất cuối cùng của cursor
- @@CURSOR_ROWS: trả về tổng số hàng hiện tại trong cursor đang mở

3.8.4. Tạo cursor

- Khai báo các biến cần giữ giá trị dữ liệu các column khi duyệt con trỏ
- Khai báo con trỏ
- Mở con trỏ
- Truy xuất dòng dữ liệu để bắt đầu tiến trình, rồi thao tác trên dòng dữ liệu đó. Nếu muốn lặp lại việc thao tác trên dòng dữ liệu khi có thể kết hợp các lệnh cấu trúc
- Truy xuất dòng dữ liệu kế tiếp
- Đóng con trỏ khi tất cả các dòng dữ liệu đã được truy xuất
- Giải phóng tài nguyên cấp phát cho con trỏ

Ví dụ:

Ví dụ dưới tạo một cursor để duyệt những nhân viên trong tên có ký tự 't' – và nếu có thì in ra chuỗi 'Có'

```
DECLARE csEmployee CURSOR
FOR
   SELECT EmpName,Email
   FROM tblEmployee
   WHERE EmpName like '%t%'
OPEN csEmployee
FETCH NEXT FROM csEmployee
WHILE @@FETCH_STATUS = 0
BEGIN
   print N'có'
   FETCH NEXT FROM csEmployee
END;
CLOSE csEmployee
DEALLOCATE csEmployee
```

Lưu ý:

Hạn chế sử dụng cursor, bởi vì SQL Server cần cung cấp tài nguyên cho cursor mỗi khi cần thực thi và điều này được khuyến cáo bằng cách sử dụng các thủ tục hay trigger thay thế cho cursor (trừ những trường hợp không thể thay thế được).

Cursor thường được sử dụng kết hợp trong các thủ tục.

3.9. Bảng tạm (Temporary Table)

Tempdb là một database hệ thống cho phép người dùng có thể sử dụng khi kết nối được đến SQL Server. Tempdb lưu giữ 2 nhóm đối tượng:

- Các đối tượng tạm do người dùng tạo ra (Temporary user object): gồm những đối tượng như bảng tạm, các thủ tục, cursor…
- Những đối tượng bên trong (Internal object): do Database Engine tạo ra, gồm work table hoặc work file hoặc các truy vấn có group by, order by, union

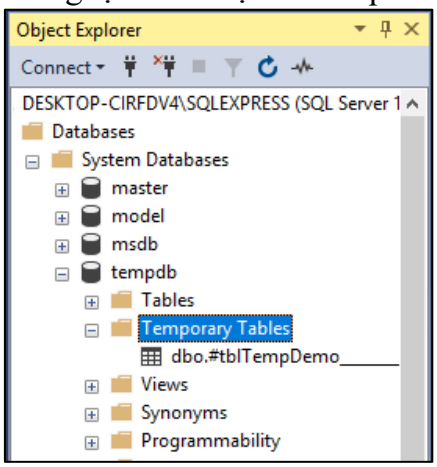
Bảng tạm cho phép lưu giữ tạm thời data của một table, và được drop tự động khi ngắt kết nối đến SQL Server.

SQL Server hỗ trợ 3 loại bảng tạm:

- Local Temporary Tables: chỉ có tác dụng trong phiên làm việc (session) khi có kết nối đến SQL Server (sử dụng ký tự '#' trước talbe_name)
- Global Temporary Tables: có tác dụng trong tất cả các session, tức là những lần kết nối đến SQL Server sau vẫn còn những bảng tạm này (sử dụng ký tự '##' trước talbe_name)
- Table Variables: cũng giống loại Local Temporary Tables, tuy nhiên phải thêm từ khóa DECLARE khi khai báo

Lưu ý:

- Thao tác với bảng tạm (CREATE, ALTER, DROP) cũng giống như với table thông thường, chỉ khác ở chỗ bảng tạm sẽ thuộc về tempdb



Hình 3.1: Thao tác với bảng tạm trong SSMS

- Cho phép truy vấn dữ liệu trong bảng tạm như với bảng thông thường
- Có thể xóa bảng tạm và tạo lại ngay trong một thủ tục, nhưng không thể làm việc này với bảng thông thường
- Bảng tạm sẽ hữu dụng trong trường hợp có 2 thủ tục, thủ tục cha gọi thủ tục con thì nếu bảng tạm tạo trong thủ tục cha thì thủ tục con sẽ nhìn thấy bảng tạm và có thể cập nhật dữ liệu trên bảng tạm này cho đến khi thoát khỏi thủ tục cha.

Ví dụ:

Tạo bảng tạm có tên tblTempDemo

```
create table #tblTempDemo
(
   id int,
   name nvarchar(50)
)
```

CHƯƠNG 4. QUẢN LÝ TRANSACTION

Như chúng ta đã từng biết một trong các chức năng chính của một hệ quản trị cơ sở dữ liệu là phải đảm bảo tính tin cậy, tính nhất quán của cơ sở dữ liệu, ngay cả khi có nhiều người dùng đồng thời truy cập vào cơ sở dữ liệu, hay thậm chí ngay cả khi xảy ra sự cố (phần cứng hay phần mềm).

Nội dung chương này giúp chúng ta tìm hiểu ba dịch vụ mà một hệ quản trị cơ sở dữ liệu cần cung cấp được, đó là: hỗ trợ giao tác, điều khiển tương tranh và khôi phục cơ sở dữ liệu. Điều khiển tương tranh và khôi phục cơ sở dữ liệu mỗi khi có sự cố, cả hai đều nhằm đảm bảo sự nhất quán về dữ liệu và tránh tổn thất thông tin.

4.1. Khái niệm

4.1.1. Transaction

Transaction, nghĩa là giao tác hay giao dịch, là tiến trình thực hiện một nhóm các câu lệnh SQL, các câu lệnh này được thực thi một cách tuần tự và độc lập (thường là các câu lệnh Insert, Update hoặc Delete).

Một Transaction được xem là thực hiện thành công khi tất cả câu lệnh đều thành công, khi đó tất cả các thay đổi dữ liệu được thực hiện trong Transaction đều được lưu vào cơ sở dữ liệu. Tuy nhiên, nếu chỉ một trong số các câu lệnh đó thất bại thì toàn bộ tiến trình sẽ thất bại, đồng nghĩa với việc dữ liệu phải rollback về trạng thái ban đầu (tức là dữ liệu được khôi phục về trạng thái trước khi thực hiện Transaction).

4.1.2. Các đặc điểm của Transaction (ACID)

a) Tính bảo toàn (**A**tomicity): nguyên tắc "all or nothing", đảm bảo rằng tất cả các câu lệnh trong nhóm lệnh được thực thi thành công. Nếu không, Transaction bị hủy bỏ tại thời điểm thất bại và tất cả các thao tác trước đó được khôi phục về trạng thái cũ đồng nghĩa với việc không có gì thay đổi về mặt dữ liệu.

b) Tính nhất quán (**C**onsistency): đảm bảo rằng cơ sở dữ liệu thay đổi chính xác các trạng thái khi một transaction được thực thi thành công.

c) Tính độc lập (**I**solation): cho phép các Transaction hoạt động độc lập và minh bạch với nhau.

d) Tính bền vững (**D**urability): đảm bảo rằng kết quả của một transaction được xác định, không có chuyện dữ liệu của Transaction sau khi thực thi có thể chuyển lại trạng thái dữ liệu lúc trước khi thực hiện.

4.1.3. Phân loại Transaction

a) Transaction tường minh (Explicit Transaction)

Explicit transaction là một Transaction mà việc khởi động và kết thúc Transaction đó đều được định nghĩa một cách tường minh.

Khi khai báo transaction tường minh, phải đảm bảo rằng sau đó nó phải được rollback hoặc commit tường minh, nếu không, transaction sẽ tiếp tục tồn tại và chiếm giữ tài nguyên, ngăn trở sự thực hiện của các transaction khác.

b) Transaction ngầm định (Implicit Transaction)

Implicit Transaction là một loại Transaction tự động được commit, không cần được bắt đầu hoặc kết thúc Transaction.

Nếu chúng ta không tạo một Transaction tường minh thì Transaction ngầm định sẽ tự động được tạo ra.

Thiết lập Transaction ngầm định bằng câu lệnh sau:

```
SET IMPLICIT_TRANSACTIONS ON
```

c) Transaction nhật ký (Transaction Log)

Transaction log là nhật ký Transaction, được duy trì bởi chính SQL Server để quản lý tất cả các Transaction.

Khi Transaction được thi hành, Transaction Log sẽ ghi nhật ký các thao tác (bước) của Transaction đó:

- Nếu có lỗi xảy ra tại bước n, rollback transaction sẽ dựa vào nhật ký để hủy (n-1) thao tác đã làm trước đó.
- Nếu tất cả các bước đều thành công thì sẽ thực hiện việc ghi dữ liệu lên đĩa dựa vào câu lệnh commit transaction.

Nội dung log bao gồm các thông tin sau:

- Điểm bắt đầu của transaction
- Ghi nhật ký cho mỗi câu lệnh SQL, cụ thể gồm:
  + Loại thao tác (Retrieve, Update, Insert, Delete)
  + Tên biến, table, field
  + Giá trị trước và sau khi dữ liệu được cập nhật
  + Số hiệu thao tác trước và sau đó
- Câu lệnh commit

4.2. Xử lý Transaction

Các lệnh điều khiển Transaction chỉ được sử dụng với các lệnh thao tác dữ liệu như INSERT, UPDATE và DELETE. Lưu ý là, chúng không thể được sử dụng trong lệnh CREATE TABLE hoặc DROP TABLE vì các hoạt động này được tự động xác định trong cơ sở dữ liệu.

Một transaction được bắt đầu bởi câu lệnh BEGIN TRANSACTION. Câu lệnh này đánh dấu điểm bắt đầu transaction và kết thúc trong các trường hợp sau:

- Câu lệnh COMMIT TRANSACTION (hoặc COMMIT WORK) được thực thi. Câu lệnh này báo hiệu sự kết thúc thành công của một transaction. Sau câu lệnh này, một transaction mới sẽ được bắt đầu.
- Khi câu lệnh ROLLBACK TRANSACTION (hoặc ROLLBACK WORK) được thực thi để huỷ bỏ một transaction và đưa cơ sở dữ liệu về trạng thái như trước khi transaction bắt đầu. Một transaction mới sẽ bắt đầu sau khi câu lệnh ROLLBACK được thực thi.
- Một transaction cũng sẽ kết thúc nếu trong quá trình thực hiện gặp lỗi (chẳng hạn hệ thống gặp lỗi, kết nối mạng,...). Trong trường hợp này, hệ thống sẽ tự động phục hồi lại trạng thái cơ sở dữ liệu như trước khi transaction bắt đầu (tương tự như khi câu lệnh ROLLBACK được thực thi để huỷ bỏ một giao tác). Tuy nhiên, trong trường hợp này sẽ không có transaction mới được bắt đầu.
- Sau khi câu lệnh ROLLBACK TRANSACTION được sử dụng để quay lui lại một điểm đánh dấu trong transaction, transaction vẫn được tiếp tục với các câu lệnh sau đó. Nhưng nếu câu lệnh này được sử dụng để quay lui lại đầu transaction, transaction sẽ kết thúc và do đó câu lệnh COMMIT TRANSACTION trong trường hợp này sẽ gặp lỗi.

4.2.1. Bắt đầu một Transaction

Cú pháp: dùng lệnh BEGIN

```
SET XACT_ABORT ON|OFF
BEGIN TRAN   [transaction_name]
Hoặc
BEGIN TRANSACTION   [transaction_name]
```

Ý nghĩa: dùng để đánh dấu đoạn lệnh bắt đầu một Transaction mà dữ liệu là đang ổn định. Transaction sẽ được thực thi cho đến khi hoặc là gặp lỗi hoặc thành công.

Tùy chọn XACT_ABORT có thể nhận một trong 2 giá trị ON hoặc OFF (OFF là mặc định).

- OFF: nếu câu lệnh T-SQL gặp lỗi, hủy toàn bộ transaction và quay lại trạng thái ban đầu.
- ON: nếu câu lệnh T-SQL phát sinh lỗi khi thực hiện, toàn bộ transaction sẽ được ngắt và rollback về savepoint hoặc trạng thái trước khi bắt đầu transaction. Tùy chọn này hay được sử dụng vì tránh được nhiều rắc rối.

4.2.2. Đánh dấu vị trí trong Transaction

Đôi khi, ta có thể quay lui Transaction trở lại một điểm nhất định mà không nhất thiết phải quay trở lại điểm bắt đầu, điều này có thể thực hiện được nhờ vào việc thiết lập một điểm đánh dấu trong Transaction, được gọi là savepoint.

Savepoint đánh dấu một vị trí mà transaction có thể trả về nếu một phần transaction bị hủy theo điều kiện. Nếu transaction roll back lại vị trí savepoint, nó phải hoàn thành các câu lệnh T-SQL kế tiếp với COMMIT, hoặc nó sẽ hủy bỏ toàn bộ transaction với ROLL BACK.

Cú pháp:

```
SAVE TRAN savepoint_name
Hoặc SAVE TRANSACTION savepoint_name
```

4.2.3. Kết thúc một Transaction

Transaction kết thúc với một trong hai câu lệnh COMMIT hoặc ROLLBACK.

4.2.3.1. Lệnh COMMIT

Khi một Transaction hoàn chỉnh được hoàn thành thì lệnh COMMIT phải được gọi ra. Đây là lệnh điều khiển Transaction được sử dụng để lưu các thay đổi gọi bởi một Transaction tới cơ sở dữ liệu.

Cú pháp:

```
COMMIT
Hoặc COMMIT  TRAN  [transaction_name]
Hoặc COMMIT TRANSACTION [transaction_name]
Hoặc COMMIT WORK [transaction_name]
```

4.2.3.2. Lệnh ROLLBACK

Lệnh ROLLBACK là lệnh điều khiển Transaction được sử dụng để trao trả Transaction về trạng thái trước khi có các thay đổi mà chưa được lưu tới Database, tức là về điểm bắt đầu hoặc về điểm dừng bên trong Transaction đó.

Lệnh ROLLBACK chỉ có thể được sử dụng để undo các Transaction trước khi xác nhận bằng lệnh Commit hay Rollback cuối cùng.

Cú pháp:

```
ROLLBACK
Hoặc ROLLBACK TRAN [transaction_name| savepoint_name]
Hoặc ROLLBACK TRANSACTION [transaction_name| savepoint_name]
```

Lệnh ROLLBACK thường được kết hợp với Khối lệnh TRY/CATCH dùng để bắt các lỗi xuất hiện và thực hiện rolled back transaction.

4.2.4. Các biến hệ thống với Transaction

Transaction có thể xảy ra lỗi sau khi thực hiện các thao tác sau:

- Insert hoặc Update: lỗi thường xảy ra khi trùng khóa chính, sai kiểu dữ liệu, sai định dạng ngày tháng,…
- Delete: lỗi thường xảy ra về ràng buộc tồn tại,…
- Select: lỗi thường xảy ra khi login không có quyền trên object,…

4.2.4.1. Biến @@trancount

Biến @@trancount: cho biết số transaction hiện đang thực hiện (chưa được kết thúc với rollback hay commit) trong connection hiện hành.

Mỗi khi câu lệnh BEGIN TRAN được thực thi, biến hệ thống @@trancount được tự động tăng lên 1, và cũng tự động giảm đi 1 khi kết thúc một Transaction.

4.2.4.2. Biến @@error

Biến @@error cho biết số mã lỗi của câu lệnh được thực hiện gần nhất. Khi một câu lệnh thực hiện thành công thì giá trị là 0.

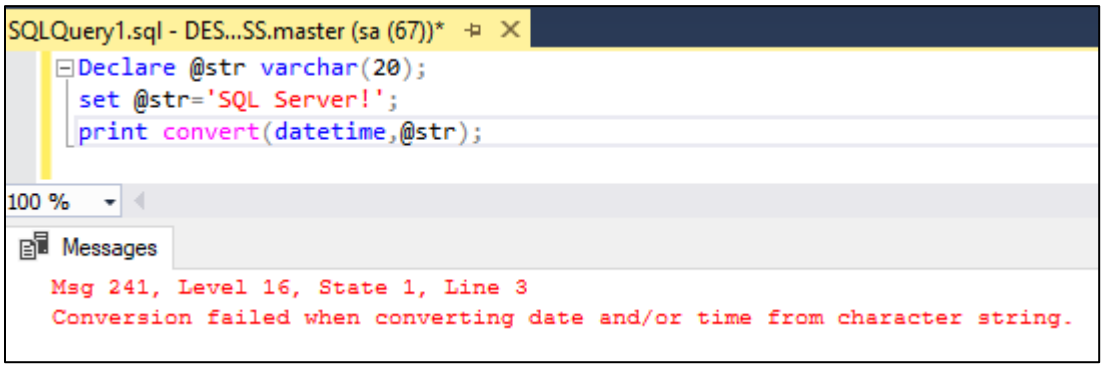
4.2.4.3. Biến @@rowcount

Biến @@rowcount trả về tổng số mẫu tin được tác động của câu lệnh truy vấn gần nhất.

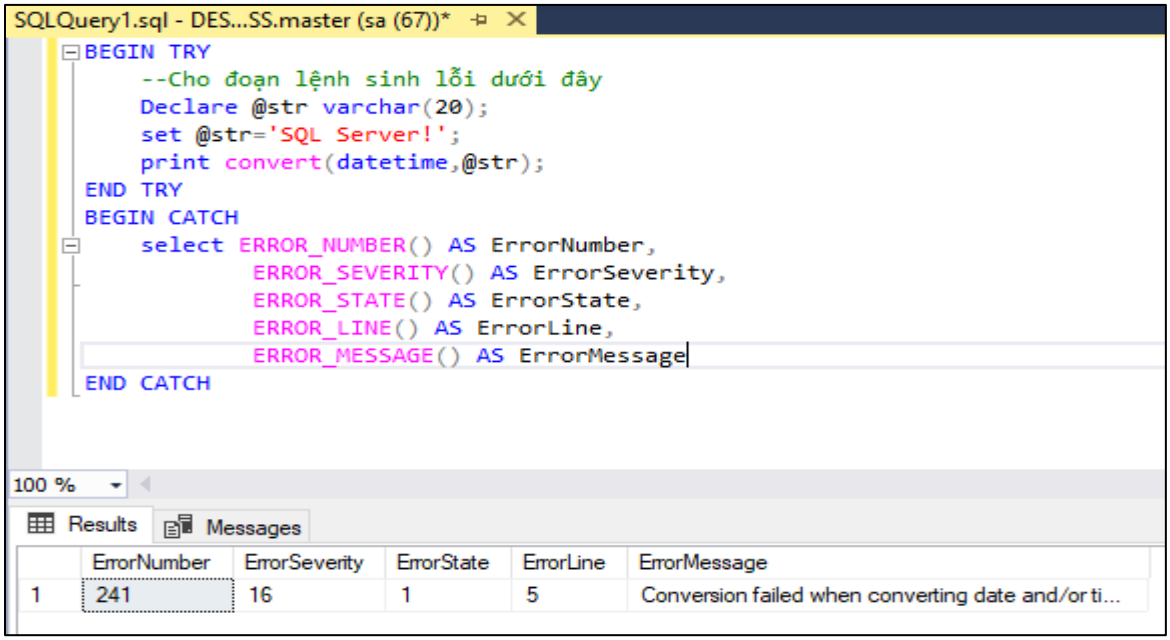
4.3. Xử lý lỗi với TRY…CATCH

SQL Server cung cấp cơ chế kiểm soát lỗi bằng TRY…CATCH như các ngôn ngữ lập trình phổ biến hiện nay.

Xét ví dụ sau:



Đoạn lệnh trên sinh lỗi, nhưng SQL Server không hiển thị cụ thể lỗi, vì vậy ta có thể dùng TRY…CATCH và các hàm phát sinh lỗi như sau:



Ý nghĩa các hàm cung cấp thông tin lỗi thông dụng gồm:

STT	TÊN HÀM	CHỨC NĂNG
1	ERROR_NUMBER()	Trả lại mã lỗi (dưới dạng số)
2	ERROR_SEVERITY()	Trả lại mức độ nghiêm trọng của lỗi
3	ERROR_STATE()	Trả lại trạng thái của lỗi (dưới dạng số)
4	ERROR_PROCEDURE()	Trả lại tên của Store Procedure hoặc tên của Trigger đã phát sinh lỗi
5	ERROR_LINE()	Trả lại vị trí dòng lệnh đã phát sinh ra lỗi
6	ERROR_MESSAGE()	Trả lại thông báo lỗi dưới hình thức văn bản (text)

4.4. Ví dụ

```
--tạo thủ tục xuất báo cáo cho
CREATE PROC prReportError
AS
   SELECT
      ERROR_NUMBER() AS ErrorNumber,
            ERROR_SEVERITY() AS ErrorSeverity,
            ERROR_STATE() AS ErrorState,
            ERROR_LINE () AS ErrorLine,
            ERROR_PROCEDURE() AS ErrorProcedure,
            ERROR_MESSAGE() AS ErrorMessage;
GO
--tạo thủ tục có transaction
CREATE PROC pr_DeletetblCustomer
(
       @cid char(7)
)
AS
BEGIN
   BEGIN TRY
      BEGIN TRANSACTION;
      -- delete the customer
      DELETE FROM tblCustomer
      WHERE CustID = @cid;
      -- if DELETE succeeds, commit the transaction
      COMMIT TRANSACTION;
   END TRY
   BEGIN CATCH
      -- report exception
      EXEC prReportError
      -- Test if the transaction is uncommittable.
      IF (XACT_STATE()) = -1
      BEGIN
         PRINT  N'The transaction is in an uncommittable state.' +
               'Rolling back transaction.'
         ROLLBACK TRANSACTION;
      END;
      -- Test if the transaction is committable.
      IF (XACT_STATE()) = 1
```

```
        BEGIN
            PRINT N'The transaction is committable.' +
                'Committing transaction.'
            COMMIT TRANSACTION;
        END;
    END CATCH
END;
```

Trong đoạn lệnh trên, XACT_STATE() để chỉ ra rằng có transaction nào được kích hoạt không và transaction có được commit không

4.5. Transaction lồng nhau

Transaction lồng nhau (Nested Transaction) là Transaction được viết bên trong một Transaction khác.

Mục đích chủ yếu của Transaction lồng nhau là để cho phép các procedure có chứa transaction có thể được gọi từ những tiến trình đã nằm bên trong một transaction hoặc từ những tiến trình không nằm bên trong một transaction nào.

CHƯƠNG 5. QUẢN LÝ TƯƠNG TRANH

SQL Server nói riêng và các cơ sở dữ liệu quan hệ nói chung là dạng cơ sở dữ liệu có độ nhất quán dữ liệu cao nhất (consistency) tại mọi thời điểm. Nghĩa là với cùng một dữ liệu thì không thể có việc đọc ghi tại cùng một thời điểm. Việc đọc ghi cùng một dữ liệu sẽ dẫn tới hiện tượng lock lẫn nhau để đảm bảo dữ liệu được consistency. Điều đó dẫn tới một số tình huống sau:

- Lệnh Select sẽ xung đột với các lệnh Update, Insert, Delete khi dữ liệu bị thay đổi tại một thời điểm mà đồng thời thực hiện lệnh select thì sẽ không thể insert, update, delete trên cùng một table hoặc ngược lại.
- Các lệnh Select không lock lẫn nhau. Các lệnh select thự hiện chế độ share lock, nghĩa là việc thực hiện lệnh đọc đồng thời cùng một dữ liệu thì không bị ảnh hưởng lẫn nhau.
- Các lệnh Insert, Update, Delete lock theo row không lock theo table. Nghĩa là có thể thực hiện đồng thời các lệnh đó trên cùng một table miễn là chúng không tranh chấp nhau cùng một row dữ liệu. Lock để đảm bảo tính consistency data là một trong các ưu điểm nổi bật của cơ sở dữ liệu quan hệ.

5.1. Đặt vấn đề

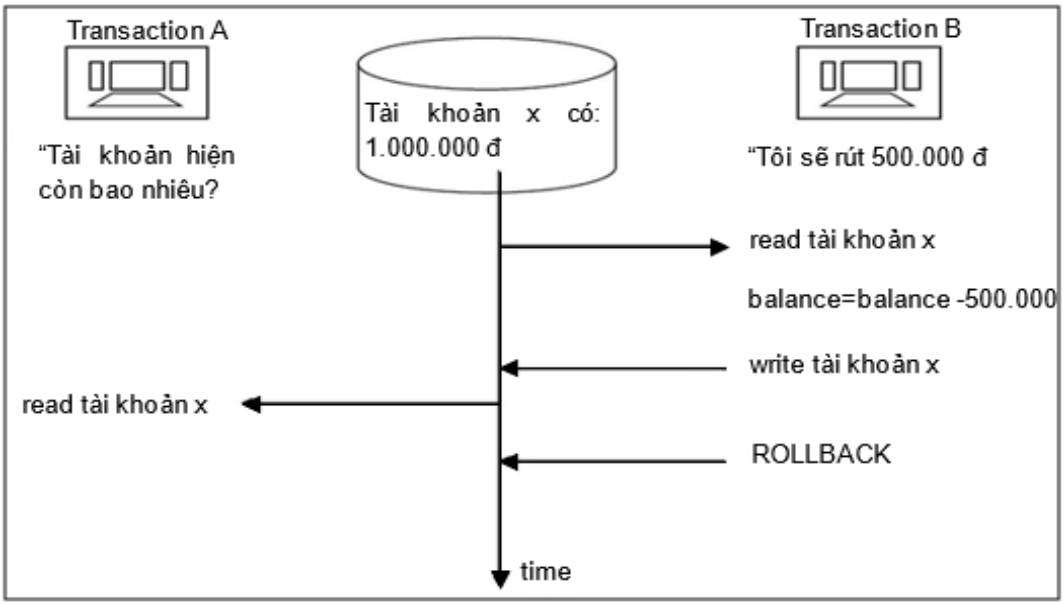
Một ví dụ điển hình của tranh chấp là bài toán sau: giả sử tài khoản x đang có một khoản tiền, và nếu có 2 giao dịch ngân hàng A và B đồng thời được thực hiện cho tài khoản x này. Khi đó, có 2 việc cần thực hiện:

1. Thực hiện giao dịch A
2. Thực hiện giao dịch B

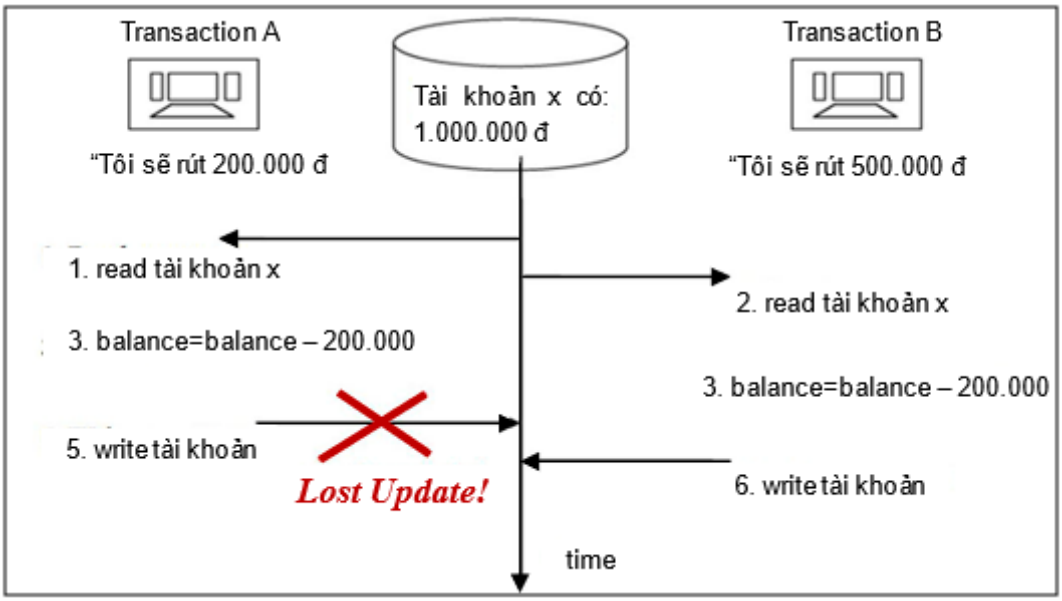
Hai công việc này phải đảm bảo hoặc là cả 2 việc đều phải thành công, hoặc là không có công việc nào thành công. Nghĩa là nếu một công việc vì một lý do nào đó không thực hiện thành công thì trạng thái ban đầu trước khi chuyển tiền phải được khôi phục để bảo toàn dữ liệu.

5.2. Các vấn đề liên quan đến truy xuất đồng thời

- **Dirty Read (Đọc dữ liệu sai)**: xảy ra khi một transaction thực hiện đọc trên một đơn vị dữ liệu mà đơn vị dữ liệu này đang bị cập nhật bởi một transaction khác mà việc cập nhật lại chưa được xác nhận là đã hoàn tất.
- **Lost Update (Cập nhật mất dữ liệu)**: tình trạng này xảy ra khi có nhiều hơn một transaction cùng thực hiện đọc và cập nhật trên một đơn vị dữ liệu. Khi đó, tác dụng của transaction cập nhật thực hiện sau sẽ đè lên tác dụng của transaction cập nhật trước.

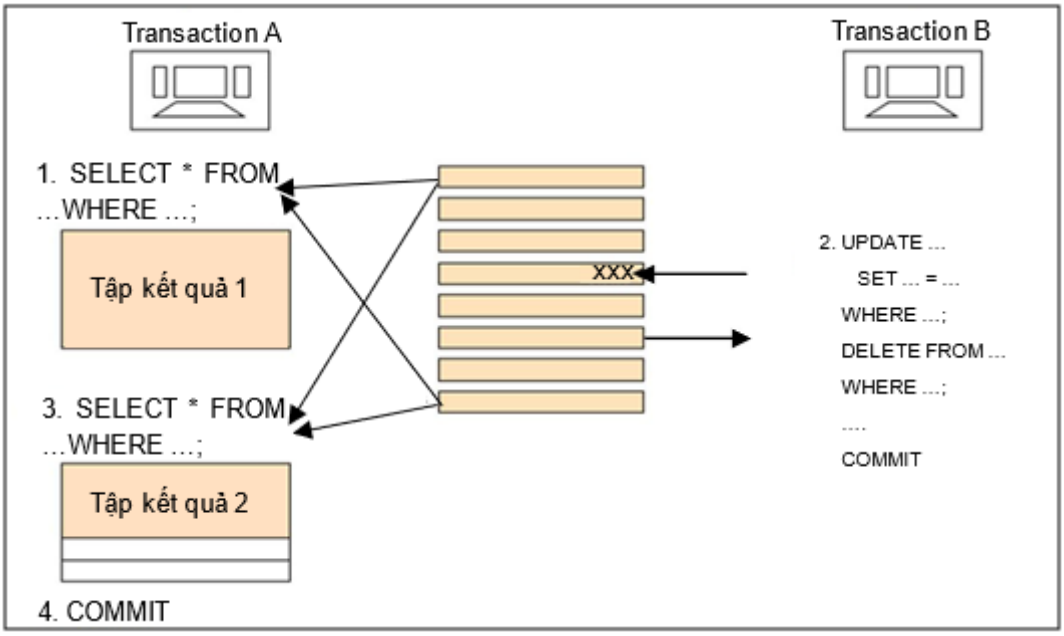


Hình 5.1: Minh họa Dirty Read

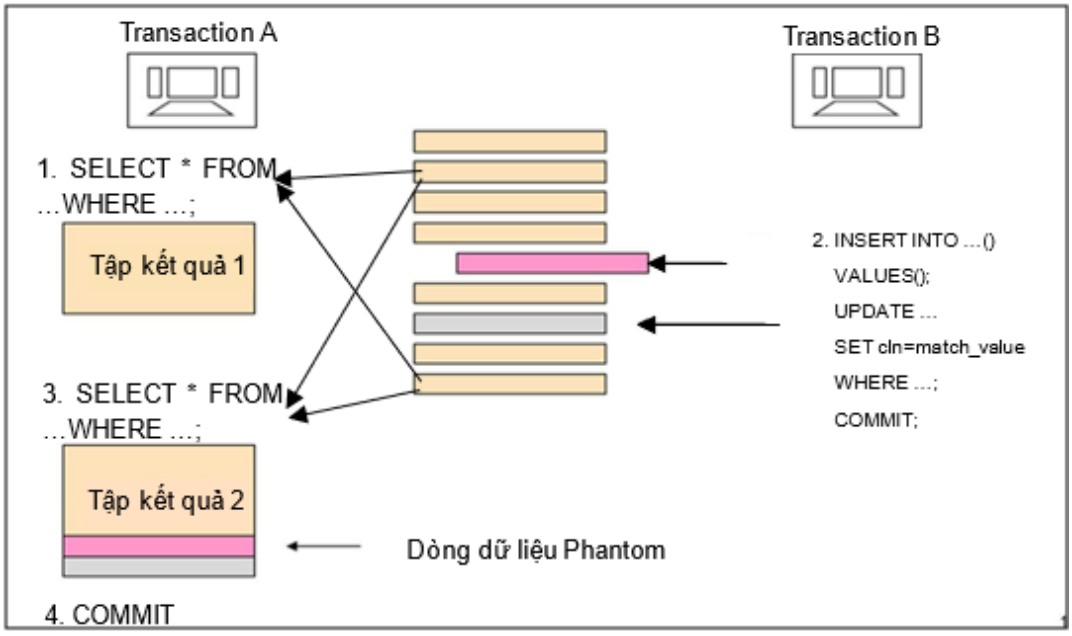


Hình 5.2: Minh họa Lost Update

- **Nonrepeatable Read (Đọc không lặp lại)**: tình trạng này xảy ra khi một transaction A vừa thực hiện xong thao tác đọc trên một đơn vị dữ liệu thì transaction B khác lại muốn cập nhật trên đơn vị dữ liệu này. Điều này làm cho lần đọc sau đó của transaction A không còn nhìn thấy dữ liệu ban đầu nữa.
- **Phantom Read (Đọc các mẫu tin ma)**: là tình trạng mà một transaction A đang thao tác trên một tập dữ liệu nhưng một transaction B khác lại chèn thêm hoặc xóa bớt đi các dòng dữ liệu mà transaction A quan tâm.



Hình 5.3: Minh họa Nonrepeatable Read



Hình 5.4: Minh họa Phantom Read

Điều khiển truy xuất đồng thời giúp cho nhiều Transaction diễn ra đồng thời mà không xảy ra tranh chấp. Việc này có thể được thực hiện nhờ vào:

- Quản lý lịch biểu (Schedule): là quản lý sao cho việc thực hiện đồng thời các transaction cho kết quả cũng giống như khi thực hiện tuần tự các transaction.
- Dùng khóa (Lock)
- Dùng nhãn thời gian (Timestamp)

5.3. Các khái niệm liên quan đến truy xuất đồng thời

5.3.1. Lịch biểu (Schedule)

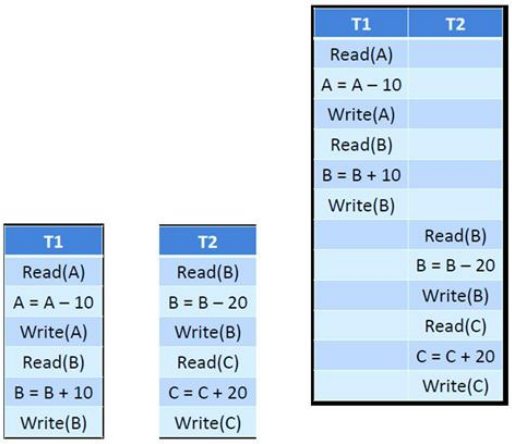
Bộ lập lịch là một bộ phận của hệ quản trị cơ sở dữ liệu chịu trách nhiệm giải quyết các yêu cầu đụng độ, ép transaction phải chờ trong trường hợp chưa đáp ứng được yêu cầu lock, hay hủy bỏ transaction khi xảy ra deadlock, hay khi định ra một thứ tự thực hiện các giao tác.

5.3.1.1. Lịch biểu (Schedule)

Lịch biểu là một dãy (có thứ tự) các thao tác của một tập các transaction mà trong đó thứ tự của các thao tác trong mỗi transaction được bảo toàn.

5.3.1.2. Lịch biểu tuần tự (Serial Schedule)

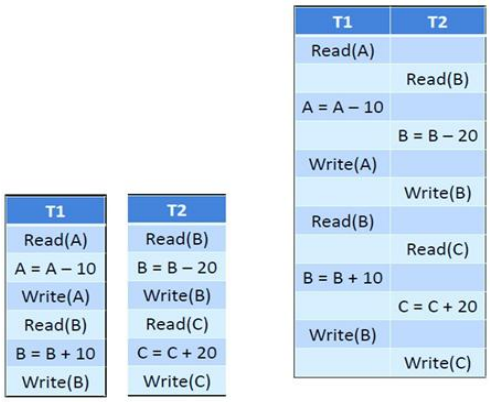
Lịch biểu tuần tự là lịch biểu mà trong mỗi transaction các thao tác được thực hiện kế tiếp nhau, không có thao tác của transaction khác xen vào (nghĩa là thực hiện lần lượt, hết transaction này đến transaction khác).



Hình 5.5: Minh họa Serial Schedule

5.3.1.3. Lịch biểu không tuần tự (Nonserial Schedule)

Lịch biểu không tuần tự là lịch biểu mà các thao tác trong các transaction được đan xen vào nhau.



Hình 5.6: Minh họa Nonserial Schedule

5.3.1.4. Lịch biểu khả tuần tự (Serializable Schedule)

Lịch biểu khả tuần tự nếu nó tương đương với một lịch biểu tuần tự, tương đương theo nghĩa cho ra cùng một trạng thái database sau khi kết thúc việc thực hiện lịch biểu.

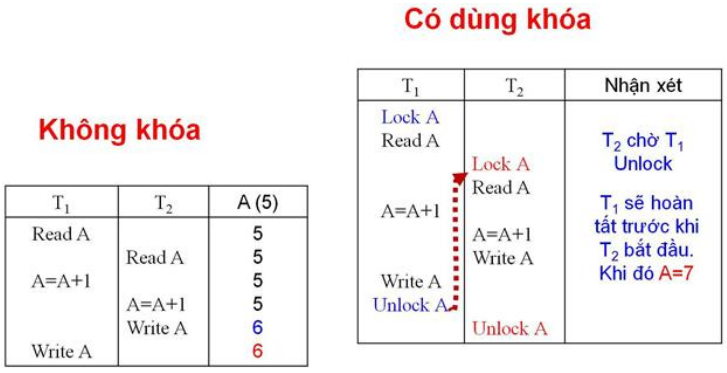
5.3.2. Khóa (Lock)

SQL Server có thể tạo ra các loại khoá sau:

- RID (Row Identifier): khoá 1 hàng trong bảng
- Key: khoá 1 hàng trong bảng index
- Table: khoá tất cả các hàng và chỉ mục của 1 bảng
- Database: được dùng khi lưu trữ cả database
- Page: khoá 1 trang dữ liệu hay trang chỉ mục
- Extent: khoá 1 nhóm các trang trong lúc phân phối không gian lưu trữ

Phương pháp thông dụng để quản lý việc truy xuất các đơn vị dữ liệu là sử dụng khóa.

- Khóa là đặc quyền truy xuất trên các đơn vị dữ liệu của một transaction.
- Khi một transaction thực hiện việc lock trên đơn vị dữ liệu n – ta nói transaction đó đang giữa lock trên n.
- Đơn vị dữ liệu: có thể là database, hoặc table, hoặc extend (=8 pages), hoặc page, hoặc row
- Tại mỗi thời điểm, chỉ có một tập con các đơn vị dữ liệu bị khóa, vì vậy bộ quản lý khóa có thể lưu các lock hiện hành trong một bảng lock với các mẫu tin có dạng (n,L,T) – giao tác T có một khóa kiểu L trên đơn vị dữ liệu n.
- Một transaction khi có yêu cầu truy xuất đến đơn vị dữ liệu đã được lock thì phải phát yêu cầu xin lock trên đơn vị dữ liệu đó, nếu yêu cầu được chấp thuận thì transaction sẽ được quyền thao tác trên đơn vị dữ liệu đó, và lưu ý là các transaction khác sẽ không được phép truy cập đến đơn vị dữ liệu đó cho đến khi transaction giữ khóa được unlock



Hình 5.7: Minh họa trình tự thao tác có và không dùng khóa

Trên thực tế, có khi transaction chỉ cần read dữ liệu của một đơn vị dữ liệu chứ không thay đổi giá trị dữ liệu, nếu không phân biệt lock cho thao tác Read hoặc Write thì khả năng sẽ có nhiều transaction phải chờ để được quyền khóa trên một đơn vị dữ liệu. Giải pháp cho vấn đề này là tách lock thành 2 yêu cầu lock riêng biệt:

- Read Lock
- Wirte Lock

5.3.2.1. Shared Lock (Read Lock)

Khi read một đơn vị dữ liệu, SQL Server tự động thiết lập Shared Lock trên đơn vị dữ liệu đó (trừ trường hợp đơn vị dữ liệu đó có sử dụng No Lock).

Shared Lock có thể được thiết lập trên một table, một page hay một row dữ liệu.

Nhiều transaction có thể đồng thời giữ Shared Lock trên cùng một đơn vị dữ liệu.

Shared Lock thường được giải phóng ngay sau khi sử dụng xong dữ liệu được đọc (trừ khi có yêu cầu giữ Shared Lock cho đến khi hết transaction, ví dụ như yêu cầu RepeatableRead, Serializable, Holdlock).

5.3.2.2. Exclusive Lock (Write Lock)

Khi thực hiện thao tác ghi dữ liệu (bởi câu lệnh Insert, Update, Delete) trên một đơn vị dữ liệu, SQL Server tự động thiết lập Exclusive Lock trên đơn vị dữ liệu đó. Lock này được áp dụng khi thực sự cập nhật dữ liệu.

Exclusive Lock luôn được giữ đến hết transaction.

Tại một thời điểm, chỉ có tối đa một transaction được quyền giữ Exclusive Lock trên một đơn vị dữ liệu.

Lưu ý: không thể thiết lập Exclusive Lock trên đơn vị dữ liệu đang có Shared Lock.

5.3.2.3. Update Lock

Update Lock sử dụng khi đọc dữ liệu với dự định ghi trở lại sau khi đọc đơn vị dữ liệu này. Lock này được áp dụng khi có khả năng diễn ra thao tác cập nhật dữ liệu.

Update Lock là chế độ khóa trung gian Shared Lock và Exclusive Lock.

Tại một thời điểm, chỉ có tối đa một transaction được quyền giữ Update Lock, và không ngăn cản việc thiết lập các Shared Lock khác trên cùng một đơn vị dữ liệu này.

Update Lock giúp tránh hiện tượng Deadlock khi có yêu cầu chuyển từ Shared Lock lên Exclusive Lock trên một đơn vị dữ liệu nào đó (do tại một thời điểm chỉ có tối đa một Update Lock trên một đơn vị dữ liệu).

5.3.2.4. Live Lock

Trong trường hợp một transaction chờ được cấp quyền lock trên một đơn vị dữ liệu nào đó mà không xác định được thời điểm được đáp ứng nhu cầu.

Trong trường hợp này, bộ lập lịch sẽ tiến hành giải quyết vấn đề thông qua cơ chế: transaction nào yêu cầu trước thì được đáp ứng trước (FIFO).

5.3.2.5. Intent Lock

Dùng giải quyết phân cấp đối tượng. Intent Lock chỉ giải quyết đến bảng chứ không quan tâm đến từng bản ghi trong bảng.

SQL Server muốn đạt được khoá loại shared hay exclusive trên 1 số tài nguyên mức thấp hơn theo thứ tự phân cấp

5.3.2.6. Schema Lock

SQL Server xem xét các khoá làm thay đổi chema khi bất kỳ lệnh DDL (data definition language) được thực thi trong bảng:

- Schema modification locks (Sch-M): giản đồ thay đổi cách tạo đối tượng, không yêu cầu các phát biểu CREATE, ALTER, hay DROP.
- Schema stability lock (Sch-S): tương tự như Share lock, lock này ngăn ngừa các yêu cầu của các phát biểu CREATE, ALTER, DROP khi đã thiết lập Schema modification lock

5.3.2.7. Deadlock

Trong trường hợp những transaction có liên quan không thể thực hiện tiếp các thao tác của nó mà cứ phải chờ nhau mãi giữa các transaction.

T_1	T_2	Nhận xét
Lock A		- T_1 chờ T_2 unlock B
	Lock B	- T_2 chờ T_1 unlock A
	Lock A	→ Deadlock
Lock B		

Hình 5.8: Minh họa Deadlock

Trong hình trên, T_1 yêu cầu và được trao khóa trên đơn vị dữ liệu A, còn T_2 yêu cầu và được trao khóa trên đơn vị dữ liệu B. Do đó, khi T_1 yêu cầu khóa trên B, nó sẽ phải đợi vì lúc này T_2 đã khóa trên B.

Tương tự, khi T_2 yêu cầu khóa trên A, nó cũng buộc phải đợi vì T_1 đã khóa trên A.

Kết quả là, không một transaction nào tiếp tục hoạt động được, mỗi transaction đều phải đợi cho transaction kia mở khóa mà chẳng bao giờ nhận được khóa nó yêu cầu.

Chiến lược giảm thiểu nguy cơ Deadlock:

- Giảm thiểu số lượng bảng cần tác động trong một transaction, transaction có số bảng tham gia càng ít thì khả năng xảy ra deadlock càng thấp.
- Kiểm soát việc thực thi các transaction cho thứ tự tác động các bảng là giống nhau như vậy sẽ chỉ xảy ra hiện tượng lock chứ không xảy ra hiện tượng deadlock.
- Thực hiện giao dịch dạng batch thay vì gọi nhiều lần từ trong ứng dụng tới database. Nghĩa là thay vì gọi rời rạc thì generate ra một script duy nhất rồi gửi vào database hoặc gọi store procedure để thực thi giao dịch. Việc này giúp giảm thiểu deadlock vì hai lý do sau:
  + Giảm thời gian round trip qua đường mạng do việc gọi đi gọi lại do đó giao dịch kết thúc nhanh hơn.
  + Việc gửi script dạng batch, hoặc thực thi store procedure giúp sql server phân tích được toàn bộ mã script từ đó tạo ra một execution plan hiệu quả, khả năng tránh deadlock cao hơn. Không như việc gọi thực thi nhiều lần từ trong ứng dụng, SQL

Server không thể biết sau lệnh thứ nhất thì lệnh tiếp theo là gì do đó không thể tạo ra chiến lược giải quyết deadlock.

5.3.2.8. Lưu ý khi sử dụng Lock

- SQL Server có nhiều mức khóa khác nhau cho phép các loại tài nguyên khác nhau được khoá bởi transaction.
- Để giảm việc hao tổn khi thực hiện khóa, SQL Server khoá tài nguyên một cách tự động ở mức phù hợp với nhiệm vụ cần thực hiện.
- Việc khoá ở mức càng nhỏ, ví dụ ở mức các hàng của bảng, làm tăng tính đồng thời, nhưng có phí tổn cao bởi vì nhiếu khoá được tạo ra nếu nhiều hàng được khoá.
- Việc khoá ở mức càng lớn, chẳng hạn mức bảng, sẽ gây ra lãng phí khi xét đến tính đồng thời vì việc khoá cả bảng sẽ hạn chế việc truy xuất đến bất kỳ phần nào của bảng đó, nhưng chi phí sẽ giảm bởi vì chỉ có 1 ít khoá cần được quản lý.

5.3.3. Nhãn thời gian (Timestamp)

Nhãn thời gian là con số được phát sinh bởi bộ lập lịch, được gán cho mỗi transaction để chỉ định thời điểm bắt đầu thực hiện transaction.

Nhãn thời gian có tính duy nhất và tăng dần: $T_i < T_j$

5.4. Các mức cô lập dữ liệu (Isolation Level)

Mục đích của các mức cô lập dữ liệu nhằm tự động đặt khóa cho các thao tác Read hoặc Write trong kết nối cơ sở dữ liệu hiện hành. Như vậy, tầm ảnh hưởng của các mức cô lập là ở mức connection chứ không ở mức transaction.

Các mức độ cô lập trong SQL Server và các ảnh hưởng có thể được liệt kê như bảng dưới đây:

- **Read Uncommitted Isolation**: các truy vấn trong một transaction bị ảnh hưởng bởi các thay đổi chưa được cam kết trong một transaction khác. Khoá không được dùng ở mức này.
- **Read Committed Isolation with Locks**: thiết lập mặc định trong SQL Server. Các cập nhật được cam kết bởi một transaction có thể được nhìn thấy bởi các transaction khác. Các truy vấn hay các kết hợp (aggregations) có thời gian thực thi lâu sẽ không được yêu cầu.
- **Read Committed Isolation with Snapshots**: chỉ các cập nhật được cam kết mới được thấy bởi các transaction khác và các truy vấn hay các kết hợp (aggregations) có thời gian thực thi lâu được yêu cầu. Khoá không được yêu cầu và cơ chế **row versioning** được sử dụng để giám sát sự thay đổi phiên bản của hàng. Lưu trữ phiên bản hàng sử dụng ở mức này có thể gây tốn kém chi phí (tài nguyên) nhưng nó sẽ tăng thông lượng (throughput) bằng cách giảm nội dung dữ liệu bị khoá.
- **Repeatable Read Isolation**: Với một transaction, việc đọc dữ liệu là nhất quán; các transaction khác không thể gây ảnh hưởng đến kết quả truy vấn của chúng ta cho đến

khi transaction hoàn thành và khoá được giải phóng. Mức này được ưu tiên sử dụng khi chúng ta đọc dữ liệu nhằm mục đích chỉnh sửa trong cùng một transaction.

- **Snapshot Isolation**: được dùng khi các truy vấn thời gian dài và các transaction nhiều lệnh được yêu cầu nhưng không có kế hoạch cập nhật dữ liệu. Khoá không được yêu cầu cho quá trình đọc dữ liệu bởi vì các thay đổi không thể được nhìn thấy cho đến khi snapshot hoàn thành và các transaction cam kết.
- **Serializable Isolation:** đặt một khoá phạm vi (a range lock) – là khoá nhiều hàng (multi-row lock) - trên tập hàng đầy đủ được truy cập, ngăn chặn người dùng khác cập nhật hay chèn các hàng mới vào tập dữ liệu cho đến khi transaction hoàn thành. Đây là mức nghiêm ngặt nhất vì nó sử dụng rất nhiều khoá (tốn chi phí nhất) nên cần cân nhắc cẩn thận trước khi sử dụng.

CHƯƠNG 6. CÁC VẤN ĐỀ THỰC TIỄN TRONG DBMS

6.1. Tìm kiếm Full-text

Trong các ứng dụng, Application hay Website, thì chức năng tìm kiếm hầu như luôn cần thiết.

Thông thường, ta sử dụng từ khóa LIKE để trả về kết quả gần đúng, và đôi khi kết quả không như mong muốn.

Ví dụ:

Hiển thị tất cả những nhân viên mà tên nhân viên có chứa ký tự 'nguyen', hay 'nguyên' hay 'nguyễn'... đều cần được trả về kết quả

```
select *
from tblEmployee
where EmpName like N'%nguyen%'
```

Kết quả: (chỉ trả về 0 row dữ liệu trước khi cài Full-text Search)

```
4  select *
5  from tblEmployee
6  --những nhân viên mà Name có từ 'nguyen' khi chưa cài full-text Search
7  select *
8  from tblEmployee
9  where EmpName like N'%nguyen%'
```

131 %

Results | Messages

	EmpID	EmpName	TelNo	Email	Position	Gender	DoB	Salary
1	NV00001	Trần Thu Dung	0933285777	dungtt@gmail.com	Nhân viên bán hàng	F	2000-01-01	3000000.00
2	NV00002	Đinh Thị Mỹ Hạnh	NULL	hanhdtm@gmail.com	Nhân viên bán hàng	F	2000-12-13	4000000.00
3	NV00003	Nguyễn Hữu Thịnh	0909285777	thinhnh@gmail.com	Nhân viên giao hàng	M	1998-11-01	3000000.00
4	NV00004	Hoàng Thái Tuấn	0913285235	tuanht@gmail.com	Nhân viên giao hàng	M	1990-01-03	3000000.00
5	NV00005	Lê Anh Tú	0933285123	tula@gmail.com	Nhân viên kho	M	1984-01-11	3500000.00
6	NV00006	Hà Lý Thụy Vân	0936285346	vanhlt@gmail.com	Nhân viên bán hàng	F	1998-02-21	4500000.00
7	NV00007	Đỗ Thị Ngọc Thoại	0966282346	thoaidtn@gmail.com	Nhân viên bán hàng	F	1993-05-05	4800000.00
8	NV00008	Lê Thị Chương Dương	0917000346	duongltc@gmail.com	Nhân viên bán hàng	F	1993-07-10	4600000.00
9	NV00009	Dương Thị Duyên An	0978341346	andtd@gmail.com	Nhân viên bán hàng	F	1993-12-13	4580000.00
10	NV00010	Vũ Thị Hoàng Ngân	0987336776	nganvth@gmail.com	Nhân viên bán hàng	F	1993-03-05	4900000.00
11	NV00011	Đặng Hồng Lê Thúy	0976485346	thuydhl@gmail.com	Nhân viên bán hàng	F	1993-06-06	5000000.00

EmpID	EmpName	TelNo	Email	Position	Gender	DoB	Salary

Hệ quản trị cơ sở dữ liệu SQL Server có tích hợp chức năng Full-text Search, cụ thể:

- Tìm từ đồng nghĩa (ví dụ với đầu vào tìm kiếm là "man" ➔ sẽ tìm các từ có "man" hoặc "men")
- Tìm từ gần đúng so với từng từ trong cụm từ, hay theo trật tự đảo ngược
- Hỗ trợ tìm kiếm có dấu tiếng việt (ví dụ với đầu vào tìm kiếm là "nhan vien" ➔ kết quả tìm kiếm đầu ra vẫn có thể tìm được "nhân viên")
- Loại bỏ từ khóa nhiễu NOISE

Các bước sử dụng Full-text Search trong SQL Server:

i. Kiểm tra xem tính năng Full-text Search đã được thiết lập hay chưa?

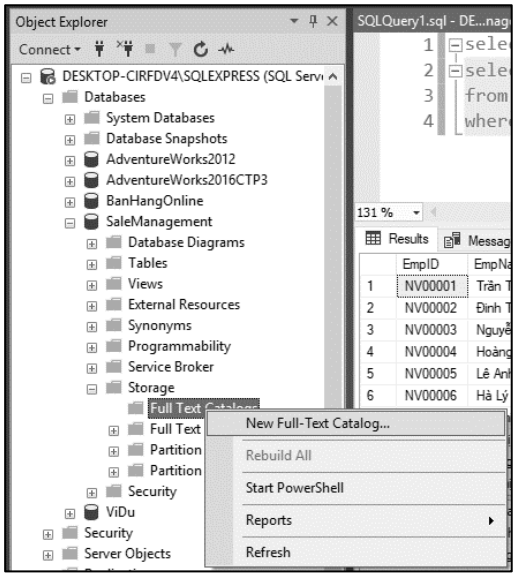
Cú pháp:

```
SELECT SERVERPROPERTY('IsFullTextInstalled')
```

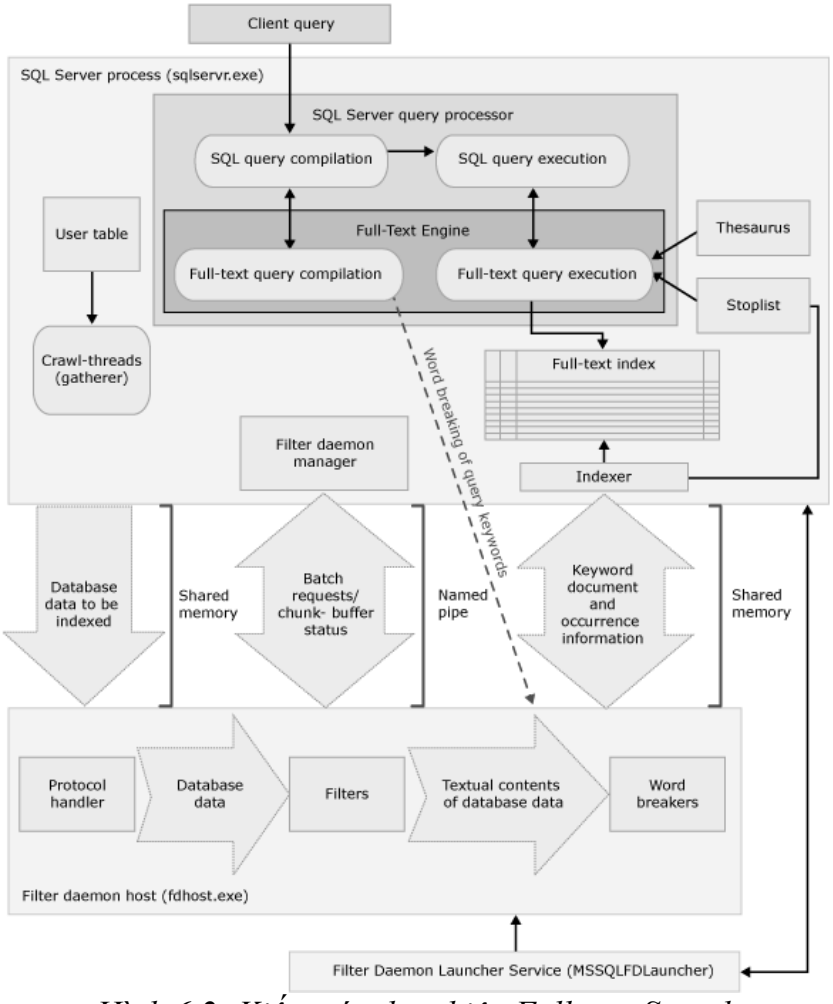
Nếu kết quả trả về là 0, nghĩa là tính năng Full-text Search chưa được cài đặt.

ii. Cài đặt tính năng Full-text Search

+ Vào New Full-text catalog (hình 6.1)



Hình 6.1: Giao diện khởi động New Full-Text Search



Hình 6.2: Kiến trúc thực hiện Full-text Search

+ Đặt tên cho Full-text catalog
  - Thiết lập phần dấu (Accent): Insensitive
  - Nếu Full-text Search Service chưa được kích hoạt thì sẽ bị báo lỗi ➔ cần kích hoạt service này
    - ➢ SQL Full-text Filter Deamon Laucher (MSSQLSERVER), đây là option trong quá trình cài đặt SQL Server. Có thể cài bổ sung từ bằng cách Add từ Programs & Features hay Programs trong Control Panel ➔ trỏ đến thư mục Setup của SQLServer2017Media ➔ Next click cho đến mục Feature Selection ➔ tick chọn Database Engine Server ➔ tick chọn Full-text Search
    - ➢ Khởi động SQL Full-text Filter Deamon Laucher (MSSQLSERVER)
+ Tạo Full-text Index cho cột Name (bằng T-SQL hoặc Wizard) cho table muốn tạo full-text search: Right Click vào table (ví dụ Employee) ➔ chọn Full-text Index ➔ Define Full-text Index ➔ select an Index for table ➔ select column_name muốn thực hiện index (chọn Language là Vietnamese) ➔ nếu muốn full-text index tự động cập nhật khi có thay đổi dữ liệu thì chọn "Automatically", nếu chọn "Manually" thì phải thiết lập Schedule để cập nhật index ➔ click Finish

iii. Thực hiện test lại với câu lệnh trên

Ví dụ: Kết quả sau khi đã cài Full-text Search (trả về 3 rows)

```
12  select *
13  from tblEmployee
14  --những nhân viên mà Name có từ 'nguyen' sau khi đã cài full-text Search
15  select *
16  from tblEmployee
17  where contains(EmpName,N'nguyen')
```

131 %

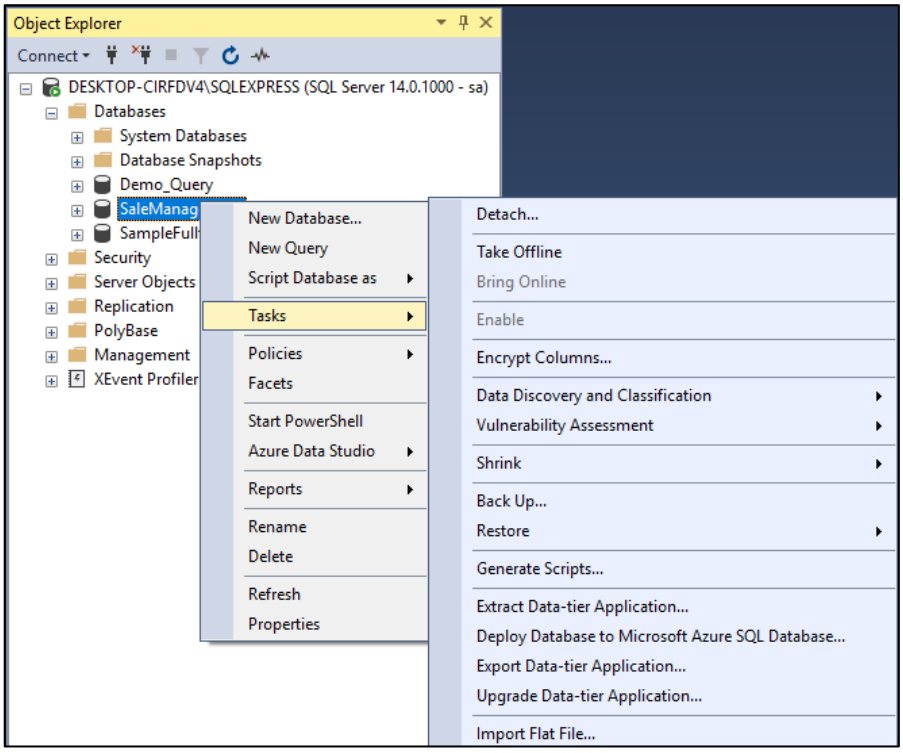
Results | Messages

	EmpID	EmpName	TelNo	Email	Position	Gender	DoB	Salary
1	NV00001	Trần Thu Dung	0933285777	dungtt@gmail.com	Nhân viên bán hàng	F	2000-01-01	3000000.00
2	NV00002	Đinh Thị Mỹ Hạnh	NULL	hanhdtm@gmail.com	Nhân viên bán hàng	F	2000-12-13	4000000.00
3	NV00003	Nguyễn Hữu Thịnh	0909285777	thinhnh@gmail.com	Nhân viên giao hàng	M	1998-11-01	3000000.00
4	NV00004	Hoàng Thái Tuấn	0913285235	tuanht@gmail.com	Nhân viên giao hàng	M	1990-01-03	3000000.00
5	NV00005	Lê Anh Tú	0933285123	tula@gmail.com	Nhân viên kho	M	1984-01-11	3500000.00
6	NV00006	Hà Lý Thụy Vân	0936285346	vanhlt@gmail.com	Nhân viên bán hàng	F	1998-02-21	4500000.00
7	NV00007	Đỗ Thị Ngọc Thoại	0966282346	thoaidtn@gmail.com	Nhân viên bán hàng	F	1993-05-05	4800000.00
8	NV00008	Lê Thị Chương Dương	0917000346	duongltc@gmail.com	Nhân viên bán hàng	F	1993-07-10	4600000.00

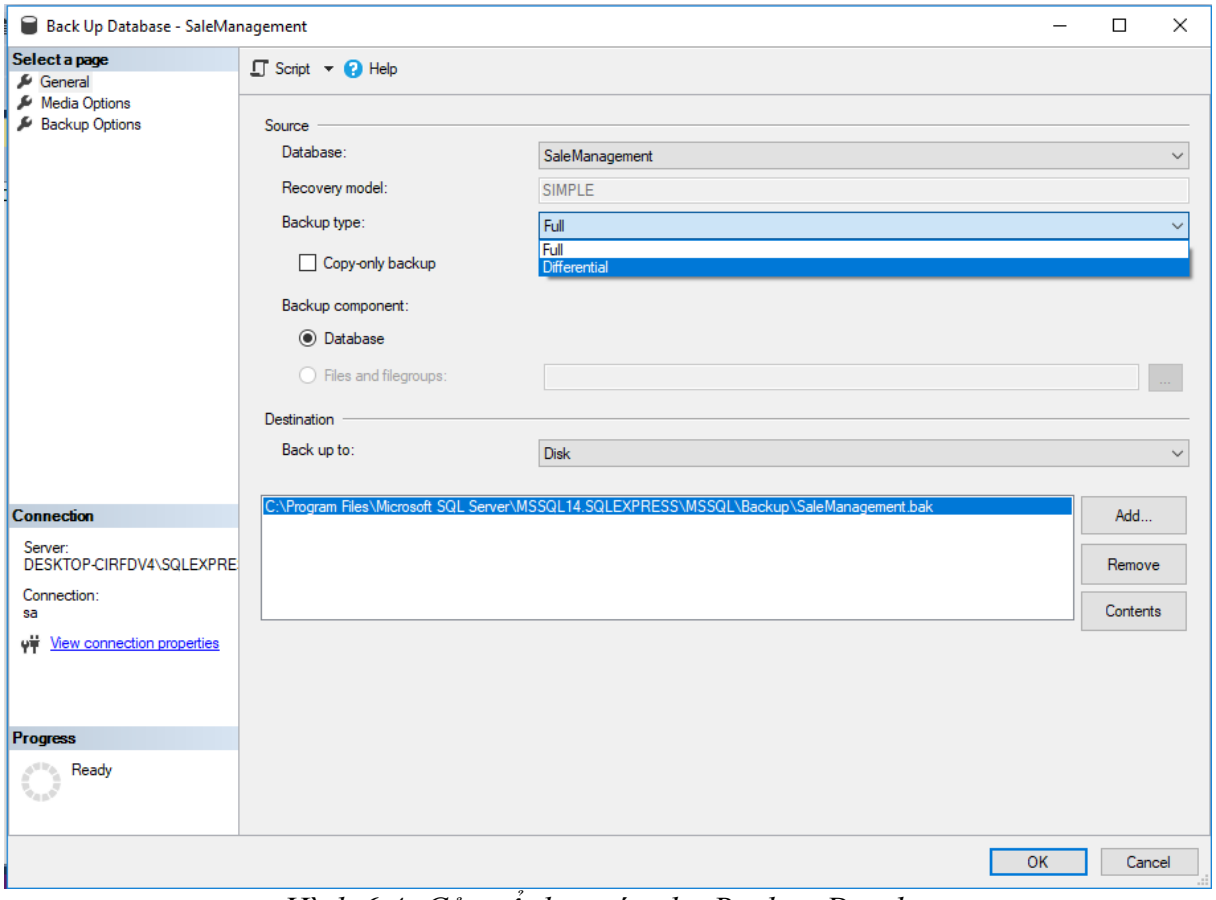
	EmpID	EmpName	TelNo	Email	Position	Gender	DoB	Salary
1	NV00003	Nguyễn Hữu Thịnh	0909285777	thinhnh@gmail.com	Nhân viên giao hàng	M	1998-11-01	3000000.00
2	NV00014	Nguyễn Việt Hồng Hiếu	0933312345	hieunvh@gmail.com	Quản lý viên	F	1981-03-07	10500000.00
3	NV00018	Quyên Nguyễn	0337729833	spktquyennth@g...	Nhân viên kho	M	NULL	4500000.00

6.2. Sao lưu dữ liệu (Database Backup)

Microsoft SQL Server cung cấp hai kỹ thuật sao lưu Database (CSDL) chính: full backup và differential backup.



Hình 6.3: Giao diện khởi động Detach/Attach/Restor/Backup



Hình 6.4: Cửa sổ thao tác cho Backup Database

6.2.1. Full backup

Full backup là sao lưu một bản đầy đủ của CSDL trên các phương tiện lưu trữ. Quá trình full backup có thể tiến hành mà không cần offline CSDL, nhưng quá trình này lại chiếm một

lượng lớn tài nguyên hệ thống và có thể ảnh hưởng nghiêm trọng tới thời gian đáp ứng các yêu cầu của hệ thống.

6.2.2. Differential Backup

Differential backup: được xây dựng nhằm làm giảm thời gian cần thiết để thực hiện quá trình full backup. Differential backup chỉ sao lưu những thay đổi trên dữ liệu kể từ lần full backup gần nhất. Trong những hệ thống CSDL lớn, quá trình differential backup sẽ sử dụng tài nguyên ít hơn rất nhiều so với quá trình full backup và có thể không ảnh hưởng đến hiệu suất của hệ thống.

Quá trình differential chỉ sao lưu những sự thay đổi của dữ liệu từ lần full backup gần nhất, do đó khi có sự cố với CSDL nếu không có bản sao lưu của quá trình full backup thì bản sao lưu của quá trình differential backup sẽ trở nên vô nghĩa.

6.2.3. Transaction log backup

Quá trình full backup và differential backup chiếm nhiều tài nguyên hệ thống và ảnh hưởng đến hiệu suất làm việc hệ thống nên thường được thực hiện vào sau giờ làm việc. Tuy nhiên điều này có thể dẫn đến các mất mát dữ liệu trong một ng ày làm việc nếu CSDL có sự cố trước khi quá trình sao lưu diễn ra. Transaction log backup là một giải pháp nhằm giảm thiểu tối đa lượng dữ liệu có thể mất khi có sự cố CSDL.

Trong quá trình hoạt động, SQL Server sử dụng transaction log để theo dõi tất cả các thay đổi trên CSDL. *Log* bảo đảm CSDL có thể phục hồi sau những sự cố đột xuất và cũng đảm bảo người dùng có thể quay ngược các kết quả trong các giao tác CSDL. Các giao tác chưa hoàn thành được lưu trong trước khi được lưu vĩnh viễn trong CSDL.

Transaction log backup sao lưu transaction log của CSDL vào thiết bị lưu trữ. Mỗi khi transaction log được sao lưu, SQL Server bỏ đi các transaction đã thực hiện thành công (committed tracsaction) và ghi các transaction vào phương tiện sao lưu. Transaction log backup sử dụng tài nguyên hệ thống ít hơn rất nhiều so với full backup và differential backup, do đó có thể sử dụng transaction log backup bất kỳ thời gian nào mà không sợ ảnh hưởng đến hiệu suất hệ thống.

6.2.4. Lập lịch backup dữ liệu với SQL Server Agent

SQL Server Agent giúp việc backup được tự động và dễ dàng hơn.

SQL Server Agent là công cụ hỗ trợ người dùng phụ thuộc vào edition mà người dùng lựa chọn để cài đặt. V

ới những edition không hỗ trợ, tính năng này không thể thực hiện được.

Nếu chọn edtion có hỗ trợ, các bước thực hiện lập lịch gồm:

i. Khởi động Jobs trong SQL Server Agent
ii. Chọn New Job và nhập tên Backup vào sau đó chọn Steps trong menu bên trái Nhập vào commands
BACKUP DATABASE ATM TO DISK = 'D:"backup-sql"ten-file-backup.bak' WITH INIT.

Management tools

Feature	Enterprise	Standard	Web	Express with Advanced Services	Express
SQL Management Objects (SMO)	Yes	Yes	Yes	Yes	Yes
SQL Configuration Manager	Yes	Yes	Yes	Yes	Yes
SQL CMD (Command Prompt tool)	Yes	Yes	Yes	Yes	Yes
Distributed Replay - Admin Tool	Yes	Yes	Yes	Yes	No
Distribute Replay - Client	Yes	Yes	Yes	No	No
Distributed Replay - Controller	Yes (Up to 16 clients)	Yes (1 client)	Yes (1 client)	No	No
SQL Profiler	Yes	Yes	No [1]	No [1]	No [1]
SQL Server Agent	Yes	Yes	Yes	No	No
Microsoft System Center Operations Manager Management Pack	Yes	Yes	Yes	No	No
Database Tuning Advisor (DTA)	Yes	Yes [2]	Yes [2]	No	No

Hình 6.5: Các tool hỗ trợ theo edition

6.3. Khôi phục dữ liệu (Restore Database)

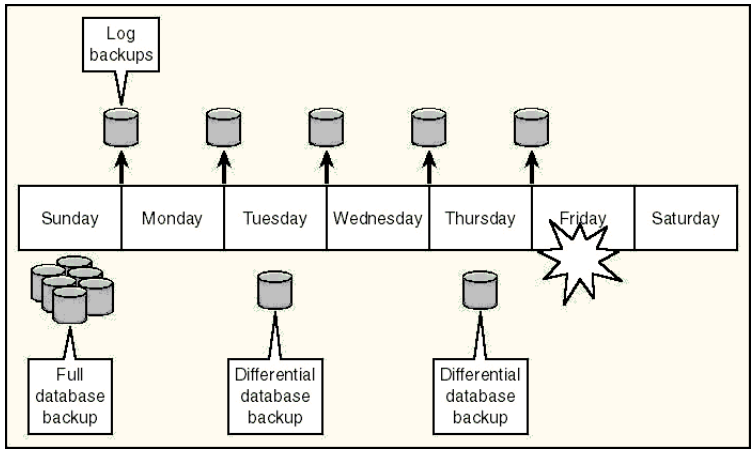
Khôi phục là quá trình sao chép dữ liệu đã sao lưu và đưa các giao dịch được ghi lại vào dữ liệu của MS SQL Server. Hiểu đơn giản, đây là quá trình lấy file sao lưu và đưa nó trở lại CSDL.

6.3.1. Các chế độ khôi phục dữ liệu

- **Chế độ Full Recovery**: Đây là chế độ cho phép phục hồi dữ liệu với ít rủi ro nhất. Nếu một database ở trong chế độ này thì tất cả các hoạt động không chỉ insert, update, delete mà kể cả insert bằng **Bulk Insert**, hay **bcp** đều được log vào file transaction log. Khi có sự cố thì ta có thể phục hồi lại dữ liệu ngược trở lại tới một thời điểm trong quá khứ. Khi file dữ liệu bị hư nếu ta có thể sao lưu được file transaction log thì ta có thể phục hồi CSDL đến thời điểm transaction gần nhất đã được xác nhận (commited).
- **Chế độ Bulk-Logged Recovery**: Ở chế độ này các hoạt động mang tính hàng loạt như Bulk Insert, bcp, Create Index, WriteText, UpdateText chỉ được log minimum vào File Transaction Log đủ để cho biết là các hoạt động này có diễn ra mà không log toàn bộ chi tiết như trong chế độ Full Recovery. Các hoạt động khác như Insert, Update, Delete vẫn được log đầy đủ để dùng cho việc phục hồi sau này.
- **Chế độ Simple Recovery**: Ở chế độ này thì File Transaction Log được cắt xén thường xuyên và không cần sao lưu. Với chế độ này bạn chỉ có thể phục hồi tới thời điểm backup gần nhất mà không thể phục hồi tới một thời điểm trong quá khứ.

Muốn biết CSDL của bạn đang ở mode nào bạn có thể **Right-click lên một database nào đó** trong SQL Server Management Studio chọn **Properties->Options->Recovery model**

6.3.2. Sao lưu và khôi phục



Hình 6.4: Minh họa backup dữ liệu

Trong ví dụ ở hình trên, cơ sở dữ liệu được lập lịch ở chế độ *Full Database Backup* vào ngày Chủ Nhật và *Differential Backup* vào các ngày thứ Ba và Thứ Năm. *Transaction Log Backup* được lập lịch hằng ngày. Vào một ngày Thứ Sáu "đen tối" một sự cố xảy ra đó là đĩa chứa file dữ liệu của CSDL bị hỏng và bạn là một quản trị CSDL, bạn được yêu cầu phải phục hồi dữ liệu và đưa CSDL hoạt động trở lại bình thường. Bạn phải làm sao?

Trước hết bạn phải sao lưu ngay File Transaction Log (trong ví dụ này File Transaction Log được chứa trong một đĩa khác với đĩa chứa File dữ liệu nên không bị hỏng và vẫn còn hoạt động). Người ta còn gọi file sao lưu trong trường hợp này là " the tail of the log" (phần đuôi). Nếu Log File được chứa trên cùng một đĩa với File dữ liệu thì bạn có thể sẽ không sao lưu được "phần đuôi" và như vậy bạn phải dùng đến file sao lưu log gần nhất. Khi sao lưu "phần đuôi" này bạn cần phải dùng tuỳ chọn **NO_TRUNCATE** bởi vì thông thường các Transaction Log Backup sẽ cắt xén những phần không cần dùng đến trong transaction log file, đó là những transaction đã được chấp nhận và đã được viết vào CSDL (còn gọi là inactive portion of the transaction log) để giảm kích thước của log file. Tuy nhiên khi sao lưu phần đuôi không được cắt xén để đảm bảo tính nhất quán (consistent) của database.

Kế đến bạn phải khôi phục CSDL (restore database) từ File Full Backup của ngày Chủ Nhật. Nó sẽ làm 2 chuyện: copy dữ liệu, log, index… từ đĩa sao lưu vào các file dữ liệu và sau đó sẽ lần lượt thực thi các transaction trong transaction log. Lưu ý ta phải dùng tuỳ chọn WITH NORECOVERY trong trường hợp này (tức là tuỳ chọn thứ 2 "Leave database nonoperational but able to restore additional transaction logs" trong SQL Server Management Studio). Nghĩa là các transaction chưa hoàn tất (incomplete transaction) sẽ không được roll back. Như vậy CDSL lúc này sẽ ở trong tình trạng inconsistent và không thể dùng được. Nếu ta chọn WITH RECOVERY (hay "Leave database operational. No additional transaction logs can be restored " trong SQL Server Management Studio) thì các incomplete transaction sẽ

được roll back và CSDL ở trạng thái consistent nhưng ta không thể nào khôi phục các transaction log backup được nữa.

Tiếp theo bạn phải khôi phục Differential Backup của ngày Thứ Năm. Sau đó lần lượt khôi phục các Transaction Log Backup kể từ sau lần Differential Backup cuối cùng nghĩa là restore Transaction Log Backup của ngày Thứ Năm và "Phần Đuôi". Như vậy ta có thể phục hồi data trở về trạng thái trước khi biến cố xảy ra. Quá trình này gọi là phục hồi CSDL (Database Recovery).

6.3.3. Thao tác khôi phục dữ liệu

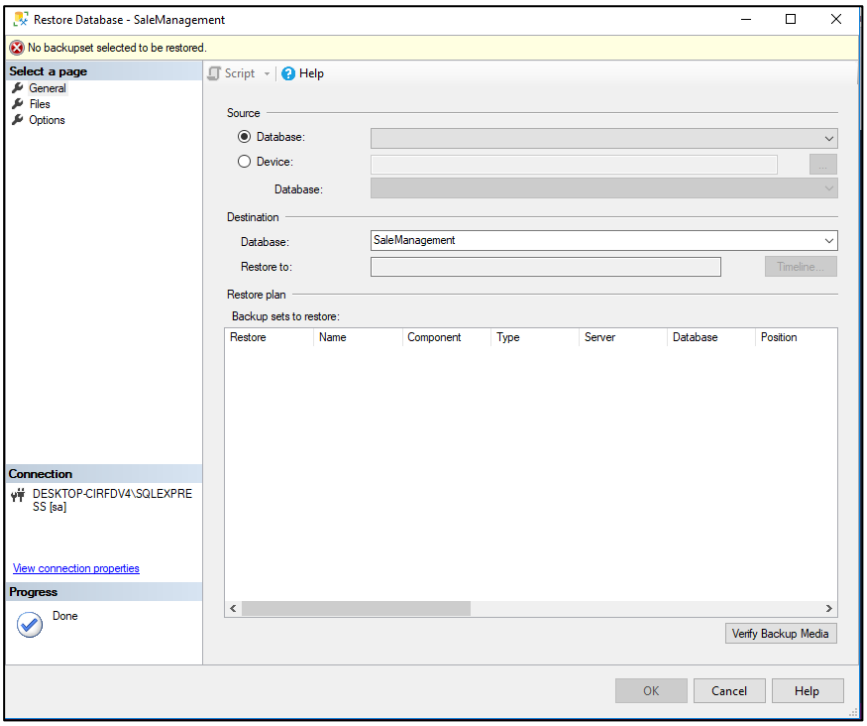
Khôi phục CSDL có thể được thực hiện bằng 2 cách:

- **Cách 1: Sử dụng T-SQL**

Cú pháp dưới đây được dùng để khôi phục CSDL.

```
Restore database <Your database name>
from disk = '<Backup file location + file name>'
```

- **Cách 2: Sử dụng SQL Server Management Studio SSMS**
  - + **Bước 1**: Kết nối tới CSDL (ví dụ SaleManagement) và Right click ➔ **Tasks** ➔ chọn **Restore** ➔ **Database** (xem hình 6.3)
  - + **Bước 2**: Chọn **Device** và chọn đường dẫn để mở tập tin sao lưu như trong hình dưới đây.



Hình 6.5: Cửa sổ thao tác với Restore Database

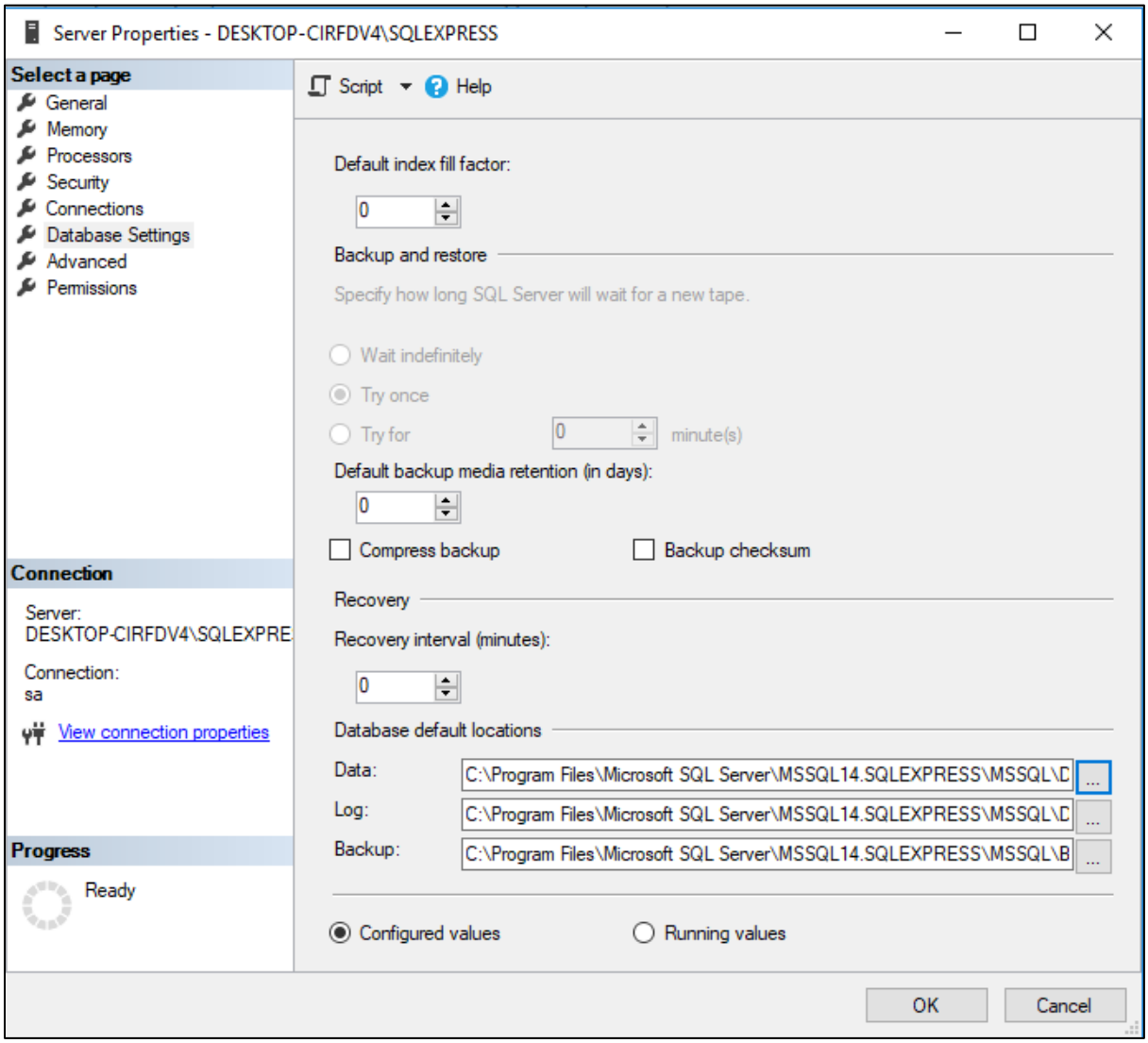
  - + **Bước 3**: Click **OK** và màn hình dưới đây sẽ hiện ra.
  - + **Bước 4**: Chọn **Files** ở góc bên trái màn hình, hộp thoại dưới đây sẽ hiện ra.

+ **Bước 5**: Chọn **Options** ở góc trái và click **OK** để bắt đầu khôi phục CSDL *TestDB* như trong hình dưới đây.

6.4. Sao chép dữ liệu

Khi tạo mới một Database, mặc định 2 file dữ liệu của database (*.mdf, và cả file log *.ldf) sẽ được lưu trữ ở thư mục DATA trong thư mục SQL Server đã cài đặt (thường ở ổ đĩa C).

Có thể thay đổi thư mục lưu trữ dữ liệu của database với Database Setting.



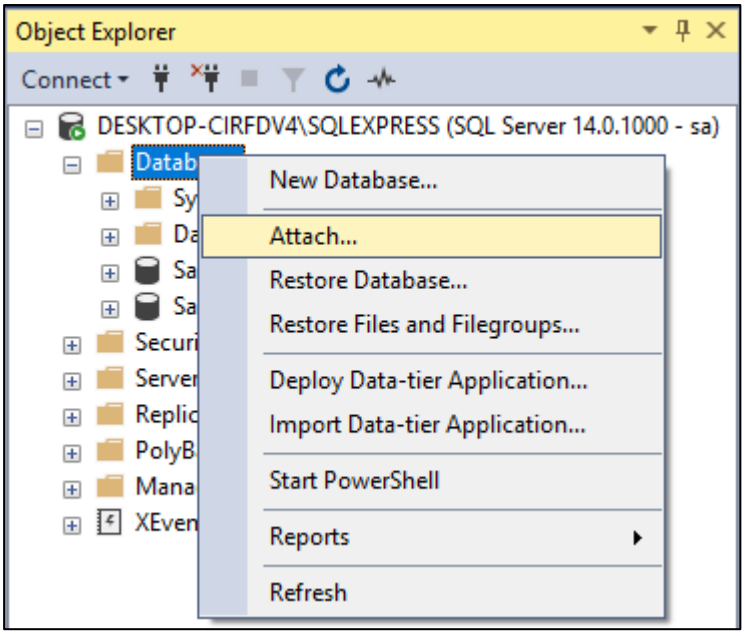
Hình 6.6: Cửa sổ tùy chỉnh cho Database

Các file dữ liệu database được quản lý bởi SQL Server, vì vậy ta không được quyền thao tác trên các file dữ liệu này nếu SQL Server đang hoạt động.

Có 2 cách cho phép sao chép file dữ liệu database gồm:

- Stop SQL Server
- Ngắt sự kiểm soát trên file dữ liệu database mà ta muốn sao chép bằng chức năng Detach (xem hình 6.3)

Sau khi đã sao chép, sử dụng chức năng Attach để cho phép thao tác với database này trong cửa sổ SSMS ➔ chọn file dữ liệu của database muốn Attach (lưu ý, cả file log).



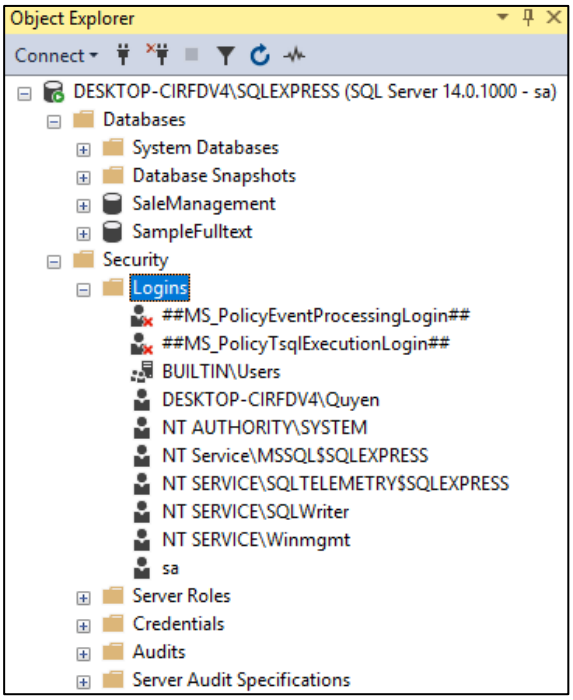
Hình 6.6: Giao diện khởi động chức năng Attach

6.5. Bảo mật dữ liệu

6.5.1. Login và User

SQL Server chỉ cho phép truy cập vào hệ thống thông qua những login.

SSMS cho phép xem tất cả các login trong node Security.



Hình 6.7: Security Node trong SSMS

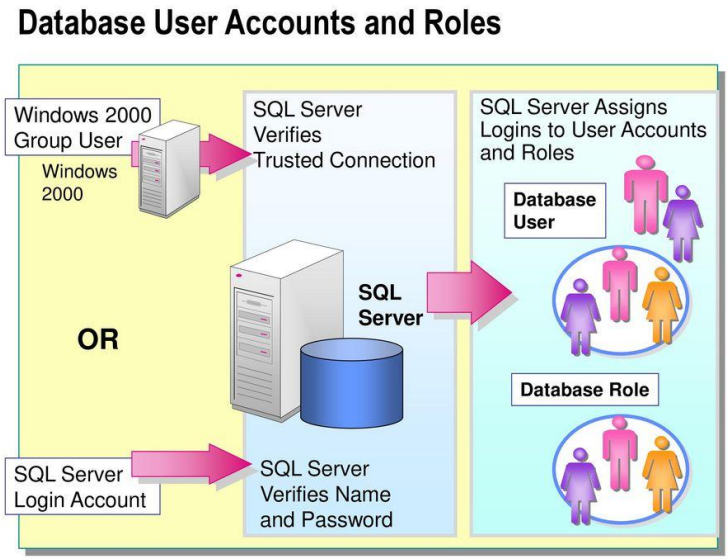
Những Login này chỉ có quyền truy cập vào Server chứ chưa thể vào được database. Với mỗi cơ sở dữ liệu lại cho phép một danh sách những user, các user này luôn luôn được gán với một login ở mức độ Server.

Nếu đăng nhập vào SQL Server qua login này, bạn sẽ có quyền truy cập vào cơ sở dữ liệu theo quyền hạn mà người dùng tương ứng với nó được cung cấp. Ví dụ, có một login có tên là l_spktquyen trong SQL Server. Ở trong cơ sở dữ liệu SaleMangement, có một user là U_BMCNTT và được gắn với login có tên l_spktquyen với quyền read only. Ở trong cơ sở dữ liệu SampleFulltext, có user U_BMCNTT cũng được gắn cùng với login l_spktquyen với quyền Read Write. Khi truy cập vào SQL Server bằng login l_spktquyen, thì chỉ có quyền đọc trên SaleManagement nhưng bạn lại có thể đọc và ghi trên SampleFulltext.

6.5.2. Authentication

SQL Server xác thực những đăng nhập với 2 cơ chế, đó là SQL Server Authentication và Windows Authentication.

Điều này bắt nguồn khi tạo những login ở trong SQL Server, khi bạn tạo mới một login, SQL Server sẽ đưa ra yêu cầu bạn phải chọn một trong hai cơ chế xác thực ở trên.



Hình 6.8: User Account

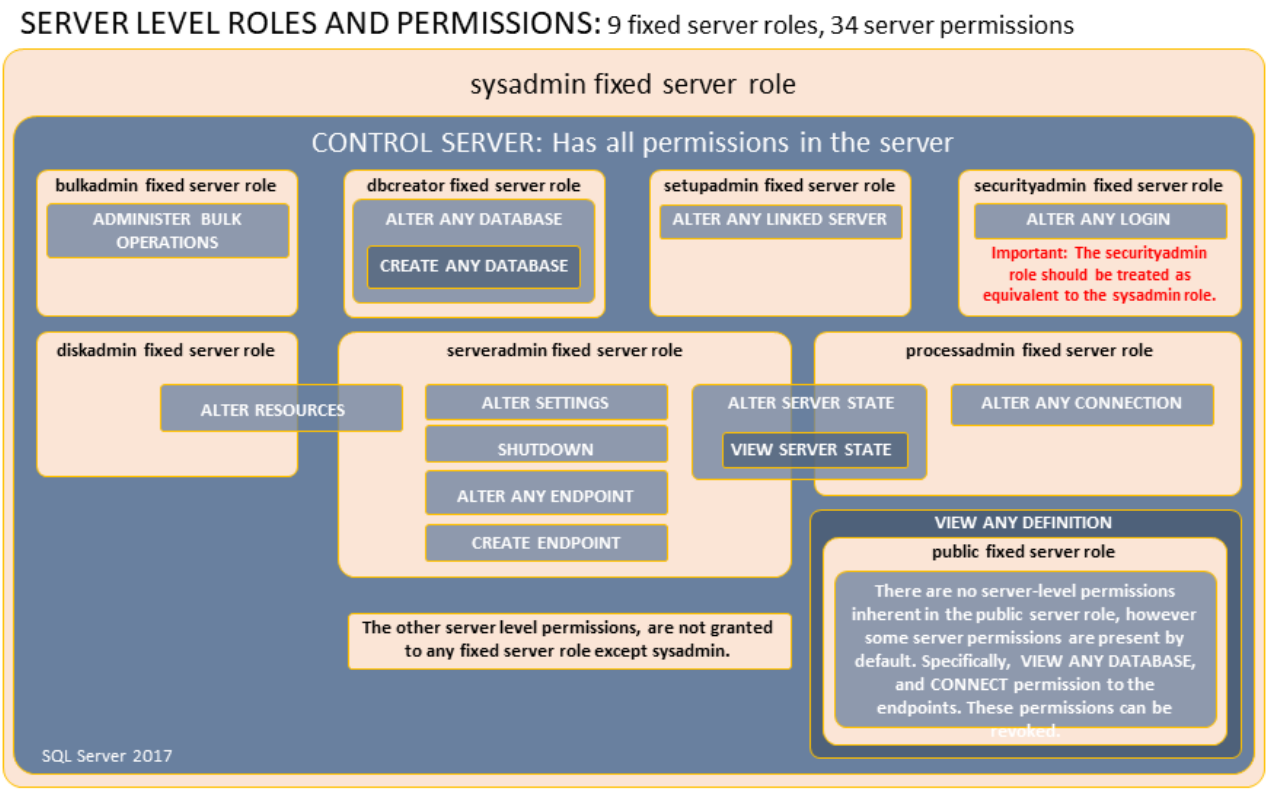
- SQL Server Authentication: nghĩa là cần phải cung cấp tên đăng nhập và mật khẩu và cả 2 đều được lưu trong SQL Server. Authentication này được khuyến cáo sử dụng nếu user đang làm việc trong workgroup, hoặc những Internet application
- Windows Authentication: nghĩa là chỉ cần cung cấp một tài khoản Windows Account và sau đó SQL Server chỉ lưu trên tài khoản trong danh sách login mà thôi. Authentication này được khuyến cáo sử dụng khi application & database trên cùng một máy, hoặc khi đang sử dụng SQL Server Express hoặc Local DB, hoặc có một server Domain Controller chịu trách nhiệm quản lý bảo mật mạng.

6.5.3. Server Role

Role là một tập hợp nhóm những quyền và được đại diện bằng một tên thể có thể dễ dàng trong việc quản lý ở mức độ Server, mà khi các login được cấp chúng sẽ có thể làm một số thao tác ở mức độ Server.

Những login khi được tạo thì role mặc định là public. Role này thực chất không có bất kỳ quyền nào cả ngoại trừ quyền truy cập vào Server. Thường thường chỉ có Database Administrator mới có quyền sysadmin, còn với những developer thì chỉ là role public và thi thoảng bổ sung thêm khi có thêm những tác vụ liên quan.

- Fixed Server Role: có một tập các quyền và phạm vi truy cập cố định vào server
- Fixed Database Role: tác dụng đối với database



Hình 6.9: Server Role and Permission

PHỤ LỤC

I. Các hàm chuyển đổi (Conversion Functions)

Sử dụng 2 hàm: CAST và CONVERT để chuyển kiểu từ kiểu dữ liệu này sang kiểu dữ liệu khác. Có 2 loại chuyển kiểu: ngầm định (implicit) và tường minh (explicit).

- Chuyển đổi ngầm định: xảy ra tự động, kiểu dữ liệu thấp hơn sẽ được chuyển đổi trước thành kiểu dữ liệu của kiểu cao hơn. Nếu việc chuyển đổi không hỗ trợ chuyển đổi ngầm định thì sẽ xảy ra lỗi.
- Chuyển đổi tường minh: sử dụng hàm CAST hoặc CONVERT.

Cú pháp hàm:

```
CAST(value as datatype)
CONVERT(datatype, value, style)
```

Trong đó:

`value`: giá trị muốn chuyển đổi

`datatype`: kiểu dữ liệu mà value muốn đổi.

`style`: định dạng mà value sẽ được hiển thị.

Vài style phổ biến cho dữ liệu kiểu datetime như: 101 (American), 103 (British/French), 105 (Italian)

Định dạng năm (yy)	Định dạng năm (yyyy)	Hiển thị dữ liệu
0	100	mon dd yyyy hh:miAM/PM (Default)
1	101	mm/dd/yy (US standard) hoặc mm/dd/yyyy
2	102	yy.mm.dd (ANSI standard)
3	103	dd/mm/yy (British/French standard) hoặc dd/mm/yyyy
4	104	dd.mm.yy (German standard) hoặc dd.mm.yyyy
5	105	dd-mm-yy (Italian standard) hoặc dd-mm-yyyy
6	106	dd mon yy
7	107	Mon dd, yy
8	108	hh:mi:ss
9	109	mon dd yyyy hh:mi:ss:mmmAM/PM
10	110	mm-dd-yy (USA standard)
11	111	yy/mm/dd (Japan standard)
12	112	yymmdd (ISO standard)

13	113	dd mon yyyy hh:mi:ss:mmm (Europe standard - 24 hour clock)
14	114	hh:mi:ss:mmm (24 hour clock)
20	120	yyyy-mm-dd hh:mi:ss (ODBC canonical - 24 hour clock)
21	121	yyyy-mm-dd hh:mi:ss:mmm (ODBC canonical - 24 hour clock)
	126	yyyy-mm-ddThh:mi:ss:mmm (ISO8601 standard)
	127	yyyy-mm-ddThh:mi:ss:mmmZ (ISO8601 standard)
	130	dd mon yyyy hh:mi:ss:mmmAM/PM (Hijri standard)
	131	dd/mm/yy hh:mi:ss:mmmAM/PM (Hijri standard)

Ví dụ:

```
print 'Today is ' + CAST(GETDATE() as varchar(30))
print 'Today is ' + CONVERT(VARCHAR, GETDATE(), 101)
print 'Today is ' + CONVERT(date, '25/09/2019', 103)
```

Kết quả:

```
Today is Aug 16 2019  2:39PM
Today is 08/16/2019
```

II. Hàm định dạng FORMAT

```
FORMAT ( value, format [, culture ] )
```

Trong đó:

`value`: giá trị muốn chuyển đổi

`datatype`: kiểu dữ liệu mà value muốn đổi

`style`: định dạng mà value sẽ được hiển thị

a) Với dữ liệu DATE

i. Khi nhập dữ liệu

Cho phép thiết lập trật tự Day – Month – Year khi nhập (insert) dữ liệu thông qua câu lệnh Insert hoặc Update

```
Set    DateFormat    {format}
Format nhận một trong các giá trị: mdy, dmy, ymd, ydm, myd and dym.
```

ii. Khi hiển thị dữ liệu

Format có thể nhận giá trị 'D' (show dạng chữ) hoặc 'd' (show dạng số).

Ví dụ:

```
DECLARE @d DATETIME = '10/01/2011';
SELECT FORMAT ( @d, 'd', 'en-US' ) AS 'US English Result' ,
       FORMAT ( @d, 'd', 'en-gb' ) AS 'Great Britain English Result' ,
       FORMAT ( @d, 'd', 'de-de' ) AS 'German Result' ,
       FORMAT ( @d, 'd', 'zh-cn' ) AS 'Simplified Chinese (PRC) Result';
```

```
DECLARE @d DATETIME = '10/01/2011';
```

US English Result	Great Britain English Result	German Result	Simplified Chinese (PRC) Result
10/1/2011	01/10/2011	01.10.2011	2011/10/1

```
SELECT FORMAT ( @d, 'D', 'en-US' ) AS 'US English Result' ,
       FORMAT ( @d, 'D', 'en-gb' ) AS 'Great Britain English Result' ,
       FORMAT ( @d, 'D', 'de-de' ) AS 'German Result' ,
       FORMAT ( @d, 'D', 'zh-cn' ) AS 'Simplified Chinese (PRC) Result';
```

US English Result	Great Britain English Result	German Result	Chinese (Simplified PRC) Result
Saturday, October 1, 2011	01 October 2011	Samstag, 1. Oktober 2011	2011年10月1日

b) Với dữ liệu giờ TIME

Trả về time – dùng dấu '.' hoặc dấu ':' để phân cách giữa hour và minute

```
SELECT FORMAT(cast('07:35' as time), N'hh.mm'),   --> returns NULL
       FORMAT(cast('07:35' as time), N'hh:mm');   --> returns NULL

--Format returns a formatted string because the . and : are escaped.
SELECT FORMAT(cast('07:35' as time), N'hh\.mm'),  --> returns 07.35
       FORMAT(cast('07:35' as time), N'hh\:mm');  --> returns 07:35

--Format returns a formatted current time with AM or PM specified
SELECT FORMAT(SYSDATETIME(), N'hh:mm tt'), -- returns 03:46 PM
       FORMAT(SYSDATETIME(), N'hh:mm t'); -- returns 03:46 P

--Format returns the specified time, displaying AM
select FORMAT(CAST('2018-01-01 01:00' AS datetime2), N'hh:mm tt'), -- returns 01:00 AM
       FORMAT(CAST('2018-01-01 01:00' AS datetime2), N'hh:mm t')  -- returns 01:00 A

--Format returns the specified time, displaying PM
select FORMAT(CAST('2018-01-01 14:00' AS datetime2), N'hh:mm tt'), -- returns 02:00 PM
       FORMAT(CAST('2018-01-01 14:00' AS datetime2), N'hh:mm t') -- returns 02:00 P

--Format returns the specified time in 24h format
select FORMAT(CAST('2018-01-01 14:00' AS datetime2), N'HH:mm') -- returns 14:00
```

c) Với dữ liệu dạng số (Money, Int, Decimal, SmallInt, ….)

i. Các chuẩn format cho các datatype dạng số (Money, Int, Decimal, …)

Format specifier	**Name**	**Description**	**Examples**
"C" or "c"	Currency	Hỗ trợ cho tất cả các kiểu dữ liệu - kết quả ở dạng tiền tệ	123.456 ("C", en-US) -> $123.46 123.456 ("C", fr-FR) -> 123,46 € 123.456 ("C", ja-JP) -> ¥123 -123.456 ("C3", en-US) -> ($123.456)

			-123.456 ("C3", fr-FR) -> -123,456 € -123.456 ("C3", ja-JP) -> -¥123.456
"D" or "d"	Decimal	Chỉ hỗ trợ cho kiểu số - kết quả dạng số nguyên	1234 ("D") -> 1234 -1234 ("D6") -> -001234
"E" or "e"	Exponential (scientific)	Kiểu dữ liệu khoa học dấu chấm động	1052.0329112756 ("E", en-US) -> 1.052033E+003 1052.0329112756 ("e", fr-FR) -> 1,052033e+003 -1052.0329112756 ("e2", en-US) -> -1.05e+003 -1052.0329112756 ("E2", fr-FR) -> -1,05E+003
"F" or "f"	Fixed-point	Chỉ hỗ trợ cho số - kết quả là số nguyên hoặc thập phân tĩnh	1234.567 ("F", en-US) -> 1234.57 1234.567 ("F", de-DE) -> 1234,57 1234 ("F1", en-US) -> 1234.0 1234 ("F1", de-DE) -> 1234,0 -1234.56 ("F4", en-US) -> -1234.5600 -1234.56 ("F4", de-DE) -> -1234,5600
"G" or "g"	General	Cho cả dấu chấm tĩnh và động	-123.456 ("G", en-US) -> -123.456 -123.456 ("G", sv-SE) -> -123,456 123.4546 ("G4", en-US) -> 123.5 123.4546 ("G4", sv-SE) -> 123,5 -1.234567890e-25 ("G", en-US) -> -1.23456789E-25

			-1.234567890e-25 ("G", sv-SE) -> -1,23456789E-25
"N" or "n"	Number	Số nguyên và thực có dấu chấm thập phân và nhóm các chữ số	1234.567 ("N", en-US) -> 1,234.57 1234.567 ("N", ru-RU) -> 1 234,57 1234 ("N1", en-US) -> 1,234.0 1234 ("N1", ru-RU) -> 1 234,0 -1234.56 ("N3", en-US) -> -1,234.560 -1234.56 ("N3", ru-RU) -> -1 234,560
"P" or "p"	Percent	Phần trăm %	1 ("P", en-US) -> 100.00 % 1 ("P", fr-FR) -> 100,00 % -0.39678 ("P1", en-US) -> -39.7 % -0.39678 ("P1", fr-FR) -> -39,7 %
"R" or "r"	Round-trip	Chuỗi từ đầu vào là số	123456789.12345678 ("R") -> 123456789.12345678 -1234567890.12345678 ("R") -> -1234567890.1234567
"X" or "x"	Hexadecimal	Số ở hệ hexa	255 ("X") -> FF -1 ("x") -> ff 255 ("x4") -> 00ff -1 ("X4") -> 00FF
Any other single character	Unknown specifier	Result: Throws a FormatException at run time.	

ii. Các ký hiệu đặc tả (Specifier) cho dạng số, hoặc kết hợp cả chuỗi ký tự, ví dụ như trường hợp đơn vị tiền tệ, đơn vị tính (cái, thùng….)

Format specifier	Name	Description	Examples
"0"	Zero placeholder	Mỗi ký tự '0' đại diện cho một chữ số trong chuỗi kết quả ➔ tự thêm 0 vào nếu không đủ độ dài (sau khi đã làm tròn)	1234.5678 ("00000") -> 01235 0.45678 ("0.00", en-US) -> 0.46 0.45678 ("0.00", fr-FR) -> 0,46
"#"	Digit placeholder	Mỗi ký tự '#' đại diện cho một chữ số trong chuỗi kết quả ➔ nếu độ dài thiếu thì chừa trống (sau khi đã làm tròn)	1234.5678 ("#####") -> 1235 0.45678 ("#.##", en-US) -> .46 0.45678 ("#.##", fr-FR) -> ,46
"."	Decimal point	Xác định vị trí dấu phân cách phần nguyên và phần phân trong chuỗi kết quả	0.45678 ("0.00", en-US) -> 0.46 0.45678 ("0.00", fr-FR) -> 0,46
","	Group separator and number scaling	Nhóm 3 chữ số để phân biệt hàng chục – đơn vị ….	Group separator specifier: 2147483647 ("##,#", en-US) -> 2,147,483,647 2147483647 ("##,#", es-ES) -> 2.147.483.647 Scaling specifier:

Format specifier	Name	Description	Examples
			2147483647 ("#,#,,", en-US) -> 2,147 2147483647 ("#,#,,", es-ES) -> 2.147
"%"	Percentage placeholder	Chèn vào ký hiệu phần trăm % sau khi nhân với 100 trong chuỗi kết quả	0.3697 ("%#0.00", en-US) -> %36.97 0.3697 ("%#0.00", el-GR) -> %36,97 0.3697 ("##.0 %", en-US) -> 37.0 % 0.3697 ("##.0 %", el-GR) -> 37,0 %
"‰"	Per mille placeholder	Chèn vào ký hiệu phần nghìn ‰ sau khi nhân với 1000 trong chuỗi kết quả	0.03697 ("#0.00‰", en-US) -> 36.97‰ 0.03697 ("#0.00‰", ru-RU) -> 36,97‰
"E0" "E+0" "E-0" "e0" "e+0" "e-0"	Exponential notation	Hiển thị chuỗi kết quả với dấu chấm động (e)	987654 ("#0.0e0") -> 98.8e4 1503.92311 ("0.0##e+00") -> 1.504e+03 1.8901385E-16 ("0.0e+00") -> 1.9e-16

Format specifier	Name	Description	Examples
"\"	Escape character	Hiển thị dạng kí tự chữ không dùng ý nghĩa thay thế (ví dụ muốn hiển thị '#' thì phải dùng '\#'	987654 ("\###00\#") -> #987654#
'string' "string"	Literal string delimiter	Giữ nguyên chuỗi	68 ("# 'degrees'") -> 68 degrees 68 ("#' degrees'") -> 68 degrees
;	Section separator	Phân tách giá trị phần nguyên và phần lẻ	12.345 ("#0.0#;(#0.0#);-\0-") -> 12.35 0 ("#0.0#;(#0.0#);-\0-") -> -0- -12.345 ("#0.0#;(#0.0#);-\0-") -> (12.35) 12.345 ("#0.0#;(#0.0#)") -> 12.35 0 ("#0.0#;(#0.0#)") -> 0.0 -12.345 ("#0.0#;(#0.0#)") -> (12.35)
Other	All other characters	Còn lại	68 ("# °") -> 68 °

iii. Ví dụ

```
declare @tien decimal = 13240000
select format(@tien,'##,#\ VNĐ','es-ES') as 'Tiền lương có đơn vị VN'
```

Tiền lương có đơn vị theo định dạng do người dùng định nghĩa
13.240.000 VNĐ

```
declare @luong money=6700000
--mặc định đơn vị tiền tệ là $
select format(@luong,'C') as 'Tiền lương',
       format(@luong,'c') as 'Tiền lương',
```

```
        format(@luong,'C0') as 'Tiền lương có phần lẻ',
        format(@luong,'C6') as 'Tiền lương có phần lẻ'
--chỉ định đơn vị tiền tệ
select format(@luong,'C0','fr-FR') as 'Tiền lương đơn vị Franc €',
        format(@luong,'C0','en-gb') as 'Tiền lương đơn vị bảng Anh £',
        format(@luong,'C0','zh-cn') as 'Tiền lương đơn vị Nhân dân tệ ¥',
        format(@luong,'C0','th-TH') as 'Tiền lương đơn vị Thái lan Baht ฿',
        format(@luong,'C0','de-DE') as 'Tiền lương đơn vị Đức €',
        format(@luong,'C0','vn-VN') as 'Tiền lương đơn vị Việt Nam đ'
```

Tiền lương	Tiền lương	Tiền lương có phần lẻ	Tiền lương có phần lẻ
$6,700,000.00	$6,700,000.00	$6,700,000	$6,700,000.000000

Tiền lương đơn vị Franc €	Tiền lương đơn vị bảng Anh £	Tiền lương đơn vị Nhân dân tệ ¥	Tiền lương đơn vị Thái lan Baht ฿	Tiền lương đơn vị Đức €	Tiền lương đơn vị Việt Nam đ
6 700 000 €	£6,700,000	¥6,700,000	฿6,700,000	6.700.000 €	₫6,700,000

```
--hiển thị đơn vị tiền VNĐ
declare @tien decimal = 13240000
select format(@tien,'##,#\ VNĐ','es-ES') as 'Tiền lương có đơn vị VNĐ',
      format(@luong,'##,#\ VNĐ','es-ES') as 'Tiền lương đơn vị Việt Nam VNĐ',
      format(@tien,'C0','vn-VN') as 'Tiền lương có phần lẻ'
```

Tiền lương có đơn vị VNĐ	Tiền lương đơn vị Việt Nam VNĐ	Tiền lương có phần lẻ
13.240.000 VNĐ	6.700.000 VNĐ	₫13,240,000

```
--truy vấn dữ liệu từ column của table
select CustID,CustName,format(Balance,'C')
from tblCustomer

--bổ sung các chữ số 0 cho đủ độ dài định trước
declare @so int=20
select format(@so,'D8') --return 00000021
```

III. Các hàm logic (logic Functions)

Sử dụng hàm IFF để trả về 1 trong các kết quả dựa trên biểu thức logic.

Cú pháp hàm:

```
IIF(boolean expression, tvalue if true, fvalue if false)
```

Nếu biểu thức logic trả về TRUE thì hàm sẽ trả về tvalue, ngược lại hàm sẽ trả về fvalue.

Ví dụ:

```
select OrderID, QtyOrdered ,
   IIF(QtyOrdered > 20, N'Nhiều', N'Ít')
from tblOrderInvoiceDetail;
```

IV. Các hàm toán học (Math Functions)

Loại hàm	Tên hàm	Mô tả
Scientific và Trig	ACOS, ASIN, ATAN, ATN2, COS,COT, RADIANS, SIN, TAN	Các hàm lượng giác, toán học
	DEGREE	
	EXP, LOG, LOG10	
	PI	Trả về giá trị của PI
	POWER	Trả về giá trị hàm mũ của số
	SQUARE	Trả về bình phương của số
	SQRT	Trả về căn bậc hai
Hàm làm tròn	CEILING	Trả về giá trị nguyên nhỏ nhất mà lớn hơn hoặc bằng số
	FLOOR	Trả về giá trị nguyên lớn nhất mà nhỏ hơn hoặc bằng số
	ROUND	Trả về giá trị đã được làm tròn về một số nguyên với số lượng số vị trí thập phân.
Hàm dấu Sign	ABS	Trả về giá trị tuyệt đối của một số
	SIGN	Trả về dấu của một số
Hàm ngẫu nhiên	RAND	Trả về số ngẫu nhiên

V. Hàm chuỗi (String Funtions)

Loại hàm	Tên hàm	Mô tả
Vị trí (Position)	CHARINDEX	Xác định vị trí của 1 chuỗi trong chuỗi khác
	LEN	Trả về độ dài của chuỗi
	PATINDEX	Trả về vị trí của mẫu (pattern) trong chuỗi
Hàm chuyển đổi (Transformation)	LEFT/RIGHT	Trả về các kí tự tính từ vị trí bắt đầu bên trái/phải của một chuỗi
	LOWER/UPPER	Trả về chuỗi thường/hoa của chuỗi cho trước

Loại hàm	Tên hàm	Mô tả
	SUBSTRING	Trả về chuỗi bên trong chuỗi khác
	REPLACE	Tìm một hoặc nhiều kí tự trong chuỗi và thay chúng bằng các kí tự khác
	REPLICATE	Lặp lại một chuỗi với số lần chỉ định
	REVERSE	Đảo ngược các kí tự trong chuỗi
	SPACE	Trả về 1 chuỗi với số kí tự khoảng trống được chỉ định
	STR	Trả về 1 số là chuỗi
	STUFF	Xóa 1 phần của chuỗi và chèn phần khác vào chuỗi bắt đầu từ vị trí chỉ định
	LTRIM/RTRIM/TRIM	Loại bỏ các khoảng trống từ vị trí bắt đầu/kết thúc hoặc đầu/cuối của một chuỗi
Tập kí tự (Character Set)	ASCII	Trả về giá trị ASCII của kí tự
	CHAR/NCHAR/UNICODE	Trả về kí tự dựa trên mã ASCII
Soundex	DIFFERENCE	So sánh 2 giá trị soundex và trả về một giá trị nguyên
	SOUNDEX	Trả về chuỗi soundex
Nối chuỗi	CONCAT	Nối các chuỗi
Lấy thành phần chuỗi phân cách nhau bởi dấu '.'	PARSENAME	Lấy thành phần chuỗi cách nhau bởi dấu chấm (từ phải sang trái – thành phần đầu tiên được gán là 1)

VI. Hàm ngày tháng (Date Functions)

Loại hàm	Tên hàm	Mô tả
Ngày giờ hiện tại (Current Date and Time)	SYSDATETIME, SYSDATEIMEOFFSET SYNUTDATETIME CURRENT_TIMESTAMP	Xác định ngày giờ hệ thống,…
	GETDATE	Trả về ngày giờ hiện tại
Date Parts	DATENAME DATEPART	Trả về các phần của ngày

	DAY/MONTH/YEAR	Trả về ngày/tháng/năm
Difference and Modification	DATEDIFF	Trả về số năm/tháng/tuần/ngày khác nhau giữa 2 ngày
	DATEADD	Cộng năm/tháng/tuần/ngày vào 1 ngày

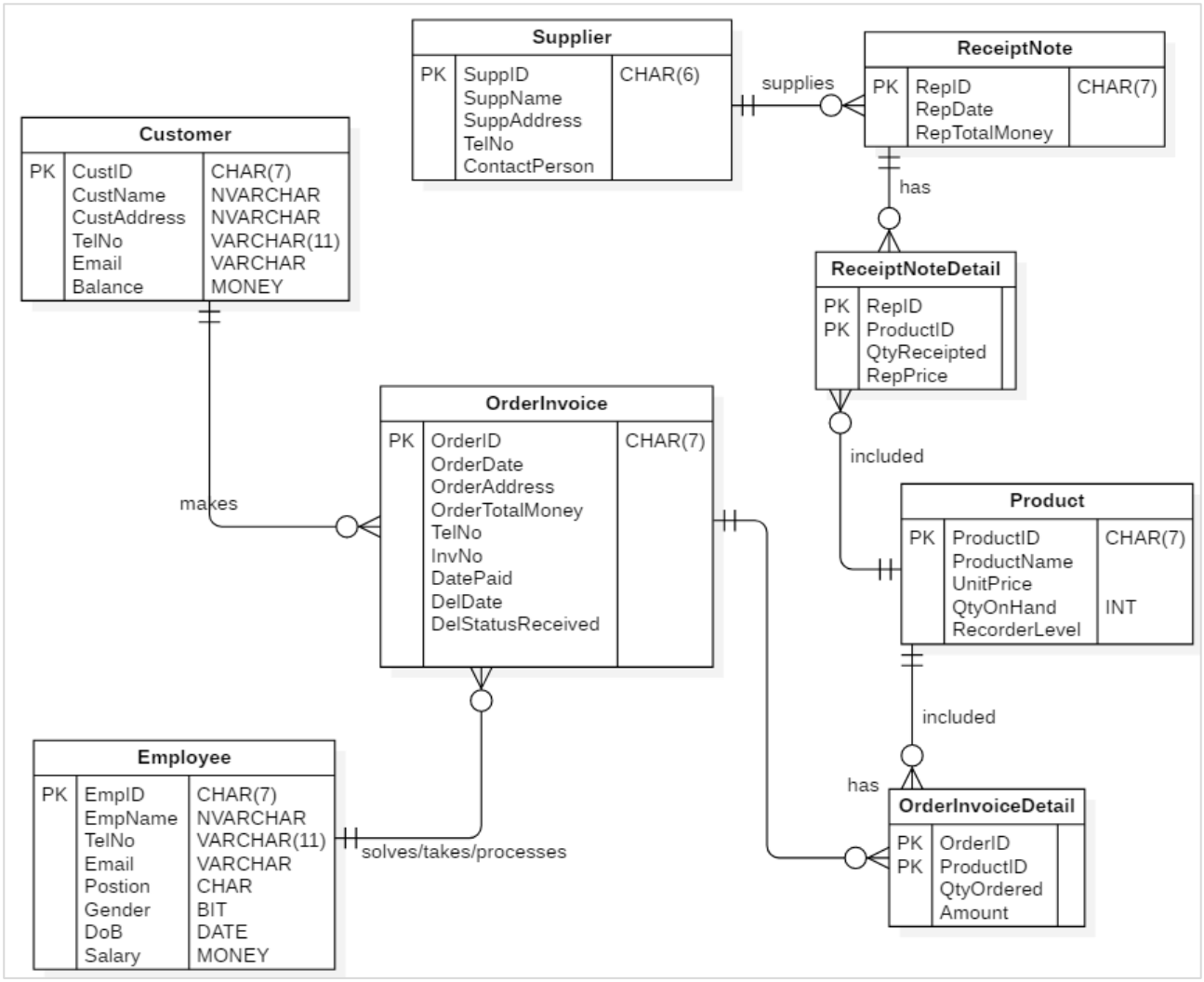
Danh mục hàm cụ thể:

No.	Mô tả
1	ADDDATE() Thêm ngày tháng
2	ADDTIME() Thêm thời gian
3	CONVERT_TZ() Chuyển đổi từ múi giờ này sang múi giờ khác
4	CURDATE() Trả về ngày hiện tại
5	CURRENT_DATE(), CURRENT_DATE Tương tự như hàm CURDATE()
6	CURRENT_TIME(), CURRENT_TIME Tương tự như hàm với CURTIME()
7	CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP Tương tự như hàm với NOW()
8	CURTIME() Trả về thời gian hiện tại
9	DATE_ADD() Thêm hai ngày
10	DATE_FORMAT() Định dạng ngày theo quy định
11	DATE_SUB() Trừ hai ngày
12	DATE() Trả về phần ngày của date hoặc biểu thức datetime
13	DATEDIFF() Trừ hai ngày
14	DAY() Tương tự như DAYOFMONTH()
15	DAYNAME() Trả về tên của ngày trong tuần
16	DAYOFMONTH() Trả về ngày trong tháng (1-31)
17	DAYOFWEEK()

	Trả về chỉ số weekday của đối số
18	DAYOFYEAR() Trả về ngày trong năm (1-366)
19	EXTRACT Trả về một phần của một ngày
20	FROM_DAYS() Chuyển đổi một số ngày cho một ngày
21	FROM_UNIXTIME() Định dạng ngày như timestamp trên UNIX
22	HOUR() Trả về giờ từ đối số truyền vào
23	LAST_DAY Trả về ngày cuối cùng của tháng
24	LOCALTIME(), LOCALTIME Tương tự như hàm với NOW()
25	LOCALTIMESTAMP, LOCALTIMESTAMP() Tương tự như hàm với NOW()
26	MAKEDATE() Tạo một ngày từ năm và ngày trong năm
27	MAKETIME MAKETIME()
28	MICROSECOND() Trả về microseconds từ đối số
29	MINUTE() Trả về phút từ đối số
30	MONTH() Trả về tháng từ ngày được truyền vào
31	MONTHNAME() Trả về tên của tháng
32	NOW() Trả về ngày và giờ hiện tại
33	PERIOD_ADD() Thêm một khoảng thời gian cho một tháng một năm
34	PERIOD_DIFF() Trả về số tháng giữa các khoảng thời gian
35	QUARTER() Trả về quý từ đối số ngày
36	SEC_TO_TIME() Chuyển đổi giây sang định dạng 'HH: MM: SS'
37	SECOND() Trả về giá trị giây (0-59)
38	STR_TO_DATE()

	Chuyển đổi một chuỗi thành một ngày
39	SUBDATE() Khi được gọi với ba đối số, tương tự như hàm DATE_SUB()
40	SUBTIME() Trừ lần
41	SYSDATE() Trả về thời gian mà hàm thực thi
42	TIME_FORMAT() Định dạng theo thời gian
43	TIME_TO_SEC() Trả về đối số được chuyển đổi thành giây
44	TIME() Trả về phần thời gian từ đối số được truyền vào
45	TIMEDIFF() Trừ đi thời gian
46	TIMESTAMP() Với một đối số duy nhất, hàm này trả về ngày hoặc biểu thức datetime. Với hai đối số, tổng của các đối số
47	TIMESTAMPADD() Thêm khoảng thời gian vào một biểu thức datetime
48	TIMESTAMPDIFF() Loại trừ khoảng thời gian từ biểu thức datetime
49	TO_DAYS() Trả về đối số ngày được chuyển thành ngày
50	UNIX_TIMESTAMP() Trả về dấu thời gian UNIX
51	UTC_DATE() Trả về ngày giờ UTC hiện tại
52	UTC_TIME() Trả về thời gian UTC hiện tại
53	UTC_TIMESTAMP() Trả về ngày tháng hiện tại của UTC
54	WEEK() Trả về số tuần
55	WEEKDAY() Trả về chỉ số ngày trong tuần
56	WEEKOFYEAR() Trả về tuần dương lịch của ngày (1-53)
57	YEAR() Trả về năm
58	YEARWEEK() Trả về năm và tuần

VII.ERD mẫu cho bài toán SaleManagement



TÀI LIỆU THAM KHẢO

1) Bài giảng Cơ sở dữ liệu 2 – tài liệu lưu hành nội bộ, Nguyễn Văn Lành
2) Thực Hành Lập Trình SQL, Trần Đức Quang, Nhà Xuất Bản Đại Học Quốc Gia TP Hồ Chí Minh, 2004.
3) Database Design – 2nd, Adrienne Watt and Nelson Eng.
4) Microsoft SQL Server 2012 BIBLE, Adam Jorgensen, Patrick LeBlanc, Jose Chinchilla, Jorge Segarra, Aaron Nelson, 2012.
5) Tutorial trên mạng: docs.microsoft.com, w3schools.com, sqlservertutorial.net, dofactory.com/sql, …